

Post-Quantum Anonymous Signatures from the Lattice Isomorphism Group Action

Chris van Noorden^{1,2} and Paola de Perthuis¹

¹ Centrum Wiskunde & Informatica (CWI), Nederland

² Universiteit Utrecht, Nederland

Abstract Post-quantum assumptions may not rely on the difficulty of finding secret subgroups as many classical schemes did. Instead, several assumptions make use of more general group actions, with the belief that quantum algorithms are not helpful in this less structured setting. Famously, some isogeny constructions use the action of an ideal class group on elliptic curves, but equivalence problems in error-correcting codes and lattices also exhibit such structures.

Previous works hence presented anonymity-preserving constructions in a generic group action framework; however, they were not general enough to encompass the group action underlying the *Lattice Isomorphism Problem* (LIP), for which the acting group is infinite (in fact, not even compact) and non-commutative.

We bridge this gap by, from zero-knowledge proofs of *OR* statements, building generic *blind signature* and *strong designated-verifier signature with non-delegatability* constructions from standard assumptions corresponding to a generalised group action inverse problem.

1 Introduction

Generalised cryptographic schemes built from group actions were first presented by Brassard and Yung in 1990 [BY91], and then by Couveignes in 1997 (published in the note [Cou06]) who formalised the notion of *Hard Homogeneous Spaces* (HSS), which generalised the discrete logarithm problem, corresponding to the action of a $\mathbb{Z}/n\mathbb{Z}$ group, for n a natural integer, in a unified framework with graph isomorphisms, or the action of ideal class groups over elliptic curves used in isogeny-based constructions.

Since then, group actions have been used in isogeny-based cryptography, [KTAT20] until group actions started appearing in code-based cryptology through new equivalence problems, stemming from [SS13], which in comparison to isogeny-based ones lead to more efficient and easier to implement schemes, though with the drawback of using non-commutative groups.

This lead to generic constructions for blind signature [HLM⁺25, DKQ⁺25] and strong designated-verifier signature [MTT25] primitives from generic group actions. However, these were not applicable to the one exhibited by the *Lattice Isomorphism Problem* (LIP) [DvW22], in which the infinite and non-commutative group of invertible integer matrices acts on quadratic forms.

Contributions. This work overcomes the aforementioned difficulty by using a more general group action fit to encompass the LIP as well as code or isogeny-based ones, in order to first build a zero-knowledge OR proof system, and then use it to build blind signatures and strong designated-verifier signatures with non-delegatability. The resulting group-action-based blind signatures not only allow lattice-based instantiations for the first time, but also improve on limitations of previous schemes in which the group required efficient square-root computations [DKQ⁺25] and new secret-key distributions on codes [KLP25, DKQ⁺25], or the use of strong assumptions such as an algebraic model and a non-standard one-more assumption [HLM⁺25] (though this last article proves unforgeability for more sessions than we do). Moreover, our strong designated-verifier signatures with non-delegatability improve on [MTT25] by granting code-equivalence and lattice-isomorphism-based instantiations for the first time, by allowing the group action to be non-abelian and removing the need for a quadratic twist.

Related Work. The current article extends upon previous research rooted in isogeny-based group actions, or techniques that used zero-knowledge proofs of disjunctions, called *OR proofs* to obtain anonymity properties.

OR Proofs. Proofs of knowledge for disjunctive statements have since [CDS94] found numerous applications beyond privacy-preserving multi-party computations: interesting examples include simulation-sound zero-knowledge proofs, first introduced by Sahai [Sah99] in relation to the concept of non-malleable proofs. Namely, simulation-sound zero-knowledge proofs are useful in contexts in which attackers may adaptively try to generate new proofs from old ones, as these constructions allow showing them simulated proofs generated without using the witness to be hidden, while preventing them from reusing them to generate new ones. Witness-indistinguishable OR proofs yield such constructions by proving that either one knows a witness corresponding to the original statement, or another witness for a hard problem, where this second branch will be used to generate simulated proofs.

This work presents two complementary usages of OR proofs in the context of post-quantum anonymity: in our blind signatures, the signer proves the knowledge of one out of two secret keys, which will grant some form of simulation soundness used to obtain one-more unforgeability; and in an opposite perspective in our strong designated-verifier signatures, OR proofs will allow the intended verifier to simulate signatures, thus ensuring that they can convince no one else that they were originally provided by the signer. These strategies are not new, but were previously not usable for general group actions including the lattice isomorphism one.

In 1994, [CDS94] showed a generic way to obtain witness-indistinguishable OR proofs of knowledge of one among two secret keys, given a public-coin honest-verifier zero-knowledge proof system and a secret sharing scheme adapted to its structure. This strategy was general enough to be readily applicable to post-quantum signature schemes, as for instance demonstrated by [AHJ21] in order to obtain blind signatures from lattice-based MLWE and MSIS assumptions.

In the context of group actions, this technique may prove less straightforward—or at least, less efficient—, namely because secret sharing schemes have to be adapted to a weaker structure which cannot benefit from the expressivity of $(\mathbb{Z}/n\mathbb{Z})$ -module operations. For instance, [KLLQ23] had to use multiplicative shares in $\{-1; 1\}$ instead of previous instantiations with Shamir secret sharing [Sha79] making use of finite fields.

Blind Signatures from Group Actions. In fact, CSI-Otter [KLLQ23] proposed a first construction of blind signatures (*i.e.* obtained by interacting with a signer who should not be able to recognise the origin session of the output signature) based on a non-discrete-logarithm group action, but it was tailored for isogenies and relied on the quadratic twists of elliptic curves, which was not transposable to other post-quantum families.

Other works [KLP25, DKQ⁺25, HLM⁺25] then recently proposed blind signatures from more generic group actions, removing the need for a quadratic twist and allowing the group to be non-commutative, hence also error-correcting-code instantiations; however, these were not yet applicable to the lattice isomorphism group action.

In more detail, [KLP25] overcame this difficulty by introducing a new code assumption, the MIMCE (in Definition 2.20), in which secret keys must be symmetric or antisymmetric. Then, [DKQ⁺25], also inspired by the construction of [KLLQ23], proposed another adaptation doubling its public key size and requiring the group to allow efficient square root computations, under the condition that it be safe to reuse group elements, also resulting in new error-correcting-code assumptions, the DmILCE and ILCE (Problems 2 and 3).

Tanuki [HLM⁺25] then proposed constructions with sequential and concurrent securities for polynomially many sessions, and as for the two previous schemes, a group action general enough to encompass both isogeny and code-based instantiations. However, they relied on an algebraic model and a non-standard one-more assumption. In fact the authors state in [HLM⁺25, Remark 3] that the open problem of a LIP-based instantiation of blind signatures would provide an interesting contribution to the field—which we now provide in this work. On the other hand, unlike [HLM⁺25], we prove unforgeability of our blind signatures for only logarithmically many sessions.

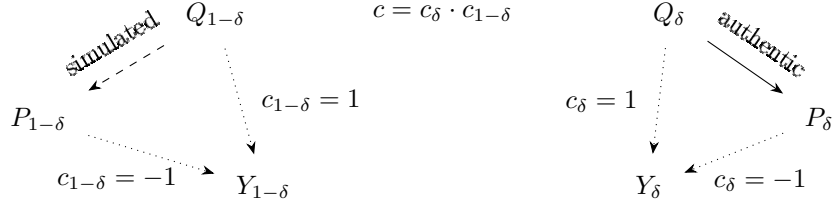
Strong Designated-Verifier Signatures with Non-Delegatability from Group Actions. While *designated-verifier signatures* ensure that only the intended verifier may be convinced of the signer’s voluntary issuance, *strong* designated-verifier signatures (SDVS) additionally require that no external observer may learn with respect to which signer identities signatures are stated, in order to enhance the scheme’s anonymity properties. Further, as SDVS were conceived for voting applications, *non-delegatability* ensures that, outside of the verifier, no one may construct a valid signature without the *full* signer secret key.

Strong designated-verifier signatures with non-delegatability (SDVSND) from group actions were recently proposed by [MTT25]. However, their construction

only applies to the abelian case and uses quadratic twist of elliptic curves, specific to isogeny-based instantiations.

Technical Overview.

OR Proofs. Our proofs of disjunctions are similar to the ones presented in [KLLQ23, DKQ⁺25], however, we do not use pairs of inverse group elements. Even though our challenge space is $\{-1; 1\}^\lambda$, for which we use multiplicative secret sharing in $\{-1; 1\}$, challenge components in $\{-1; 1\}$ will correspond to group exponents in $\{0; 1\}$: indeed, we observe that the elements in the secret sharing of c only determine the choice of one among two branches (linking the commitment either to Q or to P), and do not need to be understood literally as -1 and 1 . This allows us to remove both the use of a quadratic twist from elliptic curves and the non-standard assumptions with particular secret key distributions used in [KLP25, DKQ⁺25]. This approach is summarised in the following diagram,



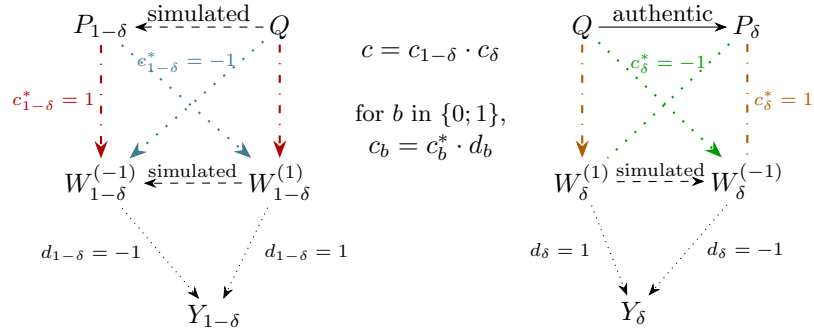
in which the prover demonstrates the knowledge of one (corresponding to the index δ) among two group elements, linking Q_0 to P_0 and Q_1 to P_1 . The $1-\delta$ branch is simulated by choosing its challenge beforehand and then sampling the corresponding commitment $Y_{1-\delta}$, for which the prover will know the linking group element. Then, as c is fixed by an honest verifier (or drawn from a hash function modelled as a random oracle), and this determines c_δ , the prover must demonstrate authentic knowledge of the group element linking P_δ to Q_δ . This is repeated λ times in parallel to obtain adequate soundness, yielding vectors $\mathbf{Y}_0, \mathbf{Y}_1$ of commitments and $\mathbf{c}_0, \mathbf{c}_1$ of challenges.

Blind Signatures. Our construction follows the [KLLQ23] general approach from OR proofs. Zero-knowledge proofs are inserted to allow the verification of the well-formedness of public keys and commitments, in order to prevent malicious attacks in which elements in the wrong orbits would be inserted in order to obtain distinguishers.

Given the element Q defined by the scheme and public keys (P_0, P_1) , a valid signature is a witness-indistinguishable proof knowledge of one of the two elements linking P_0 and P_1 to Q . The main challenge of blind signature constructions is then to enable the user to hide their intended message and commitment to random coins for the signature when communicating with the signer, and then to unblind them when releasing their final signature.

To this aim, the signer generates *two* OR proofs of knowledge of witness either for P_0 or P_1 , that are linked in the sense that using one branch (selected by the challenge) in one of them will imply using the other one in the other OR proof. The signer's first message consists of the respective commitments $\mathbf{W}_0^{(1)}$, $\mathbf{W}_1^{(1)}$, and $\mathbf{W}_0^{(-1)}$, $\mathbf{W}_1^{(-1)}$ of these (1) and (-1) sigma-protocol OR proofs (corresponding to the Y_0, Y_1 in the above diagram). Then, the user also draws random commitments, in a way to would allow her to simulate knowledge of group element linking $\mathbf{W}_0^{(1)}$ to $\mathbf{W}_0^{(-1)}$ for a challenge $\mathbf{d}_0$, and $\mathbf{W}_1^{(1)}$ to $\mathbf{W}_1^{(-1)}$ for a challenge $\mathbf{d}_1$. She then draws a challenge depending on her personal message and commitments, and sends a secret share $\mathbf{c}'$ of this challenge masked with $\mathbf{d}_0$ and $\mathbf{d}_1$ to the signer. The signer then uses $\mathbf{c}'$ as the challenge for both of her two linked sigma OR proofs using opposite challenge branches, and returns these proofs to the user. To build the final unblinded signature, the user multiplies the elements in her simulated proof of knowledge with the ones returned by the signer, and this composition yields the unmasked signature.

The reader will hereafter find a diagram summarising this overview, in which c_δ^* is determined as $c_\delta^* = c \cdot c_{1-\delta} \cdot d_\delta = c' \cdot c_{1-\delta}$.



Strong Designated-Verifier Signatures with Non-Delegatability. The authors of [MTT25] combine an OR proof with a Diffie-Hellman type of key exchange adapted to the group action setting; however, it relies on group commutativity, and is hence not applicable to code or lattice equivalence-based instantiations.

We circumvent this inconvenience by using a Key-Encapsulation Mechanism (KEM) to establish a common secret between the verifier and signer. Moreover, lattice-isomorphism group actions do not allow uniform sampling on the group, which is infinite. As a consequence, our OR proofs' distributions depend on the public keys in the statement, which obliges us to hide them in order to grant privacy of the signer's identities, allowing the signatures to be anonymous in the eyes of external observers. To this aim, we use the session-key provided by the KEM to encrypt sensitive elements in the OR proof, in a KEM-DEM paradigm [Sho01]. This requires a careful adaptation to still be able to extract signer secret-keys from non-verifier-generated signatures, in order to obtain non-delegatability.

Open Directions for Future Work. Quite interestingly, the distribution of our OR proofs depend on the relations chosen in their language: in other words, when we prove the knowledge of one out of two secret keys corresponding to two public keys, the distribution of the proof depends on the public keys. Comparing the lattice isomorphism group action to instantiations from previous works, this appears because of the infiniteness of the group, on which we do not use a uniform distribution.

1. One consequence of this observation is that removing the use of a key-encapsulation mechanism in our SDVSND does not seem straightforward.
2. Our construction verifies k -one-more unforgeability for logarithmically many sessions in the security parameter, and [BLL⁺21, DHP24, KLR24] show that opening λ concurrent sessions, for λ the security parameter, would allow successful ROS attacks to forge signatures. Future work will explore the obtention of unforgeability for polynomially many sessions, in the line of recent articles like [HLM⁺25].
3. We leave open an extension to partial blindness, which we believe should be obtained straightforwardly by the application of previous techniques.
4. For now we require freedom of the group action, which results in using automorphism-free lattices in LIP instantiations: an interesting future direction will be to extend our results to more general quadratic forms.

2 Preliminaries

Notations. Let $\mathbb{Z}$, $\mathbb{N}$, $\mathbb{R}$ and $\mathbb{R}_+$ denote integers, natural integers, reals, and positive (or null) reals, respectively. For two finite sets $\mathcal{X}$ and $\mathcal{Y}$, let $|\mathcal{X}|$ denote the cardinality of $\mathcal{X}$, and $[n]$ the subset $\{1, 2, \dots, n\} \subset \mathbb{N}$. Let 1^n denote the bit string of all ones of length n , for two bitstrings s and t , let $s||t$ denote the concatenation of s and t , and $|s|$ the length of s . $\{0;1\}^*$ will be the set of bitstrings of finite length. If an algorithm **Alg** runs in polynomial time it will be called efficient and if it is also probabilistic, a probabilistic polynomial time algorithm (PPT). $\text{out} \leftarrow \text{Alg}$ and $\text{out} \stackrel{\$}{\leftarrow} \text{Alg}$ will denote the output of deterministic and probabilistic algorithms respectively; if $\mathcal{X}$ is a finite set then $x \stackrel{\$}{\leftarrow} \mathcal{X}$ denotes sampling from it uniformly at random. A function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ is called negligible if it decreases faster than any inverse polynomial function.

Unless specified otherwise, vectors will be vertical and denoted with bold letters while their components will be denoted in regular font and with a subscript index. If only the components v_i for a vector are defined, it is assumed that they implicitly correspond to a vector $\mathbf{v}$. Matrices will be denoted with capital letters. Given a matrix B , its transpose is denoted by $B^\top$ and B^* will denote its Gram-Schmidt orthogonalisation. For $(\mathbf{b}_1, \dots, \mathbf{b}_n)$ its columns vectors, its norm will be defined as $\|B\| := \max_i \{\|\mathbf{b}_i\|\}$. $\mathcal{GL}_n(\mathbb{Q})$ will denote the general linear group of degree n over $\mathbb{Q}$, and $\mathcal{GL}_n(\mathbb{Z})$ its subgroup over $\mathbb{Z}$; similarly, $\mathcal{PGL}_n(\mathbb{Q})$ will denote the projective linear group over $\mathbb{Q}$, and $\mathcal{PGL}_n(\mathbb{Z})$ its subgroup over $\mathbb{Z}$. $\mathcal{S}_n^{>0}(\mathbb{R})$ will denote the set of real-valued positive definite symmetric matrices, also called quadratic forms. For a quadratic form Q in $\mathcal{S}_n^{>0}(\mathbb{R})$, B_Q will be the

unique upper-triangular lattice basis for Q , *i.e.* such that $Q = B_Q^\top B_Q$, obtained by Cholesky decomposition, and the norm associated to Q on $\mathbb{R}^n$ will be defined as $\|\cdot\|_Q : \mathbf{x} \mapsto \sqrt{\mathbf{x}^\top Q \mathbf{x}}$. For $i \in [n]$, the i -th minimal distance of Q is defined as $\lambda_i(Q) = \arg \min_{r \in \mathbb{R}^+} \{\text{rank}(\text{span}_{\mathbb{R}}(\mathbb{Z}^n \cap \mathcal{B}_{Q,r})) = i\}$, for $\mathcal{B}_{Q,r}$ the origin-centered ball of radius r for the norm $\|\cdot\|_Q$.

2.1 Generalised Group Actions

Previous works introduced generic constructions from group actions, usually instantiated with isogenies. We however, in this work, will provide them for more general group actions than before. Namely, for cases in which the group is not commutative (we consider right group actions) and infinite, given a good sampling algorithm, with properties specified hereafter. This will allow instantiations of resulting constructions not only from isogenies, but also from the Lattice Isomorphism Problem.

Definition 1 (Group Action). *Let $\mathbb{G}$ be a group and let $\mathcal{Q}$ be a set. A right group action of $\mathbb{G}$ on $\mathcal{Q}$ is a map $* : \mathcal{Q} \times \mathbb{G} \rightarrow \mathcal{Q}$ such that:*

1. *there exists e in $\mathbb{G}$ such that for all Q in $\mathcal{Q}$, $*(Q, e) = Q$;*
2. *for all $g_1, g_2 \in \mathbb{G}$ and $Q \in \mathcal{Q}$, $*(Q, g_1 g_2) = (*(Q, g_1), g_2)$.*

*We usually denote $Q * g := *(Q, g)$ for $g \in \mathbb{G}$ and $Q \in \mathcal{Q}$. Moreover, for Q in $\mathcal{Q}$, the orbit of Q is defined as $[Q] := \{Q * g \mid g \in \mathbb{G}\}$.*

Definition 2 (Free Group Action). *A group action $* : \mathcal{Q} \times \mathbb{G} \rightarrow \mathcal{Q}$ is free if there is a unique e in $\mathbb{G}$ such that, for any Q in $\mathcal{Q}$ and g in $\mathbb{G}$, $Q * g = Q$ iff g is equal to e .*

To be able to use an infinite group, we will require the existence of a sampling algorithm that we call the instance generator, and specify hereafter.

Definition 3 (Instance Generator). *An instance generator IG for a group action family $(*_\lambda : \mathcal{Q}_\lambda \times \mathbb{G}_\lambda \rightarrow \mathcal{Q}_\lambda)_{\lambda \in \mathbb{N}}$ is a PPT algorithm that on input $\lambda \in \mathbb{N}$ and Q in $\mathcal{Q}_\lambda$ outputs, except with negligible probability in λ , (Q', g) in $\mathcal{Q}_\lambda \times \mathbb{G}_\lambda$ such that $Q *_\lambda g = Q'$, where the distribution of Q' depends only on $[Q]$.*

Our constructions will rely on hardness of the Group Action Inverse Problem for a specified instance generator IG (GAIP_{IG}). We adapt our definition for GAIP_{IG} from Couveigne's vectorization problem [Cou06, Problem 6].

Definition 4 (Group Action Inverse Problem). *Let IG be an instance generator, specifying for any λ in $\mathbb{N}$, group actions $*_\lambda : \mathcal{Q}_\lambda \times \mathbb{G}_\lambda \rightarrow \mathcal{Q}_\lambda$ and $Q_\lambda \in \mathcal{Q}_\lambda$. We say that the GAIP_{IG} is hard if for any PPT algorithm $\mathcal{A}$ and $P \xleftarrow{\$} \text{IG}(Q_\lambda)$, there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that*

$$\Pr \left[Y = P *_\lambda g' \mid \begin{array}{l} (Y, g) \xleftarrow{\$} \text{IG}(P) \\ g' \xleftarrow{\$} \mathcal{A}(Y, P) \end{array} \right] \leq \text{negl}(\lambda).$$

When the definition of the instance generator is clear in the context, we may also refer to the GAIP_{IG} with GAIP .

Definition 5 (Useful Group Actions). *In the context of this work, a group action family $\mathcal{F} = (*_{\lambda} : \mathcal{Q}_{\lambda} \times \mathbb{G}_{\lambda} \rightarrow \mathcal{Q}_{\lambda})_{\lambda \in \mathbb{N}}$ will be called useful iff*

1. *all its group actions are free;*
2. *there exists an instance generator IG such that the GAIP_{IG} is hard for $\mathcal{F}$;*
3. *for every natural λ , there is an algorithm with polynomial time in λ to evaluate $*_{\lambda}$, and to check whether an element is in $\mathbb{G}_{\lambda}$.*

2.2 The Lattice Isomorphism Group Action (LIGA) Family

This section describes a group action family that satisfies Definition 5, stemming from the Lattice Isomorphism Problem (LIP) described in [DvW22]. Let n be a natural, and $\mathcal{S}_n^{>0}(\mathbb{R})$ the set of symmetric positive-definite matrices in $\mathbb{R}^{n \times n}$, corresponding to quadratic forms on $\mathbb{R}^n$. Given Q and n , the LIP exhibits the action of the group $\mathcal{GL}_n(\mathbb{Z})$ on $[Q]$ which to Q' in $[Q]$ and U in $\mathcal{GL}_n(\mathbb{Z})$ associates $U^T Q' U$.

Free Group Actions. In this article, the general lattice isomorphism group action is restricted in the following manner. The group $\mathcal{GL}_n(\mathbb{Z})$ will be restricted to $\mathcal{PGL}_n(\mathbb{Z})$, and automorphism-free forms will be considered, in order to obtain freedom of the corresponding group action.

Definition 6 (Lattice Isomorphism Group Actions). *In this paper, given a natural integer n and Q in $\mathcal{S}_n^{>0}(\mathbb{R})$, the Lattice Isomorphism Group Action (LIGA) will refer to the action of $\mathcal{PGL}_n(\mathbb{Z})$ on $\mathcal{S}_n^{>0}(\mathbb{R})$, restricted to $* : \mathcal{PGL}_n(\mathbb{Z}) \times [Q] \rightarrow [Q]$, $(U, R) \mapsto R * U := U^T R U$.*

For a quadratic form Q in $\mathcal{S}_n^{>0}(\mathbb{R})$, we define

$$\text{Aut}(Q) := \{U \in \mathcal{PGL}_n(\mathbb{Z}) \mid U^T Q U = Q\}$$

the automorphism group of Q , and say that Q is automorphism-free if $\text{Aut}(Q)$ is trivial, *i. e.* reduced to the identity. This article provides an instantiation of automorphism-free lattices and a free LIP-based group action in Lemma 29 of Appendix A.1.

An Instance Generator. The reader will find an instantiation of an instance generator IG for which the GAIP_{IG} hardness relies on the average-case search-LIP in Appendix A.2.

2.3 Nondeterministic Polynomial Time (NP) OR Relations

Definition 7 (NP Relation). *Let X and W be sets. We call the elements in $\mathcal{X}$ statements. We assume that elements of $\mathcal{X}$ and $\mathcal{W}$ are represented by bit strings. A subset $\mathcal{R} \subset \mathcal{X} \times \mathcal{W}$ is an NP-relation if:*

- $\mathcal{R}$ is polynomially bounded, i. e. there exists a polynomial $p \in \mathbb{N}[\lambda]$ such that $|w| < p(|x|)$ for all $(x, w) \in \mathcal{R}$;
- there exists a deterministic polynomial time algorithm that on input $(x, w) \in \mathcal{X} \times \mathcal{W}$ outputs 1 if $(x, w) \in \mathcal{R}$ and 0 otherwise.

For an NP relation $\mathcal{R}$ and tuple (x, w) in $\mathcal{R}$, we say that w is a witness for x .

Definition 8 (NP OR Relation). Let $\mathcal{R}_0 \subset \mathcal{X}_0 \times \mathcal{W}_0$ and $\mathcal{R}_1 \subset \mathcal{X}_1 \times \mathcal{W}_1$ be two NP relations. We define $\mathcal{X}^{\text{OR}} = \mathcal{X}_0 \times \mathcal{X}_1$, $\mathcal{W}^{\text{OR}} = (\{0\} \times \mathcal{W}_0) \cup (\{1\} \times \mathcal{W}_1)$ and the relation $\mathcal{R}^{\text{OR}} \subset \mathcal{X}^{\text{OR}} \times \mathcal{W}^{\text{OR}}$ such that $((x_0, x_1), (b, w))$ is in $\mathcal{R}^{\text{OR}}$ if and only if (x_b, w) is in $\mathcal{R}_b$. $\mathcal{R}^{\text{OR}}$ is the NP OR-relation for $\mathcal{R}_0$ and $\mathcal{R}_1$.

2.4 Commit and Prove Zero-Knowledge Proofs

Sigma protocols are three-round protocols designed for an NP relation $\mathcal{R} \subset \mathcal{X} \times \mathcal{W}$ that allow a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that, given a public statement $x \in \mathcal{X}$, they know a witness $w \in \mathcal{W}$ such that (x, w) is in $\mathcal{R}$.

Definition 9 (Sigma Protocol). Let $\mathcal{C}$ denote a challenge space. A sigma protocol for the relation $\mathcal{R} \subset \mathcal{X} \times \mathcal{W}$ is a collection of PPT algorithms $\Sigma_{\mathcal{R}} = (\text{Commit}, \text{Challenge}, \text{Prove}, \text{Verify})$ such that

- $\text{Commit}(x, w) \rightarrow (\text{st}_{\mathcal{P}}, a)$: on input a statement witness pair $(x, w) \in \mathcal{R}$, computes a commitment a and state $\text{st}_{\mathcal{P}}$;
- $\text{Challenge}(x, a) \rightarrow c$: on input a statement x and commitment a , samples a challenge $c \xleftarrow{\$} \mathcal{C}$ and returns it;
- $\text{Prove}(x, w, c, \text{st}_{\mathcal{P}}) \rightarrow z$: on input a challenge c , $(x, w) \in \mathcal{R}$ and prover state $\text{st}_{\mathcal{P}}$, outputs a response z ;
- $\text{Verify}(x, a, c, z) \rightarrow b$: on input a statement x and transcript (a, c, z) , consisting of a statement, commitment and response, computes and outputs $b \in \{0, 1\}$, corresponding to 1 for acceptance and 0 otherwise.

Definition 10 (Correctness). A sigma protocol satisfies correctness if for all $(x, w) \in \mathcal{R}$.

$$\Pr \left[\text{Verify}(x, a, c, z) = 1 \mid \begin{array}{l} (\text{st}_{\mathcal{P}}, a) \xleftarrow{\$} \text{Commit}(x, w) \\ c \xleftarrow{\$} \text{Challenge}(x, a) \\ z \xleftarrow{\$} \text{Prove}(x, w, c, \text{st}_{\mathcal{P}}) \end{array} \right] = 1$$

Definition 11 (Special soundness). A sigma protocol as in Definition 9 satisfies special soundness if there exists a PPT extractor algorithm $\mathcal{E}$ that given two accepting transcripts (a, c, z) and (a, c', z') for a statement x such that $c \neq c'$, outputs a witness w for x .

Definition 12 (Perfect Honest-Verifier Zero-Knowledge). A sigma protocol satisfies perfect honest-verifier zero-knowledge if there exists a PPT simulator algorithm Sim that given a statement x outputs an accepting transcript (a, c, z) for that statement, such that the output of Sim is statistically indistinguishable from honestly generated transcripts.

2.5 Blind signatures

We now define the concept of blind signatures, for which Section 4 will provide a generalised group action instantiation.

Definition 13 (Blind Signature Scheme). *A blind signature scheme is a tuple of four algorithms $(\text{Gen}, \text{BSign}, \text{BUser}, \text{Verify})$ that function as described below.*

$\text{KeyGen}(1^\lambda) \rightarrow (\text{pp}, \text{pk}, \text{sk})$: outputs public parameters pp , a secret key sk , used to sign messages, and a public key pk , to verify signatures.

$\text{BSign}(\text{sk})/\text{BUser}(m, \text{pk}) \rightarrow \sigma$: the algorithms BSign and BUser may interact during their execution. The algorithm BSign takes as input the secret key sk . The algorithm BUser , on input a message m and the public key pk , outputs a signature σ at the end of execution. We denote $\sigma \xleftarrow{\$} \langle \text{BSign}(\text{sk}), \text{BUser}(\text{pk}, m) \rangle$ the execution of these algorithms. As BSign and BUser may interact during their execution, we view the generation of σ as an alternation of signer and user algorithms. In this sequence we denote the i -th signer or user algorithm by BSign_i and BUser_i , respectively, and initiate it with BSign_1 .

$\text{Verify}(\text{pk}, m, \sigma) \rightarrow b$: on input a signature σ , a message m and the public key pk , outputs a bit b equal to one if verification passes and else zero.

Definition 14 (Perfect Correctness). *A blind signature scheme satisfies perfect correctness if for any message m and natural integer λ ,*

$$\Pr \left[\text{Verify}(\text{pk}, m, \sigma) = 1 \mid \begin{array}{l} (\text{pp}, \text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}(1^\lambda) \\ \sigma \xleftarrow{\$} \langle \text{BSign}(\text{sk}), \text{BUser}(\text{pk}, m) \rangle \end{array} \right] = 1.$$

Definition 15 (Blindness, adapted from [KLX22]). *A blind signature scheme satisfies blindness if there exists a negligible function $\text{negl}(\lambda) : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any PPT algorithm $\mathcal{A}$ and $\lambda \in \mathbb{N}$, for $\text{Exp}_{\mathcal{A}}^{\text{Blind}}$ the experiment described in Figure 1,*

$$\left| \Pr \left[\text{Exp}_{\mathcal{A}}^{\text{Blind}}(\lambda) = 1 \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

$\text{Exp}_{\mathcal{A}}^{\text{Blind}}(\lambda)$		
$(\text{pk}, m_0, m_1, \text{st}) \xleftarrow{\$} \mathcal{A}_0(1^\lambda)$	$b \xleftarrow{\$} \{0, 1\}$	
$\mathcal{U} := \text{BUser}(\text{pk}, m_b)$	$\mathcal{V} := \text{BUser}(\text{pk}, m_{1-b})$	
$\sigma_b \xleftarrow{\$} \langle \mathcal{A}_1(\text{st}), \mathcal{U} \rangle$	$\sigma_{1-b} \xleftarrow{\$} \langle \mathcal{A}_1(\text{st}), \mathcal{V} \rangle$	$\text{st}' \xleftarrow{\$} \mathcal{A}_1(\text{st})$
if $\sigma_0 = \perp$ or $\sigma_1 = \perp$ then $b' \xleftarrow{\$} \{0, 1\}$ else $b' \xleftarrow{\$} \mathcal{A}_2(\sigma_0, \sigma_1, \text{st}')$		
if $b = b'$ then return 1 else return 0		

Figure 1. The blindness security game, in which the adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1, \mathcal{A}_2)$ has a one-time access to $\mathcal{U}$ and $\mathcal{V}$'s sequential algorithms (in the order it wishes to as long as this sequentiality is preserved), through $\mathcal{A}_1(\text{st})$ which share a common internal state when interacting with them and then outputting the state st' .

Definition 16 (*k -One-More Unforgeability, adapted from [KLX22]*). A blind signature scheme satisfies k -one-more unforgeability, with k in $\mathbb{N}$, if for any PPT algorithm $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda) : \mathbb{N} \rightarrow \mathbb{R}_+$ such that, for λ in $\mathbb{N}$ and $\text{Exp}_{\mathcal{A}}^{k\text{-OMUF}}(\lambda)$ the experiment described in Figure 2,

$$\Pr \left[\text{Exp}_{\mathcal{A}}^{k\text{-OMUF}}(\lambda) = 1 \right] \leq \text{negl}(\lambda).$$

$\text{Exp}_{\mathcal{A}}^{k\text{-OMUF}}(\lambda)$	$\mathcal{O}_{\text{sk}}$
$\text{ctr} \leftarrow 0$	$\text{ctr} \leftarrow \text{ctr} + 1, s_{\text{ctr}} \leftarrow 0$
$(\text{pp}, \text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}(1^\lambda)$	$\mathcal{S}_{\text{ctr}} := \text{BSign}(\text{sk})$
$(m_0, \sigma_0), \dots, (m_k, \sigma_k) \xleftarrow{\$} \mathcal{A}^{\mathcal{O}_{\text{sk}}, \mathcal{O}_{\mathcal{S}, \text{sk}}}(\text{pk})$	$\mathcal{O}_{\mathcal{S}, \text{sk}}(i, j, \rho_{\text{Use}})$
if $\text{ctr} \leq k$ and	if $\mathcal{S}_i$ is defined and $s_i = j - 1$:
$\forall i \in \{0, \dots, k\}, \text{Verify}(\text{vk}, m_i, \sigma_i) = 1$	$\mathcal{S}_i$ runs out $\xleftarrow{\$} \text{BSign}_j(\rho_{\text{Use}}, \text{sk}),$
return 1	$s_i \leftarrow s_i + 1,$
else return 0	return out
	else return $\perp$

Figure 2. The k -one-more unforgeability game.

Proof Strategy for One-More Unforgeability. [KLX22] introduced convenient conditions implying one-more unforgeability for blind signatures, which are recalled, along with the corresponding formalisation, in Appendix B.

2.6 Strong Designated Verifier Signatures with Non-Delegatability

In Section 5, this work will introduce an instantiation of Strong Designated Verifier Signatures (SDVS) with non-delegatability based on the hardness of the GAIP, inspired by the isogeny-based construction of [MTT25]. A strong designated verifier signature scheme is a tuple of algorithms (**KeyGen**, **Sign**, **Verify**), described below. The **KeyGen** algorithm is used to generate both signer and verifier keys.

KeyGen $(1^\lambda, \ell) \rightarrow (\text{pk}, \text{sk})$: on input a security parameter λ in $\mathbb{N}$, and label ℓ in $\{s, v\}$ specifying whether the keys are intended for a signer or verifier, outputs a secret key sk and public key pk .

Sign $(\text{sk}_s, \text{pk}_s, \text{pk}_v, m) \rightarrow \sigma$: on input a message m and the keys $\text{sk}_s, \text{pk}_s, \text{pk}_v$, outputs a signature σ .

Verify $(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma) \rightarrow b$: on input a signature σ , message m and the keys sk_v, pk_s and pk_v , outputs a bit b , set to zero in case of rejection and one otherwise.

Definition 17 (Correctness). A strong designated verifier signature scheme $(\text{KeyGen}, \text{Sign}, \text{Verify})$ satisfies correctness if there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any natural integer λ , $(\text{pk}_s, \text{sk}_s) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$, $(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$ and message m ,

$$\Pr \left[\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma) = 1 \mid \sigma \xleftarrow{\$} \text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m) \right] = 1 - \text{negl}(\lambda).$$

Definition 18 (Strong Unforgeability, from [MTT25]). A strong designated verifier signature scheme $(\text{KeyGen}, \text{Sign}, \text{Verify})$ satisfies strong unforgeability if for any PPT algorithm $\mathcal{A}$, there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any $\lambda \in \mathbb{N}$, and $\text{Exp}_{\mathcal{A}}^{\text{SUF}}$ the experiment defined in Figure 3,

$$\Pr \left[\text{Exp}_{\mathcal{A}}^{\text{SUF}}(\lambda) = 1 \right] \leq \text{negl}(\lambda).$$

$\text{Exp}_{\mathcal{A}}^{\text{SUF}}(\lambda)$	$\mathcal{O}^{\text{Sign}}(m)$
$L^{\text{Sign}} \xleftarrow{\$} \{\}$	$\sigma \xleftarrow{\$} \text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m)$
$(\text{pk}_s, \text{sk}_s) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$	$L^{\text{Sign}} \leftarrow L^{\text{Sign}} \cup \{(m, \sigma)\}$
$(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$	return σ
$(m^*, \sigma^*) \xleftarrow{\$} \mathcal{A}^{\mathcal{O}^{\text{Sign}}, \mathcal{O}^{\text{Verify}}}(\text{pk}_s, \text{pk}_v)$	$\mathcal{O}^{\text{Verify}}(m, \sigma)$
if $(m^*, \sigma^*) \notin L^{\text{Sign}}$ and	$b \leftarrow \text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma)$
$\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m^*, \sigma^*) = 1$	return b
return 1	
else return 0	

Figure 3. The strong unforgeability game.

Definition 19 (Perfect Non-Transferability, adapted from [SS08]). A strong designated verifier signature scheme $(\text{KeyGen}, \text{Sign}, \text{Verify})$ satisfies perfect non-transferability if there exists a PPT algorithm Sim such that for any natural integer λ , $(\text{pk}_s, \text{sk}_s) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$, $(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$, and message m , the distributions on the outputs of $\text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m)$ and $\text{Sim}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m)$ are equal.

Definition 20 (Privacy of the Signer Identity, adapted from [LV05]). A strong designated verifier signature scheme $(\text{KeyGen}, \text{Sign}, \text{Verify})$ satisfies privacy of the signer identity if for any PPT algorithm $\mathcal{A}$, there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any λ in $\mathbb{N}$, for $\text{Exp}_{\mathcal{A}}^{\text{PSI}}$ the experiment defined in Figure 4,

$$\left| \Pr \left[\text{Exp}_{\mathcal{A}}^{\text{PSI}}(\lambda) = 1 \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

In this article's construction, the verifier's secret key will consist of one witness for an OR relation on two GAIP_{IG} instances, alongside a KEM decapsulation

$\text{Exp}_{\mathcal{A}}^{\text{PSI}}(\lambda)$	$\mathcal{O}^{\text{Sign}}(\beta, m)$
$(\text{pk}_0, \text{sk}_0) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$	return $\sigma \xleftarrow{\$} \text{Sign}(\text{sk}_\beta, \text{pk}_\beta, \text{pk}_v, m)$
$(\text{pk}_1, \text{sk}_1) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$	$\mathcal{O}^{\text{Verify}}(\beta, m, \sigma)$
$(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$	return $\beta \leftarrow \text{Verify}(\text{sk}_v, \text{pk}_\beta, \text{pk}_v, m, \sigma)$
$m^* \xleftarrow{\$} \mathcal{A}^{\mathcal{O}^{\text{Sign}}, \mathcal{O}^{\text{Verify}}}(\text{pk}_0, \text{pk}_1, \text{pk}_v)$	$\mathcal{O}^{\text{Verify}^*}(\beta, m, \sigma)$
$b \xleftarrow{\$} \{0, 1\}, \sigma^* \xleftarrow{\$} \text{Sign}(\text{sk}_b, \text{pk}_b, \text{pk}_v, m^*)$	if $(m, \sigma) = (m^*, \sigma^*)$ return $\perp$, else
$b' \xleftarrow{\$} \mathcal{A}^{\mathcal{O}^{\text{Sign}}, \mathcal{O}^{\text{Verify}^*}}(\text{pk}_0, \text{pk}_1, \text{pk}_v, \sigma^*)$	return $\beta \leftarrow \text{Verify}(\text{sk}_v, \text{pk}_\beta, \text{pk}_v, m, \sigma)$
if $b' = b$ return 1, else return 0	

Figure 4. Experiment for the privacy of signer identity.

key. Non-delegatability captures a notion showing that no signatures may be produced using neither signer nor verifier secret keys. In this work, this will mean without using a solution to a GAIP_{IG} instance, disregarding the decapsulation key; we hence define the following adapted non-delegatability notion.

Definition 21 (Non-Delegatability). *An SDVS scheme $(\text{KeyGen}, \text{Sign}, \text{Verify})$ satisfies non-delegatability with knowledge error κ if there exists a probabilistic oracle machine Ext such that for every PPT algorithm $\mathcal{F}$, there exists a polynomial poly in $\mathbb{N}[X]$ such that for any m in $\{0; 1\}^*$, λ in $\mathbb{N}$, $(\text{pk}_s, \text{sk}_s) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$, $(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$, denoting*

$$\epsilon := \Pr \left[\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma) = 1 \mid \sigma \xleftarrow{\$} \mathcal{F}(m, \text{pk}_s, \text{pk}_v) \right],$$

and $\mathcal{F}_m$ the restriction of $\mathcal{F}$ to the input m , $\text{Ext}^{\mathcal{F}_m}(m, \text{pk}_s, \text{pk}_v)$ runs in expected polynomial time, and there is a non-empty part $\mathcal{V}$ of sk_v in the output range of $\text{KeyGen}(1^\lambda, s)$ such that

$$\Pr \left[\text{sk}' \in \{\text{sk}_s, \mathcal{V}\} \mid \text{sk}' \xleftarrow{\$} \text{Ext}^{\mathcal{F}_m}(m) \right] \geq (\epsilon - \kappa) \left(\frac{\epsilon}{\text{poly}(\lambda)} - \kappa \right).$$

3 Zero-Knowledge Proofs Systems for the GAIP_{IG}

Standard zero-knowledge proofs may straightforwardly be obtained with sigma (*i.e.* three-way) commit and prove protocols analogous to discrete-logarithm based constructions, that may be made non-interactive using the Fiat-Shamir transform [FS87].

3.1 Zero-Knowledge Proofs Systems for the GAIP_{IG}

This section recalls the straightforward way to obtain a zero-knowledge proof of knowledge for a GAIP relation using a sigma protocol.

Definition 22 (GAIP NP Relation). Let IG be an instance generator such that the GAIP_{IG} is hard, and given λ in $\mathbb{N}$, $*$: $\mathcal{Q} \times \mathbb{G} \rightarrow \mathcal{Q}$ be the corresponding group action, and Q in $\mathcal{Q}$. $\mathcal{R}_{Q,\lambda} := \{(Q', U) \in [Q] \times \mathbb{G} \mid Q' = Q * U\}$ defines an NP relation that we will call the GAIP_{IG} relation for Q .

A sigma protocol for GAIP relations may be found in Appendix F.1 (Definition 54), as well as its non-interactive version obtained by a Fiat-Shamir transformation (Definition 56), and proofs for correctness, special soundness with error $2^{-\lambda}$, and perfect honest-verifier zero-knowledge (Theorem 55).

3.2 Zero-Knowledge OR Proofs Systems for the GAIP_{IG}

Definition 23 (GAIP NP OR Relation). Let $\mathcal{R}_{Q_0,\lambda}$ and $\mathcal{R}_{Q_1,\lambda}$ be two relations satisfying Definition 22 for the GAIP_{IG} , and $*$: $\mathcal{Q} \times \mathbb{G} \rightarrow \mathcal{Q}$ be the corresponding group action for both $\mathcal{R}_{Q_0,\lambda}$ and $\mathcal{R}_{Q_1,\lambda}$.

$$\mathcal{R}_{Q_0,Q_1,\lambda}^{\text{OR}} := \{((x_0, x_1), (b, w)) \in ([Q_0] \times [Q_1]) \times (\{0; 1\} \times \mathbb{G}) \mid (x_b, w) \in \mathcal{R}_{Q_b,\lambda}\}$$

defines an NP OR relation that we will call the GAIP_{IG} OR relation for (Q_0, Q_1) .

We now provide a sigma protocol for the NP OR relation $\mathcal{R}_{Q_0,Q_1,\lambda}^{\text{OR}}$ of Definition 23, adapted from the zero-knowledge proof system for the NP OR relation presented in [KLLQ23]. The original proof system is based on elliptic curve isogenies and uses the quadratic twist. Our adaptation removes the need for this operation, thus allowing the usage of more general group actions.

Definition 24 (Sigma Protocol for GAIP_{IG} OR Relations). We define a three-way sigma protocol to prove, given a statement x , the knowledge of a witness w such that (x, w) is in $\mathcal{R}_{Q_0,Q_1,\lambda}^{\text{OR}}$ satisfying Definition 23, as the tuple of algorithms $\Sigma_{Q_0,Q_1,\lambda}^{\text{OR}} = (\text{Commit}, \text{Challenge}, \text{Prove}, \text{Verify})$ described hereafter.

$\Sigma_{Q_0,Q_1,\lambda}^{\text{OR}}.\text{Commit}(x, w) \rightarrow (\text{st}_{\text{Prove}}, \rho_{\text{Prove}_1})$: on input (x, w) :

1. parses $(\delta, A_\delta) \leftarrow w$.
2. samples $\mathbf{c}_{1-\delta} \xleftarrow{\$} \{-1, 1\}^\lambda$ and, for $i \in [\lambda]$, $(Y_{\delta,i}, H_{\delta,i}) \xleftarrow{\$} \text{IG}(Q_\delta)$
and $(Y_{1-\delta,i}, U_{1-\delta,i}) \xleftarrow{\$} \begin{cases} \text{IG}(P_{1-\delta}) & \text{if } c_{1-\delta,i} = -1 \\ \text{IG}(Q_{1-\delta}) & \text{if } c_{1-\delta,i} = 1 \end{cases}$.
3. outputs $\text{st}_{\text{Prove}} := (\mathbf{H}_\delta, \mathbf{c}_{1-\delta}, \mathbf{U}_{1-\delta})$ and $\rho_{\text{Prove}_1} := (\mathbf{Y}_0, \mathbf{Y}_1)$.

$\Sigma_{Q_0,Q_1,\lambda}^{\text{OR}}.\text{Challenge}(x, \rho_{\text{Prove}_1}) \rightarrow \rho_{\text{Verify}}$: on input $(x, \rho_{\text{Prove}_1})$ samples $\mathbf{c} \xleftarrow{\$} \{-1, 1\}^\lambda$ and outputs $\rho_{\text{Verify}} := \mathbf{c}$.

$\Sigma_{Q_0,Q_1,\lambda}^{\text{OR}}.\text{Prove}(x, w, \text{st}_{\text{Prove}}, \rho_{\text{Verify}}) \rightarrow \rho_{\text{Prove}_2}$: on input $(x, w, \text{st}_{\text{Prove}}, \rho_{\text{Verify}})$:

1. parses $(\delta, A_\delta) \leftarrow w$, $(\mathbf{H}_\delta, \mathbf{c}_{1-\delta}, \mathbf{U}_{1-\delta}) \leftarrow \text{st}_{\text{Prove}}$ and $\mathbf{c} \leftarrow \rho_{\text{Verify}}$.
2. for $i \in [\lambda]$, sets $\mathbf{c}_{\delta,i} = c_i \cdot c_{1-\delta,i}$ and $U_{\delta,i} = \begin{cases} A_\delta^{-1} \cdot H_{\delta,k} & \text{if } c_{\delta,i} = -1 \\ H_{\delta,k} & \text{if } c_{\delta,i} = 1 \end{cases}$,
3. outputs $\rho_{\text{Prove}_2} := (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$.

$\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}.$ Verify($x, \rho_{\text{Prove}_1}, \rho_{\text{Verify}}, \rho_{\text{Prove}_2}$) $\rightarrow b$: on input $(x, \text{st}_{\text{Verify}}, \rho_{\text{Prove}_2})$ parses $(P_0, P_1) \leftarrow x, (Y_0, Y_1) \leftarrow \text{st}_{\text{Prove}_1}, \mathbf{c} \leftarrow \rho_{\text{Verify}}$ and $(\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1) \leftarrow \rho_{\text{Prove}_2}$.
If for each i in $[\lambda]$:

$$c_{0,i} \cdot c_{1,i} = c_i, \text{ and } \forall b \in \{0, 1\}, \begin{cases} P_b * U_{b,i} = Y_{b,i} & \text{if } c_{b,i} = -1 \\ Q_b * U_{b,i} = Y_{b,i} & \text{if } c_{b,i} = 1 \end{cases},$$

then the algorithm outputs $b \leftarrow 1$, else it returns $b \leftarrow 0$.

We prove the correctness of $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ in Appendix F.2. For $b \in \{0, 1\}$, we provide algorithms Ext_b in Figure 5 that act as special soundness witness extractors for a singular branch of $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$. In Lemma 61 (Appendix F.3) we provide conditions under which they are successful so they may be used generally.

$\text{Ext}_b(\sigma = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1), \sigma' = (\mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1))$ if $\mathbf{c}_0 = \mathbf{c}'_0$ and $\mathbf{c}_1 = \mathbf{c}'_1$, return $\perp$ else $\exists t \in [\lambda], b \in \{0, 1\} \mid c_{b,t} \neq c'_{b,t}$: if $c_{b,t} = -1$, return $U'_{b,t} \cdot U_{b,t}^{-1}$ else $c_{b,t} = 1$: return $U_{b,t} \cdot U'_{b,t}^{-1}$

Figure 5. The witness extractors Ext_b for $b \in \{0, 1\}$.

Theorem 25 (Special Soundness for $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$). *The $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ protocol satisfies special soundness with soundness error $2^{-\lambda}$.*

Proof. See Appendix F.3.

Theorem 26 (Perfect Honest-Verifier Zero Knowledge of $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$). *The $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ protocol satisfies perfect honest-verifier zero knowledge, i.e. there exists an algorithm Sim that on input $\lambda \in \mathbb{N}$ and a statement $x = (P_0, P_1) \in [Q_0] \times [Q_1]$, produces accepting transcripts for x matching the distribution of honestly generated transcripts.*

Proof. See Appendix F.4.

Definition 27 (Non-Interactive OR Proof System). *The protocol in Definition 24 can be made non-interactive using the Fiat-Shamir transform [FS87]. Let $\Pi_{Q_0, Q_1, \lambda}^{\text{OR}}$, consisting of algorithms $(\text{ComPr}, \text{Verify})$ denote the commit and prove scheme for which on input a statement x and witness w such that (x, w) is in $\mathcal{R}_{Q_0, Q_1, \lambda}^{\text{OR}}$, satisfying Definition 23, ComPr generates a non-interactive proof of knowledge of w , which may be checked with Verify .*

4 Blind Signatures from the GAIP_{IG}

This section presents a blind signature scheme whose security relies on this work's generalised GAIP_{IG}, using the corresponding zero-knowledge OR proof system described in Definition 24.

For λ a security parameter in $\mathbb{N}$, let $\mathcal{H} : \{0,1\}^* \rightarrow \{0,1\}^\lambda$ denote a hash function, and for a group action family satisfying definition 5 for the GAIP_{IG}, let $*$: $\mathcal{Q} \times \mathbb{G} \rightarrow \mathcal{Q}$ denote the one corresponding to λ , with $Q \in \mathcal{Q}$ specified. $\Pi_{Q,\lambda}$ is the GAIP proof specified in Definition 56, $\Sigma_{Q_0,Q_1,\lambda}^{\text{OR}}$ the GAIP OR proof mentioned in Definition 27, and $\Pi_{Q_0,Q_1,\lambda}^{\text{OR}}$ its interactive version. The GABS = (KeyGen, Sign₁, Use₁, Sign₂, Use₂) blind signature scheme is defined hereafter.

GABS.KeyGen(1^λ) $\rightarrow$ (sk, pk): on input a natural integer λ corresponding to Q for the GAIP_{IG}, samples $\delta \xleftarrow{\$} \{0,1\}$, $(P_0, A_0) \xleftarrow{\$} \text{IG}(Q)$ and $(P_1, A_1) \xleftarrow{\$} \text{IG}(Q)$, computes proofs $\pi_0 \xleftarrow{\$} \Pi_{Q,\lambda}.\text{ComPr}(P_0, A_0)$, $\pi_1 \xleftarrow{\$} \Pi_{Q,\lambda}.\text{ComPr}(P_1, A_1)$. It then sets and returns **sk** = (δ, A_δ) and **pk** = $(\text{pk}_0 := (P_0, \pi_0, \pi_1), \text{pk}_1 := (P_1, \pi_0, \pi_1))$.

GABS.Sign₁(sk, pk) $\rightarrow$ (st_{Sign}, $\rho_{\text{Sign},1}$): on input sk, pk:

1. parses $(\delta, A_\delta) \leftarrow \text{sk}$ and $((P_0, \pi_0, \pi_1), (P_1, \pi'_0, \pi'_1)) \leftarrow \text{pk}$;
2. aborts if $\pi_0 \neq \pi'_0$, $\pi_1 \neq \pi'_1$, π_0 does not verify, or π_1 does not verify;
3. samples $\mathbf{c}_{1-\delta}^* \xleftarrow{\$} \{-1, 1\}^\lambda$;
4. for $i \in [\lambda]$:
 - samples $(W_{\delta,i}^{(1)}, H_{\delta,i}^{(1)}) \xleftarrow{\$} \text{IG}(Q)$ and $(W_{\delta,i}^{(-1)}, H_{\delta,i}^{(-1)}) \xleftarrow{\$} \text{IG}(Q)$;
 - sets, for $\beta \in \{-1; 1\}$, $\pi_{\delta,i}^{(\beta)} \xleftarrow{\$} \Pi_{Q,P_\delta,\lambda}^{\text{OR}}.\text{ComPr}((W_{\delta,i}^{(\beta)}, W_{\delta,i}^{(\beta)}), (0, H_{\delta,i}^{(\beta)}))$,
$$(W_{1-\delta,i}^{(1)}, U_{1-\delta,i}^{*(1)}) \xleftarrow{\$} \begin{cases} \text{IG}(P_{1-\delta}) & \text{if } c_{1-\delta,i}^* = -1 \\ \text{IG}(Q) & \text{if } c_{1-\delta,i}^* = 1 \end{cases},$$

$$(W_{1-\delta,i}^{(-1)}, U_{1-\delta,i}^{*(-1)}) \xleftarrow{\$} \begin{cases} \text{IG}(Q) & \text{if } c_{1-\delta,i}^* = -1 \\ \text{IG}(P_{1-\delta}) & \text{if } c_{1-\delta,i}^* = 1 \end{cases}, \text{ and}$$

$$\pi_{\delta-1,i}^{(\beta)} \xleftarrow{\$} \Pi_{Q,P_{1-\delta},\lambda}^{\text{OR}}.\text{ComPr}\left(\left(W_{1-\delta,i}^{(\beta)}, W_{1-\delta,i}^{(\beta)}\right), \left(\frac{1-\beta \cdot c_{1-\delta,i}^*}{2}, U_{1-\delta,i}^{(\beta)}\right)\right);$$
5. sets **st_{Sign}** = $(\mathbf{c}_{1-\delta}^*, (\mathbf{H}_\delta^{(\beta)}, \mathbf{U}_{1-\delta}^{*(\beta)})_{\beta \in \{-1;1\}})$, $\rho_{\text{Sign},1} = (\mathbf{W}_b^{(\beta)}, \boldsymbol{\pi}_b^{(\beta)})_{b,\beta}$, and outputs (st_{Sign}, $\rho_{\text{Sign},1}$).

GABS.Use₁($m, \rho_{\text{Sign},1}$) $\rightarrow$ (st_{Use}, ρ_{Use}): on input $m \in \{0,1\}^*$, and first-signer message $\rho_{\text{Sign},1}$:

1. parses $(\mathbf{W}_b^{(\beta)}, \boldsymbol{\pi}_b^{(\beta)})_{b \in \{0,1\}, \beta \in \{-1;1\}} \leftarrow \rho_{\text{Sign},1}$;
2. if $\exists b \in \{0,1\}, \beta \in \{-1, 1\}, i \in [\lambda]$ such that $\pi_{b,i}^{(\beta)}$ does not verify, aborts;
3. for $b \in \{0,1\}, i \in [\lambda]$ samples $\mathbf{d}_b \xleftarrow{\$} \{-1, 1\}^\lambda$, $(Y_{b,i}, V_{b,i}) \xleftarrow{\$} \text{IG}(W_{b,i}^{(\mathbf{d}_b,i)})$;
4. sets $\mathbf{c} \leftarrow \mathcal{H}(Y_0 \| Y_1 \| m)$, $\mathbf{c}' := (c_k \cdot d_{0,k} \cdot d_{1,k})_{k \in [\lambda]}$, **st_{Use}** := $(\mathbf{d}_b, \mathbf{V}_b, \mathbf{Y}_b)_b$ and $\rho_{\text{Use}} = \mathbf{c}'$;
5. outputs (st_{Use}, ρ_{Use}).

GABS.Sign₂(sk, st_{Sign}, ρ_{Use}) $\rightarrow$ $\rho_{\text{Sign},2}$: on input sk, st_{Sign} and ρ_{Use} :

1. parses $(\delta, A_\delta) \leftarrow \text{sk}$, $(\mathbf{c}_{1-\delta}^*, (\mathbf{H}_\delta^{(\beta)}, \mathbf{U}_{1-\delta}^{*(\beta)})_{\beta \in \{-1;1\}}) \leftarrow \text{st}_{\text{Sign}}$, $\mathbf{c}' \leftarrow \rho_{\text{Use}}$;
2. for $i \in [\lambda]$ sets: $\mathbf{c}_{\delta,i}^* \leftarrow \mathbf{c}'_i \cdot c_{1-\delta,i}^*$, $U_{\delta,i}^{*(1)} \leftarrow A_\delta^{(c_{\delta,i}^* - 1)/2} \cdot H_{\delta,i}^{(1)}$ and $U_{\delta,i}^{*(-1)} \leftarrow A_\delta^{-(c_{\delta,i}^* + 1)/2} \cdot H_{\delta,i}^{(-1)}$;
3. outputs $\rho_{\text{Sign},2} := (\mathbf{c}_b^*, \mathbf{U}_b^{*(1)}, \mathbf{U}_b^{*(-1)})_{b \in \{0;1\}}$.

GABS.Use₂(st_{Use}, $\rho_{\text{Sign},2}, m, \text{pk}$) $\rightarrow \sigma$: on input st_{Use}, $\rho_{\text{Sign},2}$, m and pk:

1. parses $((P_0, \pi_0, \pi_1), (P_1, \pi'_0, \pi'_1)) \leftarrow \text{pk}, (\mathbf{d}_b, \mathbf{V}_b, \mathbf{Y}_b)_{b \in \{0,1\}} \leftarrow \text{st}_{\text{Use}}$ and $(\mathbf{c}_b^*, \mathbf{U}_b^{*(1)}, \mathbf{U}_b^{*(-1)})_{b \in \{0,1\}} \leftarrow \rho_{\text{Sign},2}$;
 2. aborts if $\pi_0 \neq \pi'_0, \pi_1 \neq \pi'_1$, or π_0 or π_1 does not verify;
 3. aborts if $\exists b \in \{0,1\}, i \in [\lambda] \mid c_{b,i}^* \notin \{-1,1\}$ or $U_{b,i}^{*(1)} \notin \mathbb{G}$ or $U_{b,i}^{*(-1)} \notin \mathbb{G}$;
 4. for $b \in \{0,1\}$ and $i \in [\lambda]$, sets $c_{b,i} \leftarrow c_{b,i}^* \cdot d_{b,i}$ and $U_{b,i} \leftarrow U_{b,i}^{*(d_{b,i})} \cdot V_{b,i}$;
 5. aborts if $\exists b \in \{0,1\}, i \in [\lambda] \mid Y_{b,i} \neq \begin{cases} P_b * U_{b,i} & \text{if } c_{b,i} = -1 \\ Q * U_{b,i} & \text{if } c_{b,i} = 1 \end{cases}$;
 6. outputs $\sigma := (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$.
- GABS.Verify(pk, m, σ) $\rightarrow b$:** on input pk, m and σ :
1. parses $((P_0, \pi_0, \pi_1), (P_1, \pi'_0, \pi'_1)) \leftarrow \text{pk}$ and $(\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1) \leftarrow \sigma$;
 2. if $\pi_0 \neq \pi'_0, \pi_1 \neq \pi'_1$, π_0 or π_1 does not verify, or $\exists b \in \{0,1\}, i \in [\lambda]$ such that $U_{b,i} \notin \mathbb{G}$, outputs 0;
 3. for $b \in \{0,1\}$ and $i \in [\lambda]$, sets $Y_{b,i} \leftarrow \begin{cases} P_b * U_{b,i} & \text{if } c_{b,i} = -1 \\ Q * U_{b,i} & \text{if } c_{b,i} = 1 \end{cases}$;
 4. computes $\mathbf{c} \leftarrow \mathcal{H}(\mathbf{Y}_0 \parallel \mathbf{Y}_1 \parallel m)$;
 5. if for each $i \in [\lambda]$, $c_i = c_{0,i} \cdot c_{1,i}$, outputs 1, else 0.

Proofs of the correctness, blindness under chosen keys, and one-more unforgeability of **GABS** may be found in Appendices G.1, G.2 and G.3 respectively. The proof for blindness differs from previous ones because the infinity of the group does not allow using uniform distributions on it. The proof for one-more unforgeability follows the strategy of [KLX22], generalised in [KLLQ23] to a group action setting.

5 Strong Designated Verifier Signatures with Non-Delegatability from the GAIP_{IG}

Using our generic zero-knowledge OR-proof in 24 we adapt the construction of [MTT25] to rely on the GAIP_{IG}.

For λ a security parameter in $\mathbb{N}$ and a group action family satisfying Definition 5 for the GAIP_{IG}, let $* : \mathcal{Q} \times \mathbb{G} \rightarrow \mathcal{Q}$ denote the one corresponding to λ , with $Q \in \mathcal{Q}$ specified by IG. Let $\mathcal{R}_{Q,\lambda}$ denote the GAIP_{IG} relation for $* : [Q] \times \mathbb{G} \rightarrow [Q]$ and let $\mathcal{R}_{Q,Q,\lambda}^{\text{OR}}$ denote the NP OR-relation constructed using two $\mathcal{R}_{Q,\lambda}$ relations. Let **KEM** = (Gen, Encaps, Decaps) be a key encapsulation mechanism satisfying correctness and SK-CCA-security (Appendix C). Let **SKE** = (KeyGen, Enc, Dec) be a symmetric key encryption scheme satisfying correctness and IND-EAV-security (Appendix D). We define a strong designated verifier signature scheme **GASDVS** = (KeyGen, Sign, Verify) hereafter, based on the OR-proof system $\Sigma_{Q,Q,\lambda}^{\text{OR}}$ (Definition 24) and encryption under **SKE** so that verification is only possible with knowledge of the encryption key.

GASDVS.KeyGen($1^\lambda, \ell$): on input a natural integer λ and a label $\ell \in \{s, v\}$. If $\ell = s$ the algorithm samples $(P_s, A_s) \xleftarrow{\$} \text{IG}(Q)$ and returns $(\text{pk}_s := P_s, \text{sk}_s := A_s)$. If $\ell = v$, the algorithm samples $(P_v, A_v) \xleftarrow{\$} \text{IG}(Q)$ samples $(\text{pk}_K, \text{sk}_K) \xleftarrow{\$} \text{KEM.Gen}(1^\lambda)$ and returns $(\text{pk}_v := (P_v, \text{pk}_K), \text{sk}_v = (A_v, \text{sk}_K))$.

GASDVS.Sign($\text{sk}_s, \text{pk}_s, \text{pk}_v, m$) $\rightarrow (\sigma, E)$: provided the keys $(\text{sk}_s, \text{pk}_s, \text{pk}_v)$ and a message m , the algorithm:

1. parses $A_s \leftarrow \text{sk}_s, P_s \leftarrow \text{pk}_s, (P_v, \text{sk}_K) \leftarrow \text{pk}_v$;
2. sets $x = (P_s, P_v)$ and $w = (0, A_s)$
3. computes $(\text{st}, \rho = (\mathbf{Y}_s, \mathbf{Y}_v)) \xleftarrow{\$} \Sigma_{Q,Q,\lambda}^{\text{OR}}.\text{Commit}(x, w)$.
4. sets $(K, E) \xleftarrow{\$} \text{KEM.Encaps}(\text{pk}_K)$ and $e = \mathcal{H}(m \| K \| E \| \mathbf{Y}_s \| \mathbf{Y}_v)$.
5. computes $(\mathbf{c}_s, \mathbf{c}_v, \mathbf{U}_s, \mathbf{U}_v) \xleftarrow{\$} \Sigma_{Q,Q,\lambda}^{\text{OR}}.\text{Prove}(x, w, \text{st}, e)$.
6. sets $V \xleftarrow{\$} \text{SKE.Enc}(K, (\mathbf{U}_s, \mathbf{U}_v))$, sets $\sigma \leftarrow (\mathbf{c}_s, \mathbf{c}_v, V, \mathbf{Y}_s, \mathbf{Y}_v)$ and returns (σ, E) .

GASDVS.Verify($\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma, E$) $\rightarrow b$: on input $(\text{sk}_v, \text{pk}_s, \text{pk}_v)$, a message m , a signature σ and an encapsulation E , the algorithm:

1. parses $(A_v, \text{sk}_K) \leftarrow \text{sk}_v, P_s \leftarrow \text{pk}_s, (P_v, \text{pk}_K) \leftarrow \text{pk}_v$ and $(\mathbf{c}_s, \mathbf{c}_v, V, \mathbf{Y}_s, \mathbf{Y}_v) \leftarrow \sigma$;
2. sets $K \leftarrow \text{KEM.Decaps}(\text{sk}_K, E)$ and $(\mathbf{U}_s, \mathbf{U}_v) \leftarrow \text{SKE.Dec}(K, V)$;
3. for $i \in [\lambda]$, sets $Y'_{s,i} \leftarrow \begin{cases} P_s * U_{s,i} & \text{if } c_{s,i} = -1 \\ Q * U_{s,i} & \text{if } c_{s,i} = 1 \end{cases}$,
sets $Y'_{v,i} \leftarrow \begin{cases} P_v * U_{v,i} & \text{if } c_{v,i} = -1 \\ Q * U_{v,i} & \text{if } c_{v,i} = 1 \end{cases}$, and $e \leftarrow \mathcal{H}(m \| K \| E \| \mathbf{Y}_s \| \mathbf{Y}_v)$;
4. if $\mathbf{Y}'_s = \mathbf{Y}_s, \mathbf{Y}'_v = \mathbf{Y}_v$ and for all $i \in [\lambda]$, $e_i = c_{s,i} \cdot c_{v,i}$, returns 1, else outputs 0.

Proofs of the correctness, non-transferability, strong unforgeability, privacy of signer identities and non-delegatability of GASDVS may be found in respective Appendices H.1, H.2, H.3, H.4 and H.5.

Acknowledgements. We thank Julia Kastner for helpful discussions. Paola de Perthuis was supported by the NWO Gravitation Project QSC.

References

- AHJ21. Nabil Alkeilani Alkadri, Patrick Harasser, and Christian Janson. BlindOR: an efficient lattice-based blind signature scheme from OR-proofs. In Mauro Conti, Marc Stevens, and Stephan Krenn, editors, *CANS 21*, volume 13099 of *LNCS*, pages 95–115. Springer, Cham, December 2021.
- BBCK24. Benjamin Benčina, Alessandro Budroni, Jesús-Javier Chi-Domínguez, and Mukul Kulkarni. Properties of lattice isomorphism as a cryptographic group action. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part I*, pages 170–201. Springer, Cham, June 2024.
- BdPP23. Théophile Brézot, Paola de Perthuis, and David Pointcheval. Covercrypt: an efficient early-abort kem for hidden access policies with traceability from the ddh and lwe. In Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis, editors, *ESORICS 2023, Part I*, volume 14344 of *LNCS*, pages 372–392, September 2023.

- BLL⁺21. Fabrice Benhamouda, Tancrede Lepoint, Julian Loss, Michele Orrù, and Mariana Raykova. On the (in)security of ROS. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 33–53. Springer, Cham, October 2021.
- BLP⁺13. Zvika Brakerski, Adeline Langlois, Chris Peikert, Oded Regev, and Damien Stehlé. Classical hardness of learning with errors. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 575–584. ACM Press, June 2013.
- BN06. Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 390–399. ACM Press, October / November 2006.
- BY91. Gilles Brassard and Moti Yung. One-way group actions. In Alfred J. Menezes and Scott A. Vanstone, editors, *CRYPTO'90*, volume 537 of *LNCS*, pages 94–107. Springer, Berlin, Heidelberg, August 1991.
- CDS94. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In Yvo Desmedt, editor, *CRYPTO'94*, volume 839 of *LNCS*, pages 174–187. Springer, Berlin, Heidelberg, August 1994.
- Cou06. Jean-Marc Couveignes. Hard homogeneous spaces. Cryptology ePrint Archive, Report 2006/291, 2006.
- DHP24. Khue Do, Lucjan Hanzlik, and Eugenio Paracucchi. M&M'S: Mix and match attacks on Schnorr-type blind signatures with repetition. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 363–387. Springer, Cham, May 2024.
- DKQ⁺25. Dung Hoang Duong, Xuan Thanh Khuc, Youming Qiao, Willy Susilo, and Chuanqi Zhang. Blind signatures from cryptographic group actions. Cryptology ePrint Archive, Report 2025/397, 2025.
- DvW22. Léo Lucas and Wessel P. J. van Woerden. On the lattice isomorphism problem, quadratic forms, remarkable lattices, and cryptography. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 643–673. Springer, Cham, May / June 2022.
- FO13. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013.
- FS87. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.
- HLM⁺25. Lucjan Hanzlik, Yi-Fu Lai, Marzio Mula, Eugenio Paracucchi, Daniel Slamanig, and Gang Tang. Tanuki: New frameworks for (concurrently secure) blind signatures from post-quantum group actions. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part IV*, volume 16248 of *LNCS*, pages 35–69. Springer, Singapore, December 2025.
- KLLQ23. Shuichi Katsumata, Yi-Fu Lai, Jason T. LeGrow, and Ling Qin. CSI-Otter: Isogeny-based (partially) blind signatures from the class group action with a twist. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 729–761. Springer, Cham, August 2023.

- KLP25. Veronika Kuchta, Jason T. LeGrow, and Edoardo Persichetti. Post-quantum blind signatures from matrix code equivalence. Cryptology ePrint Archive, Report 2025/274, 2025.
- KLR24. Shuichi Katsumata, Yi-Fu Lai, and Michael Reichle. Breaking parallel ROS: Implication for isogeny and lattice-based blind signatures. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part I*, volume 14601 of *LNCS*, pages 319–351. Springer, Cham, April 2024.
- KLX22. Julia Kastner, Julian Loss, and Jiayu Xu. The Abe-Okamoto partially blind signature scheme revisited. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 279–309. Springer, Cham, December 2022.
- KTAT20. Tomoki Kawashima, Katsuyuki Takashima, Yusuke Aikawa, and Tsuyoshi Takagi. An efficient authenticated key exchange from random self-reducibility on CSIDH. In Deukjo Hong, editor, *ICISC 20*, volume 12593 of *LNCS*, pages 58–84. Springer, Cham, December 2020.
- LV05. Fabien Laguillaumie and Damien Vergnaud. Designated verifier signatures: Anonymity and efficient construction from any bilinear map. In Carlo Blundo and Stelvio Cimato, editors, *SCN 04*, volume 3352 of *LNCS*, pages 105–119. Springer, Berlin, Heidelberg, September 2005.
- MTT25. Hiroki Minamide, Keisuke Tanaka, and Masayuki Tezuka. Strong designated verifier signatures with non-delegatability from CSIDH. In *International Conference on Information Security and Cryptology, ICISC 2025*, 2025.
- Sah99. Amit Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *40th FOCS*, pages 543–553. IEEE Computer Society Press, October 1999.
- Sha79. Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.
- Sho01. Victor Shoup. A proposal for an iso standard for public key encryption. https://shoup.net/papers/iso-2_1.pdf, December 2001.
- SS08. Siamak Fayyaz Shahandashti and Reihaneh Safavi-Naini. Construction of universal designated-verifier signatures and identity-based signatures from standard signatures. In Ronald Cramer, editor, *PKC 2008*, volume 4939 of *LNCS*, pages 121–140. Springer, Berlin, Heidelberg, March 2008.
- SS13. Nicolas Sendrier and Dimitris E. Simos. The hardness of code equivalence over and its application to code-based cryptography. In Philippe Gaborit, editor, *Post-Quantum Cryptography - 5th International Workshop, PQCrypto 2013*, pages 203–216. Springer, Berlin, Heidelberg, June 2013.

Appendices

A Automorphism Free Quadratic Forms and an Instance Generator for the Lattice Isomorphism Problem

A.1 Automorphism Free Quadratic Forms

In order to obtain a free group action, we use the following lemma.

Lemma 28 (Free Lattice Isomorphism Group Actions, from [BBCK24], Proposition 3). *Let Q be in $\mathcal{S}_n^{>0}(\mathbb{R})$. Then, the action of the group $\mathcal{PGL}_n(\mathbb{Z})$ on $[Q]$ is free if and only if Q is automorphism-free.*

And hereafter instantiate automorphism-free quadratic forms.

Lemma 29 (Automorphism-Free Quadratic Forms). *For n a natural integer, denoting p_i the i -th prime in $\mathbb{N}$,*

$$Q_n := \begin{pmatrix} 1 + \sqrt{p_1} & 1 & 1 & \dots & 1 \\ 1 & 1 + \sqrt{p_2} & 1 & \dots & 1 \\ \vdots & & \ddots & \ddots & 1 \\ 1 & 1 & \dots & 1 & 1 + \sqrt{p_n} \end{pmatrix}$$

is an automorphism-free quadratic form.

Proof. Let U be in $\text{Aut}(Q_n)$, and $\mathbf{u}_k = (u_1, \dots, u_n)$ be the k -th column of U . Then

$$\begin{aligned} \mathbf{u}_k^\top Q_n \mathbf{u}_k &= 1 + \sqrt{p_k} \\ \sum_{i=1}^n (1 + \sqrt{p_i}) u_i^2 + 2 \sum_{j \neq i} u_j u_i &= 1 + \sqrt{p_k} \\ \left(\sum_{i=1}^n \sqrt{p_i} u_i^2 \right) + \left(\sum_{i=1}^n u_i \right)^2 &= 1 + \sqrt{p_k}. \end{aligned}$$

The strict positivity of $(\sum_{i=1}^n \sqrt{p_i} u_i^2) + (\sum_{i=1}^n u_i)^2$ for any non-zero $\mathbf{u}_k$ proves the positive definiteness of the symmetric matrix Q_n . Moreover, the above equation for the column $\mathbf{u}_k$ shows that, denoting $\mathbf{e}_i$ the i -th vector of the canonical basis of $\mathbb{Z}^n$, $\mathbf{u}_k = b_k \mathbf{e}_k$, for some b_k in $\{-1; 1\}$. Thus, for any column index j different from k ,

$$\mathbf{u}_j^\top Q_n \mathbf{u}_k = 1 \iff b_j b_k = 1$$

and finally, U is equal to the identity in $\mathcal{PGL}_n(\mathbb{Z})$. $\square$

A.2 An Instance Generator for the Lattice Isomorphism Problem

For Q in $\mathcal{S}_n^{>0}(\mathbb{R})$, s a strictly positive real number, we define on the orbit of Q the following distribution: $\rho_{Q,s} : \mathbb{R}^n \rightarrow \mathbb{R}_+$, $\mathbf{x} \mapsto e^{-\pi \cdot \|\mathbf{x}\|_Q^2 s^{-2}}$, and, as the sum of its evaluations on $\mathbb{Z}^n$ converges, denote $\rho_{Q,s}(\mathbb{Z}^n) := \sum_{\mathbf{x} \in \mathbb{Z}^n} \rho_{Q,s}(\mathbf{x})$.

Definition 30 (Discrete Gaussian distribution). *The origin-centred discrete Gaussian distribution $\mathcal{D}_{Q,s}$ satisfies*

$$\Pr_{X \stackrel{\$}{\leftarrow} \mathcal{D}_{Q,s}} [X = \mathbf{x}] = \begin{cases} \frac{\rho_{Q,s}(\mathbf{x})}{\rho_{Q,s}(\mathbb{Z}^n)} & \text{if } \mathbf{x} \in \mathbb{Z}^n \\ 0 & \text{otherwise} \end{cases}.$$

Lemma 31 (Efficient Discrete Sampling, from [BLP⁺13], Lemma 2.3). *Let Q be in $\mathcal{S}_n^{>0}(\mathbb{R})$. [BLP⁺13] provides a polynomial-time algorithm to sample from $\mathcal{D}_{Q,s}$ if $s \geq \|B_Q^*\| \cdot \sqrt{\log(2n+4)/\pi}$.*

Let IG be the instance generating algorithm, originally presented in [DvW22] and defined as in Algorithm 1. The algorithm takes as input a quadratic form Q and a real number $s > 0$.

```

1  IG( $Q, s$ )


---


2  If  $Q \notin \mathcal{S}_n^{>0}(\mathbb{R})$  or  $s < \max \left\{ \lambda_n(Q), \|B_Q^*\| \sqrt{\log(2n+4)/\pi} \right\}$ , abort
3  else
4       $C \leftarrow 1 - (1 + e^{-\pi})^{-1}$ ,  $m \leftarrow \lceil \frac{2n}{C} \rceil$ 
5      while  $\text{rank}(Y) < n$  do
6          Sample  $\mathbf{y}_1, \dots, \mathbf{y}_m \stackrel{\$}{\leftarrow} \mathcal{D}_{Q,s}$  (c. f. Lemma 31)
7          and set  $Y := (\mathbf{y}_1, \dots, \mathbf{y}_m)$ .
8      end
9      Compute  $U$  in  $\mathcal{PGL}_n(\mathbb{Z})$  such that  $U^{-1}Y$  is the canonical
        upper-triangular Hermite Normal Form of  $Y$ .
10     return  $(Q * U, U)$ .
11 end
```

Algorithm 1: Lattice Isomorphism Group Action Inverse Problem Instance Generator IG, from [DvW22]. If the algorithm does not terminate within polynomial time, it returns $\perp$.

Ducas and Woerden [DvW22] then showed that the first component output by IG only depended on the class $[Q]$ and not its particular representative.

Lemma 32 (Independence and Efficiency of IG, from [DvW22], Lemma 3.4). *Let Q be in $\mathcal{S}_n^{>0}(\mathbb{R})$ and s be a real number greater than or equal to $\max \left\{ \lambda_n(Q), \|B_Q^*\| \sqrt{\log(2n+4)/\pi} \right\}$. Then $\text{IG}(Q, s)$ runs in polynomial time and returns $(R, U) \in [Q] \times \mathcal{PGL}_n(\mathbb{Z})$, where R follows a distribution that we call $\mathcal{D}_s([Q])$ and only depends on $[Q]$ and s , not Q itself, and U satisfies $R = U^\top Q U$.*

This sampler allows the definition of an average-case Group-Action Inverse Problem (GAIP), which is called the search Lattice Isomorphism Problem.

Definition 33 (ac-sLIP_s^Q, from [DvW22], Definition 3.7). Let Q be in $\mathcal{S}_n^{>0}(\mathbb{R})$, and s be a real number greater than or equal to $\lambda_n(Q)$ and $\|B_Q^*\| \sqrt{\log(2n+4)/\pi}$. The corresponding average-case search lattice isomorphism problem, ac-sLIP_s^Q is, given $R \xleftarrow{\$} \mathcal{D}_s([Q])$, to find $U \in \mathcal{PGL}_n(\mathbb{Z})$ such that $R = U^\top Q U$.

Lemma 34 (Hardness of GAIP_{IG}). Let, for any natural number n , $Q_n \in \mathcal{S}_n^{>0}(\mathbb{R})$ and s_n be arbitrarily set to be greater than or equal to $\max\{\lambda_n(Q), \|B_Q^*\| \sqrt{\log(2n+4)/\pi}\}$. The algorithm IG, taking for any λ in $\mathbb{N}$, (Q_λ, s_λ) as input fulfils Definition 3 of an instance generator, for which GAIP_{IG} is hard under the $(sLIP_{s_n}^{Q_n})_{n \in \mathbb{N}}$ assumptions.

Proof. Indeed, for any natural integer λ and PPT adversary $\mathcal{A}$, one can straightforwardly define a PPT adversary $\mathcal{B}$ behaving as $\mathcal{A}$ such that $\Pr[\text{Exp}_{\mathcal{A}}^{\text{IG}}(\lambda) = 1]$ is exactly the advantage of $\mathcal{B}$ in the sLIP_{s_λ}^{Q_λ} game with security parameter λ .

B Background on Unforgeability Proofs for Blind Signatures from [KLX22] and [KLLQ23]

In this section we provide the definitions and lemmas necessary to state the theorem from which we obtain the one-more unforgeability of GABS. The theory that we discuss was originally introduced in [KLX22] to prove the security of the Abe-Okamoto partially blind signature scheme and some definitions have been generalised in [KLLQ23] for compatibility with more general constructions.

We first state the following definition that describes setting in which the proof strategy from [KLX22] is applicable. We will later show (Section G) that this lemma is satisfied for our construction.

Definition 35 (Required Conditions for the OMUF Proof Strategy). Let BS be a blind signature scheme that is to be specified. For IG an instance generator, λ in $\mathbb{N}$, and Q the corresponding set element for IG, the following hold:

1. The signatures that are produced under BS correspond to proofs for $\Sigma_{Q,Q,\lambda}^{\text{OR}}$ (Definition 24) for the NP OR relation $\mathcal{R}_{Q,Q,\lambda}^{\text{OR}} \subset \mathcal{X}^{\text{OR}} \times \mathcal{W}^{\text{OR}}$ (Definition 23).
2. Valid public keys for BS are represented by elements in $\mathcal{X}^{\text{OR}}$. For each honestly generated public key $\text{pk} \in \mathcal{X}^{\text{OR}}$, there exist exactly two distinct secret keys $\text{sk}_0 = (0, W_0)$ and $\text{sk} = (1, W_1)$ in $\mathcal{W}^{\text{OR}} = \{0, 1\} \times \mathcal{W}$ corresponding to pk , that is, $(\text{pk}, \text{sk}_0) \in \mathcal{R}^{\text{OR}}$ and $(\text{pk}, \text{sk}_1) \in \mathcal{R}^{\text{OR}}$.
3. It can be assumed without loss of generality that the adversary in the OMUF game, if it outputs $k+1$ message signatures pairs for $k \in \mathbb{N}$, it makes $k+1$ distinct hash queries in order to do so and each hash query corresponds to exactly one message signature pair.
4. The message ρ_{Use} that BUser₁ sends to BSign₂ is in the challenge space of the underlying sigma protocol.

Definition 36 (Instances, from [KLLQ23], Definition 3.1). Consider a blind signature scheme BS which satisfies the conditions in Definition 35. Let pk be an honestly generated public key and let $\text{sk}_0 = (0, W_0)$, $\text{sk}_1 = (1, W_1)$ be the two corresponding secret keys. For b in $\{0, 1\}$, a b -side instance I consists of sk_b and any randomness used by the signer algorithm $\text{Sign}(\text{sk}_b)$, excluding the randomness used for key generation.

We introduce a second type of vector notation where vectors are denoted by an overhead arrow. This type of vector notation indicates that each of the vector components corresponds to one of the message signature pairs that the adversary outputs in the OMUF game. A component of such a vector is denoted with a subscript index or a superscript index in parenthesis if the component itself can already be indexed using subscript. Vector components are denoted without the overhead arrow. Let $\vec{h}$ denote the $(k + 1)$ -long vector of answers from a random oracle $\mathcal{O}^{\mathcal{H}}$.

$\mathcal{W}$ will be a wrapper algorithm interacting with an adversary $\mathcal{A}$ which uses randomness rand , taking as other inputs an instance I and the vector $\vec{h}$. $\mathcal{W}(I, \text{rand}, \vec{h})$ will output $\perp$ if the adversary aborts, and else will return $\mathcal{A}$'s output.

Definition 37 (Successful Tuples, from [KLX22]). The set of successful tuples is defined as

$$\text{Succ} := \{(I, \text{rand}, \vec{h}) \mid \mathcal{W}(I, \text{rand}, \vec{h}) \neq \perp\}$$

Definition 38 (Successful Forking, from [KLX22], Definition 7). Two successful input tuples $(I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h}')$ $\in \text{Succ}$, fork from each other at index $i \in [k + 1]$ if $\forall j < i, h_j = h'_j$, and $h_i \neq h'_i$. i -th successful forking sets are defined as

$$F_i(I, \text{rand}) = \{(h_i, h'_i) \mid (I, \text{rand}, \vec{h}) \text{ and } (I, \text{rand}, \vec{h}') \text{ fork at index } i.\}$$

Definition 39 (Query Transcript, from [KLX22], Definition 5). Let $\mathcal{W}$ be the wrapper algorithm from earlier which runs on input $(I, \text{rand}, \vec{h})$. Let the query transcript, denoted by $\vec{e}(I, \text{rand}, \vec{h})$, be the vector of messages from the adversary playing the role of a user to BSign during the simulation by $\mathcal{W}$.

Definition 40 (Full Transcript, from [KLX22], Definition 6). Let $\mathcal{W}$ be the wrapper algorithm from earlier which runs on input $(I, \text{rand}, \vec{h})$. Let $\text{trans}(I, \text{rand}, \vec{h})$ denote the full transcript, given by the transcript produced between the signer and the adversary.

Definition 41 (Partners, from [KLX22], Definition 8). Two successful tuples $(I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h}') \in \text{Succ}$ are partners at index $i \in [k + 1]$ if:

1. $(I, \text{rand}, \vec{h})$ and $(I, \text{rand}, \vec{h}')$ fork at index i .
2. $\vec{e}(I, \text{rand}, \vec{h}) = \vec{e}(I, \text{rand}, \vec{h}')$

We define the set

$$\text{prt}_i(I, \text{rand}) := \left\{ \left(\vec{h}, \vec{h'} \right) \mid \begin{array}{l} (I, \text{rand}, \vec{h}) \text{ and } (I, \text{rand}, \vec{h'}) \\ \text{are partners at index } i \end{array} \right\}.$$

Definition 42 (Triangles, from [KLX22], Definition 9). We denote a triangle at index $i \in [k+1]$ with respect to (I, rand) by

$$\Delta_i(I, \text{rand}) := \left\{ \begin{array}{l} ((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h'})) \in \text{prt}_i(I, \text{rand}) \\ (I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h'}) \in F_i(I, \text{rand}) \\ (I, \text{rand}, \vec{h''}), (I, \text{rand}, \vec{h'''}) \in F_i(I, \text{rand}) \end{array} \right\}.$$

For a triangle $((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h'}), (I, \text{rand}, \vec{h''})) \in \Delta_i(I, \text{rand})$ we say that $((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h'}))$ denotes the base and we call $((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h''}))$ and $((I, \text{rand}, \vec{h'}), (I, \text{rand}, \vec{h''}))$ sides.

Definition 43 (Witness Extractors, from [KLLQ23], Definition 3.9).

Let I, rand be fixed and consider a successful fork $(\vec{h}, \vec{h'}) \in F_i(I, \text{rand})$ for some $i \in [k+1]$. Note that $(I, \text{rand}, \vec{h})$ and $(I, \text{rand}, \vec{h'})$ are successful tuples. Let σ_i and σ'_i denote the signatures that correspond to h_i and h'_i , respectively. Two deterministic algorithms Ext_0 and Ext_1 are witness extractors if

$$(\text{Ext}_0(\sigma_i, \sigma'_i), \text{Ext}_1(\sigma_i, \sigma'_i)) \in \{(\text{sk}_0, \perp), (\perp, \text{sk}_1), (\text{sk}_0, \text{sk}_1)\}.$$

For $b \in \{0, 1\}$, we say that a b -side witness can be extracted from $(I, \text{rand}, \vec{h})$ and $(I, \text{rand}, \vec{h'})$ at index i if $\text{Ext}_b(\sigma_i, \sigma'_i) = \text{sk}_b$.

Lemma 44 (From [KLX22], Lemma 2).

Let $\text{rand}, \vec{h}$ be fixed. For successful tuples $(I, \text{rand}, \vec{h}) \in \text{Succ}$ there exists a map $\Phi_{\text{rand}, \vec{h}}(I)$ be a function that maps a b -side instance I to a $(1-b)$ -side instance. Furthermore, $\Phi_{\text{rand}, \vec{h}}$ is a self-inverse bijection and it respects transcripts, that is, for all $(I, \text{rand}, \vec{h}) \in \text{Succ}$,

$$\text{trans}(I, \text{rand}, \vec{h'}) = \text{trans}\left(\Phi_{\text{rand}, \vec{h}}(I, \text{rand}, \vec{h'})\right).$$

Lemma 45 (From [KLX22], Corollary 3). Consider fixed values for rand and $\vec{h}$ and consider $(\vec{h}, \vec{h'}, \vec{h''}) \in \Delta_i(I, \text{rand})$ for some $i \in [k+1]$. If a b -side witness can be extracted from the base $((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h'}))$ at index i , then then one can also extract a b -side instance at index i from at least one of the sides $((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h''}))$ or $((I, \text{rand}, \vec{h'}), (I, \text{rand}, \vec{h''}))$.

Theorem 46 (Obtaining k -One-More Unforgeability, Theorem 3.12 from [KLLQ23]). Let BS be a blind signature scheme satisfying Definition 35. In

particular, let GAIP_{IG} be the problem from condition 1. of Definition 35. Let $\mathcal{C}$ be the challenge space of the sigma protocol for the NP OR relation underlying BS. If Lemma 44 and Lemma 45 hold, then for all k, λ in $\mathbb{N}$, if there exists a PPT adversary $\mathcal{A}$ that makes $Q_{\mathcal{H}}$ hash queries to a random oracle, if $\text{Adv}_{\mathcal{A}}^{k\text{-OMUF}}(\lambda) \geq \frac{C_1}{|\mathcal{C}|} \cdot \binom{Q_{\mathcal{H}}}{k+1}$ then

$$\text{Adv}_{\mathcal{A}}^{k\text{-OMUF}}(\lambda) \leq \sqrt{\left(\binom{Q_{\mathcal{H}}}{k+1}\right)^2 \cdot \frac{(k+1)^3}{C_2} \cdot \text{Adv}_{\mathcal{A}}^{\text{GAIP}_{\text{IG}}}(\lambda)}$$

for some universal constants $C_1, C_2 > 0$.

C Background on Key Encapsulation Mechanisms

Our construction of the GASDVS scheme for strong designated verifier signatures with non-delegatability requires a Key Encapsulation Mechanism (KEM), which we define and explain in this section.

Definition 47 (Key-Encapsulation Mechanism). A Key-Encapsulation Mechanism (KEM) is a tuple of PPT algorithms $(\text{Gen}, \text{Encaps}, \text{Decaps})$. On input a security parameter λ in $\mathbb{N}$, the generation algorithm Gen outputs a keypair (pk, sk) . On input pk , the encapsulation algorithm Encaps outputs a shared key K in $\{0, 1\}^\lambda$ together with an encapsulation E . On input (sk, E) , the decapsulation algorithm Decaps outputs K in $\{0, 1\}^\lambda$.

Definition 48 (Correctness of Key-Encapsulation Mechanisms). A key-encapsulation mechanism $\text{KEM} = (\text{Gen}, \text{Encaps}, \text{Decaps})$ satisfies correctness if there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any λ in $\mathbb{N}$

$$\Pr \left[K \leftarrow \text{Decaps}(\text{sk}, E) \mid \begin{array}{l} (\text{pk}, \text{sk}) \xleftarrow{\$} \text{Gen}(1^\lambda) \\ (K, E) \xleftarrow{\$} \text{Encaps}(\text{pk}) \end{array} \right] = 1 - \text{negl}(\lambda).$$

Definition 49 (Session-Key CCA Privacy of Key-Encapsulation Mechanisms [BdPP23]). A KEM $\text{KEM} = (\text{Gen}, \text{Encaps}, \text{Decaps})$ satisfies session key indistinguishability against chosen ciphertext attacks (SK-CCA) if for any PPT algorithm $\mathcal{A}$, there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any λ in $\mathbb{N}$

$$\left| \Pr \left[\text{Exp}_{\mathcal{A}}^{\text{SK-CCA}}(\lambda) - \frac{1}{2} \right] \right| \leq \text{negl}(\lambda).$$

for $\text{Exp}_{\mathcal{A}}^{\text{SK-CCA}}$ the experiment described in Figure 6.

An SK-CCA-secure KEM relying on the ac- Δ LIP may be obtained by applying the FO transform [FO13] to the KEM provided in [DvW22, Section 5].

$\text{Exp}_{\mathcal{A}}^{\text{SK-CCA}}(\lambda)$	$\mathcal{O}^{\text{Decaps}}(E)$
$(\text{pk}, \text{sk}) \xleftarrow{\$} \text{Gen}(1^\lambda) \quad (K^*, E^*) \xleftarrow{\$} \text{Encaps}(\text{pk})$ $b \xleftarrow{\$} \{0, 1\}; \text{ if } b = 0 \text{ then } \hat{K} \leftarrow K \text{ else } \hat{K} \xleftarrow{\$} \{0, 1\}^\lambda$ $b' \xleftarrow{\$} \mathcal{A}^{\mathcal{O}^{\text{Decaps}}}(\text{pk}, \hat{K}, E^*)$ if $b' = b$ then return 1, else return 0	if $E = E^*, K \leftarrow \perp$ else $, K \leftarrow \text{Decaps}(\text{sk}, E)$ return K

Figure 6. The session-key CCA privacy (SK-CCA) game for key-encapsulation mechanisms.

D Background on Symmetric Key Cryptography

Our construction of the GASDVS scheme for strong designated verifier signatures with non-delegatability requires Symmetric Key Encryption (SKE), which we define and explain in this section.

Definition 50 (Symmetric Key Encryption Scheme). A symmetric key encryption scheme is a tuple of PPT algorithms $(\text{KeyGen}, \text{Enc}, \text{Dec})$. Let $\lambda \in \mathbb{N}$ be the security parameter. On input 1^λ , the generation algorithm KeyGen outputs a uniform random key $\text{sk} \xleftarrow{\$} \{0, 1\}^\lambda$. On input $\text{sk} \in \{0, 1\}^\lambda$ and a message $m \in \{0, 1\}^*$, $\text{Enc}(\text{sk}, m)$ outputs a ciphertext $c \in \{0, 1\}^*$. On input $\text{sk} \in \{0, 1\}^\lambda$ and a ciphertext $c \in \{0, 1\}^*$, $\text{Dec}(\text{sk}, c)$ outputs a message $m \in \{0, 1\}^*$.

Definition 51 (Correctness of SKE). A symmetric key encryption scheme $\text{SKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ satisfies perfect correctness if for any $\lambda \in \mathbb{N}$, any $\text{sk} \in \{0, 1\}^\lambda$ and any message $m \in \{0, 1\}^*$

$$\text{Dec}(\text{sk}, \text{Enc}(\text{sk}, m)) = m.$$

Definition 52 (Eavesdropper Indistinguishability). A symmetric key encryption scheme SKE satisfies Eavesdropper Indistinguishability (IND-EAV) if for any PPT algorithm $\mathcal{A}$, there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any $\lambda \in \mathbb{N}$

$$\left| \Pr \left[\text{Exp}_{\mathcal{A}}^{\text{IND-EAV}}(\lambda) - \frac{1}{2} \right] \right| \leq \text{negl}(\lambda).$$

for $\text{Exp}_{\mathcal{A}}^{\text{IND-EAV}}$ the experiment described in Figure 7.

$\text{Exp}_{\mathcal{A}}^{\text{IND-EAV}}(\lambda)$
$(\text{sk}) \xleftarrow{\$} \text{KeyGen}(1^\lambda) \quad (m_0, m_1) \xleftarrow{\$} \mathcal{A}(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\} \quad c \xleftarrow{\$} \text{Enc}(\text{sk}, m_b) \quad b' \xleftarrow{\$} \mathcal{A}(c)$ if $b' = b$ then return 1, else return 0

Figure 7. The IND-EAV security game for secret-key encryption.

E The Forking Lemma

The proofs for strong unforgeability and non-delegatability of the GASDVS scheme require the following forking lemma.

Definition 53 (Forking Lemma, adapted from [BN06], Lemma 1). *Let q be in $\mathbb{N}^*$, H be a set of at least two elements, and $\mathcal{W}$ be a PPT algorithm that takes as input $(x, h_1, \dots, h_q)$, and runs on random coins ρ , outputting j in $[q]$ and some auxiliary value out . Let IG be a generator that on input a security parameter λ generates an input x for $\mathcal{W}$, $\text{Fork}^{\mathcal{W}}$ be the forking algorithms for $\mathcal{W}$ as defined in Figure 8,*

$$\text{acc}(\lambda) := \Pr \left[j \geq 1 \mid x \xleftarrow{\$} \text{IG}(1^\lambda), h_1, \dots, h_q \xleftarrow{\$} H, (j, \text{out}) \xleftarrow{\$} \mathcal{W}(x, h_1, \dots, h_q) \right]$$

$$\text{and} \quad \text{frk}(\lambda) := \Pr \left[b = 1 \mid x \xleftarrow{\$} \text{IG}(1^\lambda), (b, \text{out}, \tilde{\text{out}}) \xleftarrow{\$} \text{Fork}^{\mathcal{W}}(x) \right].$$

Then, $\text{frk}(\lambda) \geq \text{acc}(\lambda) \cdot \left(\frac{\text{acc}(\lambda)}{q} - \frac{1}{|H|} \right)$.

$\text{Fork}^{\mathcal{W}}(x)$
Picks random coins ρ for $\mathcal{W}$.
Samples $h_1, \dots, h_q \xleftarrow{\$} H$ and sets $(j, \text{out}) \leftarrow \mathcal{W}(x, h_1, \dots, h_q; \rho)$.
If $j = 0$, returns $(0, \perp, \perp)$.
Samples $h'_1, \dots, h'_q \xleftarrow{\$} H$ and sets $(j', \text{out}') \leftarrow \mathcal{W}(x, h_1, \dots, h_{j-1}, h'_j, \dots, h'_q; \rho)$.
If $j = j'$ and $h_j \neq h'_j$, returns $(1, \text{out}, \text{out}')$, else, $(0, \perp, \perp)$.

Figure 8. The forking algorithm for $\mathcal{W}$.

F Zero-Knowledge Proofs for GAIP Relations

F.1 Zero-Knowledge Proof for GAIP Relations

Definition 54 (Sigma Protocol for GAIP_{IG} Relations). *We define a three-way sigma protocol to prove, given a statement x , knowledge of a witness w such that (x, w) is in $\mathcal{R}_{Q, \lambda}$ satisfying Definition 22, as the tuple of algorithms $\Sigma_{Q, \lambda} = (\text{Commit}, \text{Challenge}, \text{Prove}, \text{Verify})$ described hereafter.*

$\Sigma_{Q, \lambda}.\text{Commit}(x, w) \rightarrow (\text{st}_{\text{Prove}}, \rho_{\text{Prove}_1})$: samples $(\mathbf{R}, \mathbf{U}) \xleftarrow{\$} \text{IG}(Q)^\lambda$ with λ draws of $\text{IG}(Q)$, and outputs $(\text{st}_{\text{Prove}_1} := \mathbf{U}, \rho_{\text{Prove}_1} := \mathbf{R})$.

$\Sigma_{Q, \lambda}.\text{Challenge}(x, \rho_{\text{Prove}_1}) \rightarrow \rho_{\text{Verify}}$: on input $(x, \rho_{\text{Prove}_1})$, samples $\mathbf{c} \xleftarrow{\$} \{0, 1\}^\lambda$ and outputs $\rho_{\text{Verify}} := \mathbf{c}$.

$\Sigma_{Q,\lambda}.\text{Prove}(x, w, \text{st}_{\text{Prove}}, \rho_{\text{Verify}}) \rightarrow \rho_{\text{Prove}_2}$: on input $(x = Q', w = A, \text{st}_{\text{Prove}_1} = U, \rho_{\text{Verify}} = \mathbf{c})$, the algorithm sets $V_i = A^{-c_i} \cdot U_i$ for each $i \in [\lambda]$, sets $\mathbf{V} := (V_i)_{i \in [\lambda]}$, and outputs $\rho_{\text{Prove}_2} := \mathbf{V}$.

$\Sigma_{Q,\lambda}.\text{Verify}(x, \rho_{\text{Verify}}, \rho_{\text{Prove}_2}) \rightarrow b$: on input $(x = Q', \text{st}_{\text{Verify}} = \mathbf{c}, \rho_{\text{Prove}_2} = \mathbf{V})$, returns $b \leftarrow 1$ if for each $i \in [\lambda]$, V_i is in $\mathbb{G}$, $R = Q * V_i$ if $c_i = 0$, and $R = Q' * V_i$ if $c_i = 1$, and outputs $b \leftarrow 0$ otherwise.

Theorem 55. *The $\Sigma_{Q,\lambda}$ protocol satisfies correctness, special soundness with error $2^{-\lambda}$, and perfect honest verifier zero-knowledge.*

Proof. The proofs for these properties are analogous to those in [DvW22, Section 4.1], but generalised to the GAIP_{IG} setting, and detailed in the rest of this subsection.

Definition 56. *The sigma protocol in Definition 54 can be made non-interactive using the Fiat-Shamir transform [FS87]. Let $\Pi_{Q,\lambda} = (\text{ComPr}, \text{Verify})$ denote the commit and prove scheme for which on input a statement Q' and witness A for the GAIP_{IG}^Q relation, ComPr generates a non-interactive proof of knowledge, which may be checked with Verify.*

In this section we discuss proofs for the properties of $\Sigma_{Q,\lambda}^{\text{GAIP}}$ (Theorem 55).

Theorem 57. *The $\Sigma_{Q,\lambda}^{\text{GAIP}}$ protocol satisfies correctness.*

Proof. Let $x := Q' \in [Q]$ be a statement and let $w := A \in \mathbb{G}$ be a witness for x , i.e. $Q' = Q * A$. We show that there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Verify}(x, \mathbf{c}, \mathbf{V}) = 1 \mid \begin{array}{l} (\mathbf{R}, \mathbf{U}) \xleftarrow{\$} \text{Commit}(x, w) \\ \mathbf{c} \xleftarrow{\$} \text{Challenge}(x, \mathbf{R}) \\ \mathbf{V} \leftarrow \text{Prove}(x, w, \mathbf{U}, \mathbf{c}) \end{array} \right] = 1 - \text{negl}(\lambda).$$

We consider the values $(\mathbf{R}, \mathbf{U}) \xleftarrow{\$} \text{Commit}(x, w)$, $\mathbf{c} \xleftarrow{\$} \text{Challenge}(x, \mathbf{R})$ and $\mathbf{V} \leftarrow \text{Prove}(x, w, \mathbf{U}, \mathbf{c})$. Let $\epsilon_{\text{IG}}^{\text{Fail}}$ denote the probability that IG outputs $\perp$. Note that $\epsilon_{\text{IG}}^{\text{Fail}}$ is negligible in λ .

For all $i \in [\lambda]$, $R_i = Q * U_i$ as $(R_i, U_i) \xleftarrow{\$} \text{IG}(Q)$. Furthermore, $\mathbf{c}$ is in $\{0, 1\}^\lambda$ and $\mathbf{V}$ is computed as $V_i = A^{-c_i} \cdot U_i$. $U_i \in \mathbb{G}$ and $A \in \mathbb{G}$, so it follows that $V_i \in \mathbb{G}$. Recall that $Q' = Q * A$. We also see that

$$\begin{aligned} R = Q * U_i &= \begin{cases} Q * U_i & \text{if } c_i = 0 \\ Q * (A \cdot A^{-c_i} \cdot U_i) & \text{if } c_i = 1 \end{cases} \\ &= \begin{cases} Q * V_i & \text{if } c_i = 0 \\ Q' * V_i & \text{if } c_i = 1 \end{cases}. \end{aligned}$$

It follows that $\text{Verify}(x, \mathbf{c}, \mathbf{V}) = 1$ if all calls to IG are successful.

Execution of $\Sigma_{Q,\lambda}^{\text{GAIP}}$ requires λ calls to IG. It follows that

$$\Pr \left[\text{Verify}(x, \mathbf{c}, \mathbf{V}) = 1 \mid \begin{array}{l} (\mathbf{R}, U) \xleftarrow{\$} \text{Commit}(x, w) \\ \mathbf{c} \xleftarrow{\$} \text{Challenge}(x, \mathbf{R}) \\ \mathbf{V} \leftarrow \text{Prove}(x, w, U, \mathbf{c}) \end{array} \right] = 1 - \lambda \cdot \epsilon_{\text{IG}}^{\text{Fail}}.$$

□

Theorem 58. *The $\Sigma_{Q,\lambda}^{\text{GAIP}}$ protocol satisfies special soundness with soundness error $2^{-\lambda}$.*

Proof. We consider proofs for statements $Q' \in [Q]$. Let $(\mathbf{R}, \mathbf{c}, \mathbf{V})$ and $(\mathbf{R}, \mathbf{c}', \mathbf{V}')$ be two accepting transcripts such that $\mathbf{c} \neq \mathbf{c}'$. We provide an extractor algorithm Ext that extracts a witness $U \in \mathbb{G}$ such that $Q' = Q * U$.

Ext $((\mathbf{R}, \mathbf{c}, \mathbf{V}), (\mathbf{R}, \mathbf{c}', \mathbf{V}')) \rightarrow U$: On input two accepting transcripts $(\mathbf{R}, \mathbf{c}, \mathbf{V})$ and $(\mathbf{R}, \mathbf{c}', \mathbf{V}')$, the algorithm proceeds as follows:

1. Try to find an index i such that $c_i \neq c'_i$, else abort.
2. If $c_i = 0$ and $c'_i = 1$, return $U := V_i \cdot V_i'^{-1}$.
3. If $c_i = 1$ and $c'_i = 0$, return $U := V_i' \cdot V_i^{-1}$.

We show that Ext indeed extracts a valid witness U . Consider two accepting transcripts $(\mathbf{R}, \mathbf{c}, \mathbf{V}), (\mathbf{R}', \mathbf{c}', \mathbf{V}')$ such that $\mathbf{c} \neq \mathbf{c}'$. Let $i \in [\lambda]$ such that $c_i \neq c'_i$.

If $c_i = 0$, it must hold that $c'_i = 1$ and as $(\mathbf{R}, \mathbf{c}, \mathbf{V})$ and $(\mathbf{R}, \mathbf{c}', \mathbf{V}')$ are both accepting we find that

$$Q * V_i = R_i = Q' * V_i' = Q * A * V_i'.$$

It follows that $Q * (V_i \cdot V_i'^{-1}) = Q * A = Q'$, so that $V_i \cdot V_i'^{-1}$ is an witness for Q' .

If $c_i = 1$, it must hold that $c'_i = 0$ and as $(\mathbf{R}, \mathbf{c}, \mathbf{V})$ and $(\mathbf{R}, \mathbf{c}', \mathbf{V}')$ are both accepting we find that

$$Q * V_i' = R_i = Q' * V_i = Q * A * V_i.$$

It follows that $Q * (V_i' \cdot V_i^{-1}) = Q * A = Q'$, so that $V_i' \cdot V_i^{-1}$ is an witness for Q' .

We conclude that Ext indeed extracts a valid witness for Q' . The success of Ext, does rely on the fact that $\mathbf{c} \neq \mathbf{c}'$. For honestly generated transcripts, $\mathbf{c}$ and $\mathbf{c}'$ are drawn uniformly at random from $\{0, 1\}^\lambda$. Therefore, the probability that $\mathbf{c} = \mathbf{c}'$ under honest generation is $\frac{1}{2^\lambda}$. It follows that the knowledge soundness of $\Sigma_{Q,\lambda}^{\text{GAIP}}$ is $2^{-\lambda}$.

□

Theorem 59. *The $\Sigma_{Q,\lambda}^{\text{GAIP}}$ protocol satisfies perfect honest-verifier zero-knowledge.*

Proof. We construct a simulator Sim that on input a statement $x := Q' \in [Q]$ produces an accepting transcript for $\Sigma_{Q,\lambda}^{\text{GAIP}}$ according to the same distribution as an honest prover.

$\text{Sim}(Q') \rightarrow (\mathbf{R}, \mathbf{c}, \mathbf{V})$: On input Q' ,

1. samples $\mathbf{c} \xleftarrow{\$} \{0, 1\}$;
2. for each $i \in [\lambda]$, samples $(R_i, V_i) \xleftarrow{\$} \begin{cases} \text{IG}(Q) & \text{if } c_i = 0 \\ \text{IG}(Q') & \text{if } c_i = 1 \end{cases}$;
3. returns $(\mathbf{R}, \mathbf{c}, \mathbf{V})$.

By inspection, Sim outputs accepting transcripts. Let $(\mathbf{R}, \mathbf{c}, \mathbf{V})$ be an honestly generated transcript for the statement Q' and let $(\mathbf{R}', \mathbf{c}', \mathbf{V}') \xleftarrow{\$} \text{Sim}(Q')$. Both $\mathbf{c}$ and $\mathbf{c}'$ are drawn independently and uniformly at random from $\{0, 1\}^\lambda$, so they are equally distributed. Furthermore, for $i \in [\lambda]$, R_i is computed as $(R_i, U_i) \xleftarrow{\$} \text{IG}(Q)$, and R'_i as $(R'_i, U'_i) \xleftarrow{\$} \text{IG}(Q)$ or $(R'_i, U'_i) \xleftarrow{\$} \text{IG}(Q')$. As IG is an instance generator (Definition 3) and $Q' \in [Q]$, so $[Q] = [Q']$, one finds that $\mathbf{R}$ and $\mathbf{R}'$ are also equally distributed, and independent of the sampling of $\mathbf{c}$ and $\mathbf{c}'$. Lastly, the values of $\mathbf{V}$ and $\mathbf{V}'$ are uniquely fixed by $(\mathbf{R}, \mathbf{c})$ respectively $(\mathbf{R}', \mathbf{c}')$ under the constraint that $(\mathbf{R}, \mathbf{c}, \mathbf{V})$ and $(\mathbf{R}', \mathbf{c}', \mathbf{V}')$ be accepting transcripts. $(\mathbf{R}, \mathbf{c})$ and $(\mathbf{R}', \mathbf{c}')$ are independently and equally distributed, so $\mathbf{V}$ and $\mathbf{V}'$ are also independently and equally distributed. Finally, the distribution of $\text{Sim}(Q')$ is exactly the same as the one of an honest-verifier accepting transcript. $\square$

F.2 Correctness of $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$

Theorem 60 (Correctness for $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$). *The $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ protocol satisfies correctness, i.e. there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for $\lambda \in \mathbb{N}$ and for any valid statement witness pair (x, w) ,*

$$\Pr \left[\begin{array}{l} \text{Verify}(x, \rho_{\text{Prove}_1}, \rho_{\text{Verify}}, \rho_{\text{Prove}_2}) \\ = 1 \end{array} \middle| \begin{array}{l} (\text{st}_{\text{Prove}}, \rho_{\text{Prove}_1}) \xleftarrow{\$} \text{Commit}(x, w) \\ \rho_{\text{Verify}} \xleftarrow{\$} \text{Challenge}(x, \rho_{\text{Prove}}) \\ \rho_{\text{Prove}_2} \leftarrow \text{Prove}(x, w, \text{st}_{\text{Prove}}, \rho_{\text{Verify}}) \end{array} \right] \\ = 1 - \text{negl}(\lambda).$$

Proof. Let (x, w) be a statement witness pair, so $x = (P_0, P_1) \in [Q_0] \times [Q_1]$ and $w = (\delta, A_\delta) \in \{0; 1\} \times \mathbb{G}$ with $P_\delta = Q_\delta * A_\delta$. Let $\epsilon_{\text{IG}}^{\text{Fail}}$ denote the probability that IG outputs $\perp$. Note that $\epsilon_{\text{IG}}^{\text{Fail}}$ is negligible in λ . $\text{Verify}(x, \rho_{\text{Prove}_1}, \rho_{\text{Verify}}, \rho_{\text{Prove}_2}) = 1$ precisely when

$$\forall i \in [\lambda], \quad c_{0,i} \cdot c_{1,i} = c_i \quad \text{and} \\ \forall b \in \{0, 1\} \text{ and } \forall i \in [\lambda], \quad \begin{cases} P_b * U_{b,i} = Y_{b,i} & \text{if } c_{b,i} = -1 \\ Q_b * U_{b,i} = Y_{b,i} & \text{if } c_{b,i} = 1 \end{cases},$$

where $\rho_{\text{Prove}_1} = (\mathbf{Y}_0, \mathbf{Y}_1)$, $\rho_{\text{Verify}} = \mathbf{c}$ and $\rho_{\text{Prove}_2} = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$.

During the execution of the protocol $\mathbf{c}_{1-\delta}$ is sampled from $\{-1, 1\}^\lambda$ and $\mathbf{c}_{1-\delta}$ is computed as $\mathbf{c}_\delta = (c_k \cdot c_{1-\delta,k})_{k \in [\lambda]}$. It follows directly that $\forall i \in [\lambda]$, $c_{0,i} \cdot c_{1,i} = c_i$.

Furthermore, for $i \in [\lambda]$, $Y_{1-\delta,i}$ and $U_{1-\delta,i}$ are computed as

$$\begin{cases} (Y_{1-\delta,i}, U_{1-\delta,i}) \stackrel{\$}{\leftarrow} \text{IG}(P_{1-\delta}) & \text{if } c_{1-\delta,i} = -1 \\ (Y_{1-\delta,i}, U_{1-\delta,i}) \stackrel{\$}{\leftarrow} \text{IG}(Q_{1-\delta}) & \text{if } c_{1-\delta,i} = 1 \end{cases}$$

Hence $P_{1-\delta} * U_{1-\delta,i} = Y_{1-\delta,i}$ if $c_{1-\delta,i} = -1$ and $Q_{1-\delta} * U_{1-\delta,i} = Y_{1-\delta,i}$ if $c_{1-\delta,i} = 1$.

We know that $Y_{\delta,i}$ and $U_{\delta,i}$ are computed as

$$(Y_{\delta,i}, H_{\delta,i}) \stackrel{\$}{\leftarrow} \text{IG}(Q_{\delta}) \quad \text{and} \quad U_{\delta,i} = \begin{cases} A_{\delta}^{-1} \cdot H_{\delta,i} & \text{if } c_{\delta,i} = -1 \\ H_{\delta,i} & \text{if } c_{\delta,i} = 1 \end{cases}.$$

As $Q_{\delta} * H_{\delta,i} = Y_{\delta,i}$,

$$\begin{cases} Y_{\delta,i} = Q_{\delta} * H_{\delta,i} = (Q_{\delta} * A_{\delta}) * (A_{\delta}^{-1} H_{\delta,i}) = P_{\delta} * U_{\delta,i} & \text{if } c_{\delta,i} = -1 \\ Y_{\delta,i} = Q_{\delta} * H_{\delta,i} = Q_{\delta} * U_{\delta,i} & \text{if } c_{\delta,i} = 1 \end{cases},$$

and for any $b \in \{0, 1\}$ and any $i \in [\lambda]$, if $c_{b,i} = -1$ then $P_b * U_{b,i} = Y_{b,i}$, and if $c_{b,i} = 1$ then $Q_b * U_{b,i} = Y_{b,i}$.

Finally, if the protocol is honestly executed and all calls to **IG** are successful, then

$$\text{Verify}(x, \rho_{\text{Prove}_1}, \rho_{\text{Verify}}, \rho_{\text{Prove}_2}) = 1.$$

Execution of $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ requires 2λ calls to **IG**. It follows that

$$\begin{aligned} \Pr \left[\begin{array}{l} \text{Verify}(x, \rho_{\text{Prove}_1}, \rho_{\text{Verify}}, \rho_{\text{Prove}_2}) \\ = 1 \end{array} \right] & \left| \begin{array}{l} (\text{st}_{\text{Prove}}, \rho_{\text{Prove}_1}) \stackrel{\$}{\leftarrow} \text{Commit}(x, w) \\ \rho_{\text{Verify}} \stackrel{\$}{\leftarrow} \text{Challenge}(x, \rho_{\text{Prove}}) \\ \rho_{\text{Prove}_2} \leftarrow \text{Prove}(x, w, \text{st}_{\text{Prove}}, \rho_{\text{Verify}}) \end{array} \right| \\ & = 1 - 2\lambda \cdot \epsilon_{\text{IG}}^{\text{Fail}}. \end{aligned}$$

□

F.3 Extractors and the Special Soundness of $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$

Lemma 61. *Let $\mathcal{R}_{Q_0, Q_1, \lambda}^{\text{OR}}$ be a GAIP_{IG} OR relation satisfying Definition 23 for the group action $*$: $\mathcal{Q} \times \mathbb{G} \rightarrow \mathcal{Q}$, $P_0 \in [Q_0]$, $P_1 \in [Q_1]$, $c_0, c_1, c'_0, c'_1 \in \{0, 1\}^\lambda$, $U_0, U_1, U'_0, U'_1 \in \mathbb{G}^\lambda$, $\sigma := (c_0, c_1, U_0, U_1)$ and $\sigma' := (c'_0, c'_1, U'_0, U'_1)$. For $b \in \{0, 1\}$ and $i \in [\lambda]$ let*

$$Y_{b,i} := \begin{cases} P_b * U_{b,i} & \text{if } c_{b,i} = -1 \\ Q_b * U_{b,i} & \text{if } c_{b,i} = 1 \end{cases} \quad \text{and} \quad Y'_{b,i} := \begin{cases} P_b * U'_{b,i} & \text{if } c'_{b,i} = -1 \\ Q_b * U'_{b,i} & \text{if } c'_{b,i} = 1 \end{cases}$$

*be such that $Y_0 = Y'_0$ and $Y_1 = Y'_1$. For $b \in \{0, 1\}$ and Ext_b the algorithm described in Figure 5, if $c_b \neq c'_b$, then $\text{Ext}_b(\sigma, \sigma')$ outputs A such that $Q_b * A = P_b$.*

Proof. Suppose that $\mathbf{c}_b \neq \mathbf{c}'_b$. Let i be an index of $[\lambda]$ such that $c_{b,i} \neq c'_{b,i}$. If $c_{b,i} = -1$, then $\text{Ext}(\sigma, \sigma')$ outputs $A := U'_{b,i} \cdot U_{b,i}^{-1}$. In this case $c'_{b,i} = 1$. As $P_b * U_{b,i} = Y_{b,i} = Y'_{b,i} = Q_b * U'_{b,i}$, one has $Q_b * (U'_{b,i} \cdot U_{b,i}^{-1}) = P_b$. If $c_{b,i} = 1$, then $\text{Ext}(\sigma, \sigma')$ outputs $A := U_{b,i} \cdot U'^{-1}_{b,i}$. In this case $c'_{b,i} = -1$. As $P_b * U'_{b,i} = Y_{b,i} = Y'_{b,i} = Q * U_{b,i}$, one has $Q * (U_{b,i} \cdot U'^{-1}_{b,i}) = P_b$. $\square$

We give a proof for Theorem 25, which states that $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ satisfies special soundness.

Proof. Let $T := (\mathbf{Y}_0, \mathbf{Y}_1, \mathbf{c}, \mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$ and $T' := (\mathbf{Y}_0, \mathbf{Y}_1, \mathbf{c}', \mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1)$ be two accepting transcripts for a statement $x = (P_0, P_1) \in [Q_0] \times [Q_1]$ such that $\mathbf{c} \neq \mathbf{c}'$. We provide an algorithm $\mathcal{E}$ that extracts a witness w such that $(x, w) \in \mathcal{R}_{Q_0, Q_1, \lambda}^{\text{OR}}$. Let $\sigma := (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$ and $\sigma' := (\mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1)$.

The fact that both transcripts are accepting gives us that $\forall i \in [\lambda], c_{0,i} \cdot c_{1,i} = c_i$ and $\forall i \in [\lambda], c'_{0,i} \cdot c'_{1,i} = c'_i$. As $\mathbf{c} \neq \mathbf{c}'$ follows that there must exist $b \in \{0, 1\}$ and $k \in [\lambda]$ such that $c_{b,k} \neq c'_{b,k}$. First, the algorithm $\mathcal{E}$ finds values $b \in \{0, 1\}$ and $k \in [\lambda]$ such that $c_{b,k} \neq c'_{b,k}$. It then runs $A \leftarrow \text{Ext}_b(\sigma, \sigma')$ as defined in Figure 5 and returns (b, A) . We verify that $\mathcal{E}$ actually extracts a witness w for x . Both the transcripts T and T' are accepting, so for $b \in \{0, 1\}$,

$$\begin{cases} P_b * \mathbf{U}_{b,i} & \text{if } c_{b,i} = -1 \\ Q_b * \mathbf{U}_{b,i} & \text{if } c_{b,i} = 1 \end{cases} = Y_{b,i} = \begin{cases} P_b * \mathbf{U}'_{b,i} & \text{if } c'_{b,i} = -1 \\ Q_b * \mathbf{U}'_{b,i} & \text{if } c'_{b,i} = 1 \end{cases}.$$

The extractor $\mathcal{E}$ runs Ext_b for a value b such that $\mathbf{c}_b \neq \mathbf{c}'_b$. Following Lemma 61, for $A \leftarrow \text{Ext}_b(\sigma, \sigma')$, $Q_b * A = P_b$. Therefore, $w := (b, A)$ is a valid witness for $x := (P_0, P_1)$.

The success of $\mathcal{E}$ relies on the fact that $\mathbf{c} \neq \mathbf{c}'$. For honestly generated transcripts, $\mathbf{c}$ and $\mathbf{c}'$ are drawn uniformly at random from $\{0, 1\}^\lambda$, and the probability that $\mathbf{c} = \mathbf{c}'$ is $\frac{1}{2^\lambda}$. Hence, the knowledge soundness of $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ is $2^{-\lambda}$. $\square$

F.4 $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ is Honest-Verifier Zero Knowledge

We give a proof for Theorem 26, which states that $\Sigma_{Q_0, Q_1, \lambda}^{\text{OR}}$ satisfies perfect honest-verifier zero knowledge.

Proof. We first define the algorithm Sim .

$\text{Sim}(x, \lambda) \rightarrow (\mathbf{Y}_0, \mathbf{Y}_1, \mathbf{c}, \mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$: on input $x = (P_0, P_1) \in [Q_0] \times [Q_1]$,

1. samples $\mathbf{c} \xleftarrow{\$} \{-1, 1\}^\lambda$ and $\mathbf{c}_0 \xleftarrow{\$} \{-1, 1\}^\lambda$ and set $\mathbf{c}_1 = (c_{0,k} \cdot c_k)_{k \in [\lambda]}$;
2. for $b \in \{0, 1\}$ and $i \in [\lambda]$,

$$\text{samples } (Y_{b,i}, U_{b,i}) \xleftarrow{\$} \begin{cases} \text{IG}(P_b) & \text{if } c_{b,i} = -1; \\ \text{IG}(Q_b) & \text{if } c_{b,i} = 1 \end{cases};$$
3. outputs $(\mathbf{Y}_0, \mathbf{Y}_1, \mathbf{c}, \mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$.

Let $T = (\mathbf{Y}_0, \mathbf{Y}_1, \mathbf{c}, \mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1) \xleftarrow{\$} \text{Sim}(x, (Q_0, Q_1), \lambda, s)$ be a transcript that is generated by Sim and let $T' = (\mathbf{Y}'_0, \mathbf{Y}'_1, \mathbf{c}', \mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1)$ be an honestly generated transcript. We first show that T is an accepting transcript. It is clear that for all $i \in [\lambda]$, $c_{0,i} \cdot c_{1,i} = c_i$, due to how Sim generates $\mathbf{c}$, $\mathbf{c}_0$ and $\mathbf{c}_1$. Let $b \in \{0, 1\}$ and $i \in [\lambda]$. If $c_{b,i} = -1$, $Y_{b,i}$ and $U_{b,i}$ are generated as $(Y_{b,i}, U_{b,i}) \xleftarrow{\$} \text{IG}(P_b)$, so $Y_{b,i} = P_b * U_{b,i}$, except with probability $\epsilon_{\text{IG}}^{\text{Fail}}$ equal to the one for T' , and similarly for $Y_{b,i} = Q_b * U_{b,i}$ if $c_{b,i} = 1$. So, for all $b \in \{0, 1\}$ and $i \in [\lambda]$, $P_b * U_{b,i} = Y_{b,i}$ if $c_{b,i} = -1$, and $Q_b * U_{b,i} = Y_{b,i}$ if $c_{b,i} = 1$ and T is an accepting transcript, except with probability $2\epsilon_{\text{IG}}^{\text{Fail}}$ as in the honest case.

We still need to show that the distribution of T matches the distribution of T' . The values of $\mathbf{c}$, $\mathbf{c}_0$ are sampled independently and uniformly at random from $\{-1, 1\}^\lambda$. From the computation of $\mathbf{c}_1$ it is clear that $\mathbf{c}_0$ and $\mathbf{c}_1$ are distributed independently and uniformly in $\{-1, 1\}$, as are their counterparts in T' . We know for any $(X_b, V) \xleftarrow{\$} \text{IG}(Q_b)$ and $(X'_b, V') \xleftarrow{\$} \text{IG}(P_b)$ that X_b and X'_b are equally distributed as $[P_b] = [Q_b]$. Therefore, the entries for $\mathbf{Y}_0, \mathbf{Y}'_0$ are all distributed equally, as are the entries of $\mathbf{Y}'_1, \mathbf{Y}_1$. Furthermore, these distributions are independent from each other and they are also independent from $\mathbf{c}$, $\mathbf{c}_0$, $\mathbf{c}_1$, $\mathbf{c}'$, $\mathbf{c}'_0$ and $\mathbf{c}'_1$. Since the group action is free and accepting transcripts must satisfy the equations

$$\begin{cases} P_b * U_{b,i} = Y_{b,i} & \text{if } c_{b,i} = -1 \\ Q_b * U_{b,i} = Y_{b,i} & \text{if } c_{b,i} = 1 \end{cases},$$

for $b \in \{0, 1\}$, we find that the values of $\mathbf{Y}_b$ and $\mathbf{c}_b$ fix precisely the value of $\mathbf{U}_b$ under the constraint that $(\mathbf{Y}_0, \mathbf{Y}_1, \mathbf{c}, \mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$ is an accepting transcript. It follows that for $b \in \{0, 1\}$, the distributions on $\mathbf{c}_b$ and $\mathbf{Y}_b$ fix the distribution on $\mathbf{U}_b$, while the distributions on $\mathbf{c}'_b$ and $\mathbf{Y}'_b$ fix the distribution on $\mathbf{U}'_b$. Therefore, the entries for $\mathbf{U}_0, \mathbf{U}'_0$ are all distributed equally, as are the entries of $\mathbf{U}'_1, \mathbf{U}_1$, and again they are independent from each other. We conclude that the distribution on the output transcript T of Sim matches the distribution on the honestly generated transcript T' .

□

F.5 Auxiliary Lemma for the One-More Unforgeability Proof of GABS

Lemma 62. *Consider the non-interactive proof protocol $\Pi_{Q_0, Q_1, \lambda}^{\text{OR}}$ from Definition 27. Let $((P_0, P_1), (\delta, A_\delta)) \in \mathcal{R}_{Q_0, Q_1, \lambda}^{\text{OR}}$ be a statement-witness pair and let π be any accepting proof for the statement (P_0, P_1) . There is a unique collection of randomness rand under which π is obtained from $\Pi_{Q_0, Q_1, \lambda}^{\text{OR}}((P_0, P_1), \delta, A_\delta)$.*

Proof. Let $\mathcal{H}$ be the hash function that is used for the Fiat-Shamir transformation resulting in $\Pi_{Q_0, Q_1, \lambda}^{\text{OR}}$. Let $((P_0, P_1), (\delta, A_\delta)) \in \mathcal{R}_{Q_0, Q_1, \lambda}^{\text{OR}}$ be a statement-witness pair and let $\pi = (\mathbf{Y}_0, \mathbf{Y}_1, \mathbf{c} = \mathcal{H}(\mathbf{Y}_0, \mathbf{Y}_1), \mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$ be any accepting proof for the statement (P_0, P_1) . We define internal randomness $\text{rand} := (\mathbf{H}_\delta, \mathbf{c}_{1-\delta}, \mathbf{U}_{1-\delta})$ where $\mathbf{H}_\delta$ is determined by the condition that for all $i \in [\lambda]$,

$Y_{\delta,i} = Q_{\delta} * H_{\delta,i}$. Note that this uniquely defines $\mathbf{H}_{\delta}$ since we use a free group action. The values $\mathbf{c}_{1-\delta}$ and $\mathbf{U}_{1-\delta}$ are taken directly from π .

We first show that $\Pi_{Q_0, Q_1, \lambda}^{\text{OR}}((P_0, P_1), \delta, A_{\delta})$ produces π under the randomness **rand**. Suppose that $\Pi_{Q_0, Q_1, \lambda}^{\text{OR}}((P_0, P_1), \delta, A_{\delta})$ produces

$$\pi' = (\mathbf{Y}'_0, \mathbf{Y}'_1, \mathbf{c}' := \mathcal{H}(\mathbf{Y}'_0, \mathbf{Y}'_1), \mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1).$$

The value $\mathbf{Y}'_{\delta}$ is computed as $Y'_{\delta,i} = Q_{\delta} * H_{\delta,i}$ for all $i \in [\lambda]$, while also $Y_{\delta,i} = Q_{\delta} * H_{\delta,i}$. So, we find that $\mathbf{Y}'_{\delta} = \mathbf{Y}_{\delta}$. Furthermore, $\mathbf{c}'_{1-\delta} = \mathbf{c}_{1-\delta}$ and $\mathbf{U}'_{1-\delta} = \mathbf{U}_{1-\delta}$ as $\mathbf{c}_{1-\delta}$ and $\mathbf{U}_{1-\delta}$ are directly taken from π . $\mathbf{Y}'_{1-\delta}$ is computed as

$$Y'_{1-\delta,i} = \begin{cases} P_{1-\delta} * U'_{1-\delta,i} & \text{if } c'_{1-\delta,i} = -1 \\ Q_{1-\delta} * U'_{1-\delta,i} & \text{if } c'_{1-\delta,i} = 1 \end{cases}$$

for $i \in [\lambda]$, while

$$Y_{1-\delta,i} = \begin{cases} P_{1-\delta} * U_{1-\delta,i} & \text{if } c_{1-\delta,i} = -1 \\ Q_{1-\delta} * U_{1-\delta,i} & \text{if } c_{1-\delta,i} = 1 \end{cases}$$

as π is a valid proof. It follows that $\mathbf{Y}'_{1-\delta} = \mathbf{Y}_{1-\delta}$ and thus

$$\mathbf{c}' = \mathcal{H}(\mathbf{Y}'_0, \mathbf{Y}'_1) = \mathcal{H}(\mathbf{Y}_0, \mathbf{Y}_1) = \mathbf{c}.$$

Now, $\mathbf{c}'_{\delta}$ is computed as $c'_{\delta,i} = c'_i \cdot c'_{1-\delta,i}$ for $i \in [\lambda]$, while $c_{\delta,i} \cdot c_{1-\delta,i} = c_{0,i} \cdot c_{0,i} = c_i$ since π is a valid proof. Recall that $c'_i = c_i$, $c'_{1-\delta,i} = c_{1-\delta,i} \in \{-1, 1\}$, so it follows that for all $i \in [\lambda]$, $c'_{\delta,i} = c_{\delta,i}$ and thus $\mathbf{c}'_{\delta} = \mathbf{c}_{\delta}$. Lastly, for $i \in [\lambda]$, $\mathbf{U}'_{\delta}$ is computed as

$$U'_{\delta,i} = \begin{cases} A_{\delta}^{-1} \cdot H_{\delta,i} & \text{if } c'_{\delta,i} = -1 \\ H_{\delta,i} & \text{if } c'_{\delta,i} = 1 \end{cases}$$

and

$$\begin{cases} Q_{\delta} * H_{\delta,i} = Y_{\delta,i} = P_{\delta} * U_{\delta,i} = Q_{\delta} * (A_{\delta} \cdot U_{\delta,i}) & \text{if } c_{\delta,i} = -1 \\ Q_{\delta} * H_{\delta,i} = Y_{\delta,i} = Q_{\delta} * U_{\delta,i} & \text{if } c_{\delta,i} = 1 \end{cases}$$

since π is a valid proof. From the freeness of the group action it follows that

$$U_{\delta,i} = \begin{cases} A_{\delta}^{-1} \cdot H_{\delta,i} & \text{if } c_{\delta,i} = -1 \\ H_{\delta,i} & \text{if } c_{\delta,i} = 1 \end{cases}$$

and since $\mathbf{c}'_{\delta} = \mathbf{c}_{\delta}$ we obtain that $\mathbf{U}'_{\delta} = \mathbf{U}_{\delta}$. We conclude that $\pi' = \pi$, so that $\Pi_{Q_0, Q_1, \lambda}^{\text{OR}}((P_0, P_1), \delta, A_{\delta})$ indeed produces π under the randomness **rand**.

Lastly, we show that the value of **rand** is uniquely fixed under the condition that $\Pi_{Q_0, Q_1, \lambda}^{\text{OR}}((P_0, P_1), \delta, A_{\delta})$ produces π under the randomness **rand**. Suppose

that $\mathbf{rand}' = (\mathbf{H}'_\delta, \mathbf{c}'_{1-\delta}, \mathbf{U}'_{1-\delta})$ is another set of randomness that produces π for $((P_0, P_1), \delta, A_\delta)$.

Since $\mathbf{c}'_{1-\delta}$ and $\mathbf{U}'_{1-\delta}$ are components of π , we directly obtain that $\mathbf{c}'_{1-\delta} = \mathbf{c}_{1-\delta}$ and $\mathbf{U}'_{1-\delta} = \mathbf{U}_{1-\delta}$. Furthermore, under randomness $\mathbf{rand}'$, the component $\mathbf{Y}_\delta$ is obtained under the computation $Y_{\delta,i} = Q_\delta * H'_{\delta,i}$ for $i \in [\lambda]$, while we have already seen that $Y_{\delta,i} = Q_\delta * H_{\delta,i}$. By the freeness of the group action it follows that $\mathbf{H}'_\delta = \mathbf{H}_\delta$, so that $\mathbf{rand}' = \mathbf{rand}$. $\square$

G Proofs for the GABS scheme

In this section we discuss the more lengthy or technical proofs for the security of the GABS construction.

G.1 Correctness of GABS.

Theorem 63 (Correctness of GABS). *The GABS blind signature scheme satisfies correctness; i.e., there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any λ in $\mathbb{N}$, $(\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}(1^\lambda)$, and message $m \in \{0, 1\}^*$,*

$$\Pr \left[\text{Verify}(\text{pk}, m, \sigma) = 1 \mid \begin{array}{l} (\text{st}_{\text{Sign}}, \rho_{\text{Sign},1}) \xleftarrow{\$} \text{Sign}_1(\text{sk}, \text{pk}), \\ (\text{st}_{\text{Use}}, \rho_{\text{Use}}) \xleftarrow{\$} \text{Use}_1(m, \rho_{\text{Sign},1}), \\ \rho_{\text{Sign}_2} \leftarrow \text{Sign}_2(\text{sk}, \text{st}_{\text{Sign}}, \rho_{\text{Use}}), \\ \sigma \leftarrow \text{Use}_2(\text{st}_{\text{Use}}, \rho_{\text{Sign},2}) \end{array} \right] = 1 - \text{negl}(\lambda).$$

Proof. Let λ in $\mathbb{N}$ and let $\epsilon_{\text{IG}}^{\text{Fail}}$ denote the probability, that the instance generator IG outputs $\perp$. Note that $\epsilon_{\text{IG}}^{\text{Fail}}$ is negligible in λ . Let

$$\begin{aligned} \text{sk} &= (\delta, A_\delta) \in \{0, 1\} \times [Q] \quad \text{and} \\ \text{pk} &= (\text{pk}_0 := (P_0, \pi_0, \pi_1), \text{pk}_1 := (P_1, \pi_0, \pi_1)) \end{aligned}$$

be a keypair generated by $\text{KeyGen}(1^\lambda)$. In particular, $P_\delta = Q * A_\delta$.

Let $m \in \{0, 1\}^*$ and let $\sigma = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$ be generated honestly according to the GABS protocol. We first show that $\text{Verify}(\text{pk}, m, \sigma) = 1$ if all calls to IG are successful.

In order to show that $\text{Verify}(\text{pk}, m, \sigma) = 1$, one needs to check that

$$\mathcal{H}(\mathbf{Z}_0 || \mathbf{Z}_1 || m) = (c_{0,i} \cdot c_{1,i})_{i \in [\lambda]} \tag{1}$$

where for $b \in \{0, 1\}$ and $i \in [\lambda]$

$$\begin{cases} Z_{b,i} := P_b * U_{b,i} & \text{if } c_{b,i} = -1 \\ Z_{b,i} := Q * U_{b,i} & \text{if } c_{b,i} = 1 \end{cases}.$$

Let $\mathbf{Y}_0$ and $\mathbf{Y}_1$ be as specified in the GABS protocol for the generation of σ . We first show that for $b \in \{0, 1\}$,

$$\mathbf{Z}_b = \mathbf{Y}_b. \quad (2)$$

We first consider the case $b = \delta$. Let $k \in [\lambda]$. Observe that $Y_{\delta,k}$ is computed as $(Y_{\delta,k}, V_{\delta,k}) \xleftarrow{\$} \text{IG} \left(W_{\delta,k}^{(d_{\delta,k})} \right)$, where $W_{\delta,k}^{(d_{\delta,k})}$ is computed as

$$\begin{cases} (W_{\delta,k}^{(d_{\delta,k})}, H_{\delta,k}^{(1)}) \xleftarrow{\$} \text{IG}(Q) & \text{if } d_{\delta,k} = 1 \\ (W_{\delta,k}^{(d_{\delta,k})}, H_{\delta,k}^{(-1)}) \xleftarrow{\$} \text{IG}(Q) & \text{if } d_{\delta,k} = -1 \end{cases}$$

and $d_{\delta,k} \xleftarrow{\$} \{-1, 1\}$ is sampled uniformly at random. It follows that

$$\begin{cases} Y_{\delta,k} = Q * (H_{\delta,k}^{(1)} \cdot V_{\delta,k}) & \text{if } d_{\delta,k} = 1 \\ Y_{\delta,k} = Q * (H_{\delta,k}^{(-1)} \cdot V_{\delta,k}) & \text{if } d_{\delta,k} = -1 \end{cases}. \quad (3)$$

Furthermore, $U_{\delta,k}$ is computed as

$$U_{\delta,k} = \begin{cases} U_{\delta,k}^{*(1)} \cdot V_{\delta,k} & \text{if } d_{\delta,k} = 1 \\ U_{\delta,k}^{*(-1)} \cdot V_{\delta,k} & \text{if } d_{\delta,k} = -1 \end{cases}, \quad (4)$$

where $V_{\delta,k}$ is as before and $U_{\delta,k}^{*(1)}$ and $U_{\delta,k}^{*(-1)}$ are computed as

$$\begin{aligned} U_{\delta,k}^{*(1)} &= \begin{cases} H_{\delta,k}^{(1)} & \text{if } c_{\delta,k}^* = 1 \\ A_{\delta}^{-1} \cdot H_{\delta,k}^{(1)} & \text{if } c_{\delta,k}^* = -1 \end{cases} \quad \text{and} \\ U_{\delta,k}^{*(-1)} &= \begin{cases} A_{\delta}^{-1} \cdot H_{\delta,k}^{(-1)} & \text{if } c_{\delta,k}^* = 1 \\ H_{\delta,k}^{(-1)} & \text{if } c_{\delta,k}^* = -1 \end{cases}, \end{aligned} \quad (5)$$

where $H_{\delta,k}^{(1)}$ is as before and $c_{\delta,k}^*$ is a value in $\{-1, 1\}$.

The value $c_{\delta,k}$ is computed as $c_{\delta,k} = c_{\delta,k}^* \cdot d_{\delta,k}$ and we consider all four possible values for $(d_{\delta,k}, c_{\delta,k}^*) \in \{-1, 1\}^2$ and show that the equality in (2) holds.

If $(d_{\delta,k}, c_{\delta,k}^*) = (1, 1)$, then $c_{\delta,k} = c_{\delta,k}^* \cdot d_{\delta,k} = 1$. We then find that

$$\begin{aligned} \mathbf{Y}_{\delta,k} &= Q * (H_{\delta,k}^{(1)} \cdot V_{\delta,k}) && \text{by (3)} \\ &= Q * (H_{\delta,k}^{(1)} \cdot (U_{\delta,k}^{*(1)})^{-1} \cdot U_{\delta,k}) && \text{by (4)} \\ &= Q * (U_{\delta,k}^{*(1)} \cdot (U_{\delta,k}^{*(1)})^{-1} \cdot U_{\delta,k}) && \text{by (5)} \\ &= Q * U_{\delta,k}, \end{aligned}$$

so that (2) is satisfied.

If $(d_{\delta,k}, c_{\delta,k}^*) = (-1, 1)$, then $c_{\delta,k} = c_{\delta,k}^* \cdot d_{\delta,k} = -1$. We then find that

$$\mathbf{Y}_{\delta,k} = Q * (H_{\delta,k}^{(-1)} \cdot V_{\delta,k}) \quad \text{by (3)}$$

$$= Q * (H_{\delta,k}^{(-1)} \cdot (U_{\delta,k}^{*(-1)})^{-1} \cdot U_{\delta,k}) \quad \text{by (4)}$$

$$= Q * (A_{\delta} \cdot U_{\delta,k}^{*(-1)} \cdot (U_{\delta,k}^{*(-1)})^{-1} \cdot U_{\delta,k}) \quad \text{by (5)}$$

$$= (Q * A_{\delta}) * U_{\delta,k}$$

$$= P_{\delta} * U_{\delta,k}$$

so that (2) is again satisfied. The validation for the cases $(d_{\delta,k}, c_{\delta,k}^*) = (1, -1)$ and $(d_{\delta,k}, c_{\delta,k}^*) = (-1, -1)$ are analogous to the previous cases. We find that for $b = \delta$ and all $k \in [\lambda]$, the equation in (2) is satisfied.

We also consider the case $b = 1 - \delta$. Let $k \in [\lambda]$. We show that the equation in (2) is satisfied.

The value $Y_{1-\delta,k}$ is computed as $(Y_{1-\delta,k}, V_{1-\delta,k}) \xleftarrow{\$} \text{IG} \left(W_{1-\delta,k}^{(d_{1-\delta,k})} \right)$ where $d_{1-\delta,k} \xleftarrow{\$} \{-1, 1\}$ is sampled uniformly at random and $c_{1-\delta,k}^*$ is a value in $\{-1, 1\}$.

If $c_{1-\delta,k}^* = 1$, $W_{1-\delta,k}^{(d_{1-\delta,k})}$ is computed as

$$\begin{cases} (W_{1-\delta,k}^{(d_{1-\delta,k})}, U_{1-\delta,k}^{*(1)}) \xleftarrow{\$} \text{IG}(Q) & \text{if } d_{1-\delta,k} = 1 \\ (W_{1-\delta,k}^{(d_{1-\delta,k})}, U_{1-\delta,k}^{*(-1)}) \xleftarrow{\$} \text{IG}(P_{1-\delta}) & \text{if } d_{1-\delta,k} = -1 \end{cases}$$

while if $c_{1-\delta,k}^* = -1$, $W_{1-\delta,k}^{(d_{1-\delta,k})}$ is computed as

$$\begin{cases} (W_{1-\delta,k}^{(d_{1-\delta,k})}, U_{1-\delta,k}^{*(1)}) \xleftarrow{\$} \text{IG}(P_{1-\delta}) & \text{if } d_{1-\delta,k} = 1 \\ (W_{1-\delta,k}^{(d_{1-\delta,k})}, U_{1-\delta,k}^{*(-1)}) \xleftarrow{\$} \text{IG}(Q) & \text{if } d_{1-\delta,k} = -1 \end{cases}.$$

So

$$\begin{cases} Y_{1-\delta,k} = Q * (U_{1-\delta,k}^{*(1)} \cdot V_{1-\delta,k}) & \text{if } d_{1-\delta,k} = 1 \text{ and } c_{1-\delta,k}^* = 1 \\ Y_{1-\delta,k} = P_{1-\delta} * (U_{1-\delta,k}^{*(-1)} \cdot V_{1-\delta,k}) & \text{if } d_{1-\delta,k} = -1 \text{ and } c_{1-\delta,k}^* = 1 \\ Y_{1-\delta,k} = P_{1-\delta} * (U_{1-\delta,k}^{*(1)} \cdot V_{1-\delta,k}) & \text{if } d_{1-\delta,k} = 1 \text{ and } c_{1-\delta,k}^* = -1 \\ Y_{1-\delta,k} = Q * (U_{1-\delta,k}^{*(-1)} \cdot V_{1-\delta,k}) & \text{if } d_{1-\delta,k} = -1 \text{ and } c_{1-\delta,k}^* = -1 \end{cases} \quad (6)$$

Moreover, $U_{1-\delta,k}$ is computed as

$$U_{1-\delta,k} = \begin{cases} U_{1-\delta,k}^{*(1)} \cdot V_{1-\delta,k} & \text{if } d_{1-\delta,k} = 1 \\ U_{1-\delta,k}^{*(-1)} \cdot V_{1-\delta,k} & \text{if } d_{1-\delta,k} = -1. \end{cases}$$

with $U_{1-\delta,k}^{*(1)}$, $U_{1-\delta,k}^{*(-1)}$ and $V_{1-\delta,k}$, as in (6). The value $c_{1-\delta,k}$ is computed as $c_{1-\delta,k} = c_{1-\delta,k}^* \cdot d_{1-\delta,k}$. By substituting $U_{1-\delta,k}$ into (6) and using the fact that $c_{1-\delta,k} = c_{1-\delta,k}^* \cdot d_{1-\delta,k}$, we obtain

$$\begin{cases} Y_{1-b,k} = P_b * U_{1-b,k} & \text{if } c_{1-b,k} = -1 \\ Y_{1-b,k} = Q * U_{1-b,k} & \text{if } c_{1-b,k} = 1 \end{cases},$$

so equation (2) is satisfied.

We now show that (1) holds. Let $\mathbf{h} := \mathcal{H}(\mathbf{Z}_0 || \mathbf{Z}_1 || m) = \mathcal{H}(\mathbf{Y}_0 || \mathbf{Y}_1 || m)$ and we note that $\{\delta, 1 - \delta\} = \{0, 1\}$. Let $k \in [\lambda]$. The value $c_{\delta,k}$ is computed as $c_{\delta,k} = c_{\delta,k}^* \cdot d_{\delta,k}$, where $d_{\delta,k} \xleftarrow{\$} \{-1, 1\}$ is sampled uniformly at random and $c_{\delta,k}^*$ is computed as $c_{\delta,k}^* = c'_k \cdot c_{1-\delta,k}^*$ for c'_k which is computed as $c'_k = h_k \cdot d_{\delta,k} \cdot d_{1-\delta,k}$. $c_{1-\delta,k}^*$ is sampled uniformly on $\{-1, 1\}$. We note that for any value $v \in \{-1, 1\}$, $v^2 = 1$. It follows that

$$\begin{aligned} c_{\delta,k} &= h_k \cdot d_{\delta,k} \cdot d_{1-\delta,k} \cdot c_{1-\delta,k}^* \cdot d_{\delta,k} \\ &= h_k \cdot d_{1-\delta,k} \cdot c_{1-\delta,k}^*. \end{aligned}$$

Furthermore, $c_{1-\delta,k}$ is computed as $c_{1-\delta,k} = c_{1-\delta,k}^* \cdot d_{1-\delta,k}$ where $c_{1-\delta,k}^*, d_{1-\delta,k} \in \{-1, 1\}$.

We find that for all $k \in [\lambda]$,

$$\begin{aligned} c_{0,k} \cdot c_{1,k} &= c_{\delta,k} \cdot c_{1-\delta,k} \\ &= h_k \cdot d_{1-\delta,k} \cdot c_{1-\delta,k}^* \cdot c_{1-\delta,k}^* \cdot d_{1-\delta,k} \\ &= h_k, \end{aligned}$$

and equality (1) holds, for any $b \in \{0, 1\}$ and $k \in [\lambda]$. Thus, if all calls to **IG** are successful, an honestly **GABS** signature passes verification.

Including the generation of the non-interactive OR proofs $\pi_0^{(\pm 1)}$ and $\pi_1^{(\pm 1)}$, execution of the **GABS** scheme requires $8\lambda^2 + 6\lambda$ calls to **IG**. It follows that

$$\begin{aligned} \Pr \left[\text{Verify}(\text{pk}, m, \sigma) = 1 \mid \begin{array}{l} (\text{st}_{\text{Sign}}, \rho_{\text{Sign},1}) \xleftarrow{\$} \text{Sign}_1(\text{sk}, \text{pk}), \\ (\text{st}_{\text{Use}}, \rho_{\text{Use}}) \xleftarrow{\$} \text{Use}_1(m, \rho_{\text{Sign},1}), \\ \rho_{\text{Sign}_2} \leftarrow \text{Sign}_2(\text{sk}, \text{st}_{\text{Sign}}, \rho_{\text{Use}}), \\ \sigma \leftarrow \text{Use}_2(\text{st}_{\text{Use}}, \rho_{\text{Sign},2}) \end{array} \right] \\ = 1 - \epsilon_{\text{GABS}}^{\text{Fail}}. \end{aligned}$$

where $\epsilon_{\text{GABS}}^{\text{Fail}} := (8\lambda^2 + 6\lambda) \cdot \epsilon_{\text{IG}}^{\text{Fail}}$ is negligible in λ . □

G.2 Blindness of **GABS**.

Theorem 64 (Blindness under chosen keys for **GABS).** *The **GABS** blind signature protocol satisfies blindness under chosen keys: for any PPT algorithm $\mathcal{A}$ and natural integer λ ,*

$$\left| \Pr \left[\text{Exp}_{\mathcal{A}}^{\text{Blind}}(\lambda) = 1 \right] - \frac{1}{2} \right| \leq \frac{4\lambda + 1}{2^{\lambda-1}} + 2\epsilon_{\text{Fail}}^{\text{GABS}}.$$

Proof. As the group in our generalised GAIP may be infinite, and without a uniform distribution, this proof differs from previous ones in the blind signature

state-of-the-art. Let us show through a sequence of games that a PPT adversary $\mathcal{A}$ will have a negligible advantage in distinguishing between the original security game and one in which it is provided no information depending on the bit b it is trying to guess.

Game $\mathbf{G}_0$: is the original blindness game. The adversary's advantage is denoted $\text{Adv}_{\mathcal{A}}^{\text{Blind}}(\lambda)$.

Game $\mathbf{G}_1$: is as the previous game, except that now, the challenger calls the extractor algorithm from the proof for special soundness for $\Sigma_{Q,P_0,\lambda}^{\text{OR}}$ and $\Sigma_{Q,P_1,\lambda}^{\text{OR}}$, that extracts witnesses $(\mathbf{U}_{\alpha,\mathcal{Y}}^{*(\beta)})_{\alpha \in \{0;1\}, \beta \in \{-1;1\}, \mathcal{Y} \in \{\mathcal{U}, \mathcal{V}\}}$ from $\mathcal{A}$ for the statements in the proofs $(\pi_{\alpha,\mathcal{Y}}^{(\beta)})_{\alpha, \beta, \mathcal{Y}}$ returned by $\mathcal{A}_1$ in each signer first message for users $\mathcal{U}$ and $\mathcal{V}$. If any of these extractions fail, the challenger aborts and replaces $\mathcal{A}$'s guess b' by a uniformly random bit. As the $\pi_{b,\mathcal{Y}}^{(\beta)}$ vectors have length λ , the probability for the adversary to be able to distinguish between this game and the previous one is bounded by 8λ times the soundness error of the proofs: $|\text{Adv}_{\mathcal{A}}^{\mathbf{G}_1}(\lambda) - \text{Adv}_{\mathcal{A}}^{\text{Blind}}(\lambda)| \leq 8\lambda \cdot \text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda) \leq \lambda \cdot 2^{3-\lambda}$.

Game $\mathbf{G}_2$: is as the previous game, except that now, the challenger calls the extractor algorithm from the special soundness for $\Sigma_{Q,\lambda}$ to obtain witnesses A_0 and A_1 from π_0 and π_1 respectively in the public key provided by $\mathcal{A}$. If the extraction fails, it aborts and replaces $\mathcal{A}$'s guess b' by a uniformly random bit. The adversary's advantage in distinguishing this game from the previous one is bounded by twice the soundness error of the proofs: $|\text{Adv}_{\mathcal{A}}^{\mathbf{G}_2}(\lambda) - \text{Adv}_{\mathcal{A}}^{\mathbf{G}_1}(\lambda)| \leq 2 \cdot \text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda) \leq 2^{1-\lambda}$.

Game $\mathbf{G}_3$: is as the previous game, except that now, if σ_0 and σ_1 are both valid, the challenger picks $\delta \xleftarrow{\$} \{0;1\}$, sets $\text{sk} \leftarrow (\delta, A_\delta)$ from A_0 and A_1 obtained by extraction, and replaces the σ_0 and σ_1 provided to $\mathcal{A}_2$ by the respective self-generated signatures

$$\begin{aligned} \tilde{\sigma}_0 &= (\tilde{\mathbf{c}}_0, \tilde{\mathbf{c}}_1, \tilde{\mathbf{U}}_0, \tilde{\mathbf{U}}_1) \xleftarrow{\$} \langle \text{BSign}(\text{sk}), \text{BUser}(\text{pk}, m_0) \rangle \\ \text{and} \quad \tilde{\sigma}_1 &= (\tilde{\mathbf{b}}_0, \tilde{\mathbf{b}}_1, \tilde{\mathbf{T}}_0, \tilde{\mathbf{T}}_1) \xleftarrow{\$} \langle \text{BSign}(\text{sk}), \text{BUser}(\text{pk}, m_0) \rangle, \end{aligned}$$

except if either $\tilde{\sigma}_0$ or $\tilde{\sigma}_1$ is equal to $\perp$, which happens with probability $\epsilon_{\text{Fail}}^{\text{GABS}}$. In this game, for $\tilde{\sigma}_0$, each $(\tilde{Y}_{\alpha,i}, \tilde{V}_{\alpha,i})$ is drawn from $\text{IG}(\tilde{W}_{\alpha,i})$ for some $\tilde{W}_{\alpha,i} \in [Q]$, whereas in the previous one each $(Y_{\alpha,i}, V_{\alpha,i})$ was drawn from $\text{IG}(W_{\alpha,i})$, where the challenger knew witnesses $B_{\alpha,i}$ such that $W_{\alpha,i} = Q * B_{\alpha,i}$. Hence, as IG satisfies Definition 3 for instance generators, the distributions of $(\tilde{Y}_{\alpha,u})_{\alpha, u \in \{0;1\}}$ and $(Y_{\alpha,u})_{\alpha, u \in \{0;1\}}$ are identical. Additionally, in the random oracle model, $\mathbf{c} \leftarrow \mathcal{H}(\mathbf{Y}_0 \| \mathbf{Y}_1 \| m_0)$ and $\tilde{\mathbf{c}} \leftarrow \mathcal{H}(\tilde{\mathbf{Y}}_0 \| \tilde{\mathbf{Y}}_1 \| m_0)$ both follow a uniform distribution on $\{-1;1\}^\lambda$, and act as a perfect one time pad on the $(\mathbf{d}_\alpha)_\alpha$ and $(\tilde{\mathbf{d}}_\alpha)_\alpha$ respectively in $\rho_{\text{Use}} = (c_k \cdot d_{0,k} \cdot d_{1,k})_k$ and $\tilde{\rho}_{\text{Use}} = (\tilde{c}_k \cdot \tilde{d}_{0,k} \cdot \tilde{d}_{1,k})_k$ transmitted to $\mathcal{A}_1$. Then, as the $(\mathbf{d}_\alpha)_\alpha$ and $(\tilde{\mathbf{d}}_\alpha)_\alpha$ are sampled uniformly at random from $\{-1;1\}^\lambda$, the $\mathbf{c}_\alpha = (c_{\alpha,k}^* \cdot d_{\alpha,k})_k$ and $\tilde{\mathbf{c}}_\alpha = (\tilde{c}_{\alpha,k}^* \cdot \tilde{d}_{\alpha,k})_k$, for adversary-provided $c_{\alpha,k}^*$'s and $\tilde{c}_{\alpha,k}^*$'s, are uniform in $\{-1;1\}^\lambda$.

Moreover, both $Y_{\alpha,i} = Q * A_{\alpha}^{(1-c_{\alpha,i})/2} U_{\alpha,i}$ and $\tilde{Y}_{\alpha,i} = Q * A_{\alpha}^{(1-\tilde{c}_{\alpha,i})/2} \tilde{U}_{\alpha,i}$, with $(Y_{\alpha,i}, c_{\alpha,i})$ and $(\tilde{Y}_{\alpha,i}, \tilde{c}_{\alpha,i})$ following identical distributions, so, by freedom of the group action, σ_0 and $\tilde{\sigma}_0$ follow the same distribution. The same reasoning also applies to $\tilde{\sigma}_1$ and σ_1 , so finally, $|\text{Adv}_{\mathcal{A}}^{\mathbf{G}_3}(\lambda) - \text{Adv}_{\mathcal{A}}^{\mathbf{G}_2}(\lambda)| \leq 2\epsilon_{\text{Fail}}^{\text{GABS}}$.

Game $\mathbf{G}_4$: is as the previous game, except that now, $\mathcal{U}$ is set to be $\text{BUser}(\text{pk}, m_{1-b})$ and $\mathcal{V}$ to be $\text{BUser}(\text{pk}, m_b)$. As the only $\mathcal{U}$ and $\mathcal{V}$ dependent states that $\mathcal{A}$ receives are respectively $\rho_{\mathcal{U}} = (c_k^{\mathcal{U}} \cdot d_{0,k}^{\mathcal{U}} \cdot d_{1,k}^{\mathcal{U}})_k$ and $\rho_{\mathcal{V}} = (c_k^{\mathcal{V}} \cdot d_{0,k}^{\mathcal{V}} \cdot d_{1,k}^{\mathcal{V}})_k$, where, in the random oracle model, $(c_k^{\mathcal{U}})_k$ and $(c_k^{\mathcal{V}})_k$ are uniformly random in $\{-1; 1\}$, they both follow the same distribution, and $\text{Adv}_{\mathcal{A}}^{\mathbf{G}_4}(\lambda) = \text{Adv}_{\mathcal{A}}^{\mathbf{G}_3}(\lambda)$. Hence, the adversary's view in this last game is independent of the bit b , and $\text{Adv}_{\mathcal{A}}^{\mathbf{G}_4}(\lambda) = 0$.

Finally, $\text{Adv}_{\mathcal{A}}^{\text{Blind}}(\lambda) \leq (4\lambda + 1)2^{1-\lambda} + 2\epsilon_{\text{Fail}}^{\text{GABS}}$.

□

G.3 One-More Unforgeability of GABS

We rely on Theorem 46 from [KLLQ23] in order to prove that GABS satisfies one-more unforgeability as described in 16. This section works towards proving k -one-more unforgeability of GABS (Theorem 77). In order to prove this theorem we follow the proof strategy from [KLX22] as adapted in [KLLQ23].

In the OMUF game, the adversary is allowed to make hash queries and signing queries, and the goal for the adversary outputs more verifying message signature pairs than the amount of signing queries that it made. We note that a hash query corresponds to a message signature pair (m, σ) if the input for the hash query corresponds to the input for the hash function as computed in $\text{GABS.Verify}(\text{pk}, m, \sigma)$. This corresponds to the discussion in [KLX22, Section 4.1].

Lemma 65. *The conditions in Definition 35 are satisfied for GABS.*

Proof. By inspection of the definition of GABS, conditions 1. and 4. from Definition 35 are satisfied. Signatures produced under the GABS protocol take the form of OR-proofs for $\Sigma_{Q,Q,\lambda}^{\text{OR}}$ for an NP OR-relation that uses the same NP relation for GAIP_{IG} for both branches, so that condition 1. is satisfied. Let us denote this NP OR-relation by $\mathcal{R}_{Q,Q,\lambda}^{\text{OR}} \subset \mathcal{X}^{\text{OR}} \times \mathcal{W}^{\text{OR}}$, following Definition 23. The challenge space of the sigma protocol for this NP-relation is the set $\{-1, 1\}^{\lambda}$, in which ρ_{Use} lies, so that condition 4. is satisfied.

We show that condition 2. is satisfied by freeness of the underlying group action $* : \mathcal{Q} \times \mathbb{G} \rightarrow \mathcal{Q}$. Recall that the GABS construction considers a fixed value $Q \in \mathcal{Q}$ and public keys take the form of $\text{pk} = ((P_0, \pi_0, \pi_1), (P_1, \pi_0, \pi_1))$. For an honestly generated public key, $P_0, P_1 \in [Q]$ and there exist group elements $A_0, A_1 \in \mathbb{G}$ such that $P_0 = Q * A_0$ and $P_1 = Q * A_1$. The components π_b , for $b \in \{0, 1\}$ are zero-knowledge proofs showing that such A_0 and A_1 exists. The tuple (P_0, P_1) describes a statement in $\mathcal{X}_{\text{OR}}$ and the proofs π_0 and π_1 are added

so that one can verify that two corresponding witnesses exist and thus that the public key is valid.

A secret key for such a public key $\mathbf{pk}$ is a tuple (δ, A_δ) such that $P_\delta = Q * A_\delta$. For $b \in \{0, 1\}$, if there exists a value $A_b \in \mathbb{G}$ such that $P_b = Q * A_b$, then it follows by freeness of the group action that A_b is unique. For a valid public key, such A_0 and A_1 always exist, hence condition 3. is satisfied.

In order to see that condition 3. is satisfied, we refer to the discussion in [KLX22, Section 4.1] which almost directly applies to the **GABS** construction. The argument from [KLX22] requires that it can be assumed that no two distinct message signature pairs correspond to the same hash query. Their argument for this relies on the discrete logarithm problem, which is not applicable to us. In our case, if two distinct message signature pairs correspond to the same hash query, then the extractors in Figure 5 can be used to extract one of the two possible secret keys corresponding to the public key. This equates to solving one of two GAIP_{IG} instances embedded in the public key. $\square$

In the rest of the section we use the overhead arrow vector notation as introduced in Section B.

Instances for GABS. We discuss what 0-side and 1-side instances look like for GABS. A b -side instance

$$I_b = (b, A_b, \mathbf{pk}_{1-b}, \overrightarrow{\mathbf{H}_b^{(1)}}, \overrightarrow{\mathbf{H}_b^{(-1)}}, \overrightarrow{\mathbf{c}_{1-b}^*}, \overrightarrow{\mathbf{U}_{1-b}^{*(1)}}, \overrightarrow{\mathbf{U}_{1-b}^{*(-1)}}, \overrightarrow{\Pi}),$$

for $b \in \{0, 1\}$, is defined as follows:

- (b, A_b) is the secret key for if $\delta = b$.
- $\mathbf{pk}_{1-b} = (P_{1-b}, \pi_0, \pi_1)$ is the part of the public key $\mathbf{pk} = (\mathbf{pk}_0, \mathbf{pk}_1)$ whose secret is unknown.
- Consider $\overrightarrow{\mathbf{H}_b^{(1)}}$ of which we denote the j -th component by $\mathbf{H}_b^{(1)(j)}$ for $j \in [k+1]$. The vector $\mathbf{H}_b^{(1)(j)}$ corresponds to the value of $\mathbf{H}_b^{(1)}$ for the commitment $\mathbf{W}_b^{(1)}$ in the j -th first signer message such that

$$W_{b,i}^{(1)} = Q * H_{b,i}^{(1)} \text{ for all } i \in [\lambda]$$

when $\delta = b$.

- Consider $\overrightarrow{\mathbf{H}_b^{(-1)}}$ of which we denote the j -th component by $\mathbf{H}_b^{(-1)(j)}$ for $j \in [k+1]$. The vector $\mathbf{H}_b^{(-1)(j)}$ corresponds to the value of $\mathbf{H}_b^{(-1)}$ for the commitment $\mathbf{W}_b^{(-1)}$ in the j -th first signer message such that

$$W_{b,i}^{(-1)} = Q * H_{b,i}^{(-1)} \text{ for all } i \in [\lambda]$$

when $\delta = b$.

- Consider $\overrightarrow{\mathbf{c}_{1-b}^*}$ of which we denote the j -th component by $\mathbf{c}_{1-b}^{*(j)}$ for $j \in [k+1]$. The vector $\mathbf{c}_{1-b}^{*(j)}$ describes the simulated challenge $\mathbf{c}_{1-b}^*$ in the j -th first signer message for if $\delta = b$.
- Consider $\overrightarrow{\mathbf{U}_{1-b}^{*(1)}}$ of which we denote the j -th component by $\mathbf{U}_{1-b}^{*(1)(j)}$ for $j \in [k+1]$. The vector $\mathbf{U}_{1-b}^{*(1)(j)}$ corresponds to the value of $\mathbf{U}_{1-b}^{*(1)}$ for the commitment $\mathbf{W}_{1-b}^{(1)}$ in the j -th first signer message such that

$$\mathbf{W}_{1-b,i}^{(1)} = Q * \mathbf{U}_{1-b,i}^{*(1)} \text{ for all } i \in [\lambda]$$

when $\delta = b$.

- Consider $\overrightarrow{\mathbf{U}_{1-b}^{*(-1)}}$ of which we denote the j -th component by $\mathbf{U}_{1-b}^{*(-1)(j)}$ for $j \in [k+1]$. The vector $\mathbf{U}_{1-b}^{*(-1)(j)}$ corresponds to the value of $\mathbf{U}_{1-b}^{*(-1)}$ for the commitment $\mathbf{W}_{1-b}^{(-1)}$ in the j -th first signer message such that

$$\mathbf{W}_{1-b,i}^{(-1)} = Q * \mathbf{U}_{1-b,i}^{*(-1)} \text{ for all } i \in [\lambda]$$

when $\delta = b$.

- Consider $\overrightarrow{\Pi}$ of which we denote the j -th component by $\Pi^{(j)}$ for $j \in [k+1]$. The collection Π contains all internal randomness used to generate the proofs $(\pi_0^{(1)}, \pi_0^{(-1)}, \pi_1^{(1)}, \pi_1^{(-1)})$ showing that the entries of $\mathbf{W}_0^{(\pm 1)}$ and $\mathbf{W}_1^{(\pm 1)}$ lie in the orbit of Q, P_0 or P_1 under the group action.

We may assume without loss of generality that for all $j \in [k+1]$, none of the components of $\mathbf{H}_b^{(1)(j)}$, $\mathbf{H}_b^{(-1)(j)}$, $\mathbf{U}_{1-b}^{*(1)(j)}$ or $\mathbf{U}_{1-b}^{*(-1)(j)}$ are equal to $\perp$, as this corresponds to the first signer algorithm aborting before interaction with the adversary. Similarly we may assume that the proofs $(\pi_0^{(1)}, \pi_0^{(-1)}, \pi_1^{(1)}, \pi_1^{(-1)})$ are generated successfully under the randomness in $\Pi^{(j)}$.

We first construct the map $\Phi_{\text{rand}, \vec{h}}$ that we need to show that Lemma 44 holds for GABS. We can define $\Phi_{\text{rand}, \vec{h}}$ mostly explicitly and first provide that part of its definition.

Explicit definition for $\Phi_{\text{rand}, \vec{h}}$. Let $\text{rand}, \vec{h}$ be fixed. We define a map $\Phi_{\text{rand}, \vec{h}}$ that maps 0-side instances to 1-side instances and vice versa. We consider values $b \in \{0, 1\}$. Provided a b -side instance

$$I_b = (b, A_b, \text{pk}_{1-b}, \overrightarrow{\mathbf{H}_b^{(1)}}, \overrightarrow{\mathbf{H}_b^{(-1)}}, \overrightarrow{\mathbf{c}_{1-b}^*}, \overrightarrow{\mathbf{U}_{1-b}^{*(1)}}, \overrightarrow{\mathbf{U}_{1-b}^{*(-1)}}, \overrightarrow{\Pi})$$

such that $(I, \text{rand}, \vec{h}) \in \text{Succ}$, we know the following:

- There exists a unique value $A_{b-1} \in \mathbb{G}$ such that $\text{pk}_{1-b} = Q * A_{b-1}$.
- The b -side instance I_b , together with fixed values for rand and $\vec{h}$, fixes a query transcript $\vec{c}^* = \vec{c}(I, \text{rand}, \vec{h})$.

- The public key component $\mathbf{pk}_{1-b}$ is of the form (P_{1-b}, π_0, π_1) .

The map $\Phi_{\text{rand}, \vec{h}}$ is defined as follows. It maps a b -side instance I_b as above to

$$\Phi_{\text{rand}, \vec{h}}(I_b) = (1-b, A_{1-b}, \mathbf{pk}_b := (Q * A_b, \pi_0, \pi_1), \overrightarrow{\mathbf{H}_{1-b}^{(1)}}, \overrightarrow{\mathbf{H}_{1-b}^{(-1)}}, \overrightarrow{\mathbf{c}_b^*}, \overrightarrow{\mathbf{U}_b^{*(1)}}, \overrightarrow{\mathbf{U}_b^{*(-1)}}, \overrightarrow{\Theta})$$

where A_{1-b} is such that $P_{1-b} = Q * A_{b-1}$ and for $j \in [k+1]$ and $i \in [\lambda]$,

$$\begin{aligned} H_{1-b,i}^{(1)(j)} &:= \begin{cases} A_{1-b} \cdot U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = -1 \\ U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases}, \\ H_{1-b,i}^{(-1)(j)} &:= \begin{cases} U_{1-b,i}^{*(-1)(j)} & \text{if } c_{1-b,i}^{*(j)} = -1 \\ A_{1-b} \cdot U_{1-b,i}^{*(-1)(j)} & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases}, \\ c_{b,i}^{*(j)} &:= c_i^{*(j)} \cdot c_{1-b,i}^{*(j)}, \quad \text{with } \overrightarrow{\mathbf{c}^*} = \overrightarrow{\mathbf{c}}(I_b, \text{rand}, \vec{h}) \\ U_{b,i}^{*(1)(j)} &:= \begin{cases} A_b^{-1} \cdot H_{b,i}^{(1)(j)} & \text{if } c_{b,i}^{*(j)} = -1 \\ H_{b,i}^{(1)(j)} & \text{if } c_{b,i}^{*(j)} = 1 \end{cases}, \\ U_{b,i}^{*(-1)(j)} &:= \begin{cases} H_{b,i}^{(-1)(j)} & \text{if } c_{b,i}^{*(j)} = -1 \\ A_b^{-1} \cdot H_{b,i}^{(-1)(j)} & \text{if } c_{b,i}^{*(j)} = 1 \end{cases}. \end{aligned}$$

The definition of $\overrightarrow{\Theta}$ is implicit. Lemma 62 can be used to show that there exists a unique value for $\overrightarrow{\Theta}$ such that $\Phi_{\text{rand}, \vec{h}}$ maintains transcripts. We define $\overrightarrow{\Theta}$ to be exactly this value, but we will come back to this.

Lemma 66. *The map $\Phi_{\text{Rand}, \vec{h}}$ respects the components $(\overrightarrow{\mathbf{W}_0^{(\pm 1)}}, \overrightarrow{\mathbf{W}_1^{(\pm 1)}})$ of the first signer messages $\overrightarrow{\rho_{\text{Sign}_1}}$. That is, for any instance I and for the first signer messages from the transcripts $\text{trans}(I, \text{rand}, \vec{h})$ and $\text{trans}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$, the components $(\overrightarrow{\mathbf{W}_0^{(\pm 1)}}, \overrightarrow{\mathbf{W}_1^{(\pm 1)}})$ are equal. Note that the computation of these components does not depend on the definition of $\overrightarrow{\Theta}$ for $\Phi_{\text{rand}, \vec{h}}$, which we still need to define.*

Proof. Let I_b be a b -side instance and let rand and $\vec{h}$ be fixed. The components $(\overrightarrow{\mathbf{W}_0^{(\pm 1)}}, \overrightarrow{\mathbf{W}_1^{(\pm 1)}})$ in the first signer message in the transcript $\text{trans}(I_b, \text{rand}, \vec{h})$ are equal to

$$\begin{aligned} W_{b,i}^{(1)(j)} &:= Q * H_{b,i}^{(1)(j)}, \quad W_{1-b,i}^{(1)(j)} := \begin{cases} P_{1-b} * U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = -1 \\ Q * U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases}, \\ W_{b,i}^{(-1)(j)} &:= Q * H_{b,i}^{(-1)(j)}, \quad W_{1-b,i}^{(-1)(j)} := \begin{cases} Q * U_{1-b,i}^{*(-1)(j)} & \text{if } c_{1-b,i}^{*(j)} = -1 \\ P_{1-b} * U_{1-b,i}^{*(-1)(j)} & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases} \end{aligned}$$

and the components $\left(\overrightarrow{\mathbf{W}'_0^{(\pm 1)}}, \overrightarrow{\mathbf{W}'_1^{(\pm 1)}}\right)$ from the first signer message in the transcript $\text{trans}(\Phi_{\text{rand}, \vec{h}}(I_b), \text{rand}, \vec{h})$ are equal to

$$\begin{aligned} W'_{1-b,i}^{(1)(j)} &:= Q * H_{1-b,i}^{(1)(j)}, & W'_{b,i}^{(1)(j)} &:= \begin{cases} P_b * U_{b,i}^{*(1)(j)} & \text{if } c_{b,i}^{*(j)} = -1 \\ Q * U_{b,i}^{*(1)(j)} & \text{if } c_{b,i}^{*(j)} = 1 \end{cases}, \\ W'_{1-b,i}^{(-1)(j)} &:= Q * H_{1-b,i}^{(-1)(j)}, & W'_{b,i}^{(-1)(j)} &:= \begin{cases} Q * U_{b,i}^{*(-1)(j)} & \text{if } c_{b,i}^{*(j)} = -1 \\ P_b * U_{1-b,i}^{*(-1)(j)} & \text{if } c_{b,i}^{*(j)} = 1 \end{cases}. \end{aligned}$$

Let us now prove that

$$\begin{aligned} W_{b,i}^{(1)(j)} &= W'_{b,i}^{(1)(j)}, & W_{b,i}^{(-1)(j)} &= W'_{b,i}^{(-1)(j)}, \\ W_{1-b,i}^{(1)(j)} &= W'_{1-b,i}^{(1)(j)}, & W_{1-b,i}^{(-1)(j)} &= W'_{1-b,i}^{(-1)(j)}. \end{aligned}$$

From the definition of $\Phi_{\text{rand}, \vec{h}}$,

$$H_{1-b,i}^{(j)} = \begin{cases} A_{1-b} \cdot U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = -1 \\ U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = 1, \end{cases}$$

Hence, as $P_{1-b} = Q * A_{1-b}$,

$$\begin{aligned} W'_{1-b,i}^{(1)(j)} &= Q * H_{1-b,i}^{(1)(j)} = \begin{cases} Q * (A_{1-b} \cdot U_{1-b,i}^{*(1)(j)}) & \text{if } c_{1-b,i}^{*(j)} = -1 \\ Q * U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases} \\ &= \begin{cases} P_{1-b} * U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = -1 \\ Q * U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases} \\ &= W_{1-b,i}^{(1)(j)}. \end{aligned}$$

From the description of $\Phi_{\text{rand}, \vec{h}}$ we also obtain that

$$U_{b,i}^{*(1)(j)} = \begin{cases} A_b^{-1} \cdot H_{b,i}^{(1)(j)} & \text{if } c_{b,i}^{*(j)} = -1 \\ H_{b,i}^{(1)(j)} & \text{if } c_{b,i}^{*(j)} = 1 \end{cases},$$

where $P_b = Q * A_b$. So, we find that

$$\begin{aligned} W'_{b,i}^{(1)(j)} &= \begin{cases} P_{1-b} * U_{b,i}^{*(1)(j)} & \text{if } c_{b,i}^{*(j)} = -1 \\ Q * U_{b,i}^{*(1)(j)} & \text{if } c_{b,i}^{*(j)} = 1 \end{cases} \\ &= \begin{cases} Q * (A_b \cdot A_b^{-1} \cdot H_{b,i}^{(1)(j)}) & \text{if } c_{b,i}^{*(j)} = -1 \\ Q * H_{b,i}^{(1)(j)} & \text{if } c_{b,i}^{*(j)} = 1 \end{cases} \\ &= Q * H_{b,i}^{(1)(j)} \\ &= W_{b,i}^{(1)(j)}. \end{aligned}$$

The verification process for the equations $W_{1-b,i}^{(-1)(j)} = W_{1-b,i}^{(-1)(j)}$ and $W_{b,i}^{(-1)(j)} = W_{b,i}^{(-1)(j)}$ is analogous. $\square$

Finishing the definition of $\Phi_{\text{rand}, \vec{h}}$, defining $\vec{\Theta}$. We finish the definition of $\Phi_{\text{rand}, \vec{h}}$ by defining the value $\vec{\Theta}$ that we left undefined earlier. For $j \in [k+1]$ and $i \in [\lambda]$, let

$$\begin{aligned} W_{b,i}^{(1)(j)} &= W_{b,i}'^{(1)(j)}, & W_{b,i}^{(-1)(j)} &= W_{b,i}'^{(-1)(j)}, \\ W_{1-b,i}^{(1)(j)} &= W_{1-b,i}'^{(1)(j)}, & W_{1-b,i}^{(-1)(j)} &= W_{1-b,i}'^{(-1)(j)}. \end{aligned}$$

be as in the proof for Lemma 66. The value of $\vec{\Theta}$ is uniquely determined by the condition that $\forall j \in [k+1]$ and $\forall i \in [\lambda]$:

1. The output of $\Pi_{\text{OR}}^{W_{1-b,i}'^{(1)(j)}, \lambda}((Q, P_{1-b}), 0, H_{1-b,i}^{(1)(j)})$ under the randomness $\vec{\Theta}$ is equal to the output of $\begin{cases} \Pi_{\text{OR}}^{W_{1-b,i}'^{(1)(j)}, \lambda}((Q, P_{1-b}), 1, U_{1-b,i}^{*(1)(j)}) & \text{if } c_{1-b,i}^{*(j)} = -1 \\ \Pi_{\text{OR}}^{W_{1-b,i}'^{(1)(j)}, \lambda}((Q, P_{1-b}), 0, U_{1-b,i}^{*(1)(j)}) & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases}$ under the randomness $\vec{\Pi}$.
2. The output of $\Pi_{\text{OR}}^{W_{1-b,i}'^{(-1)(j)}, \lambda}((Q, P_{1-b}), 0, H_{1-b,i}^{(-1)(j)})$ under the randomness $\vec{\Theta}$ is equal to the output of $\begin{cases} \Pi_{\text{OR}}^{W_{1-b,i}'^{(-1)(j)}, \lambda}((Q, P_{1-b}), 0, U_{1-b,i}^{*(-1)(j)}) & \text{if } c_{1-b,i}^{*(j)} = -1 \\ \Pi_{\text{OR}}^{W_{1-b,i}'^{(-1)(j)}, \lambda}((Q, P_{1-b}), 1, U_{1-b,i}^{*(-1)(j)}) & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases}$ under the randomness $\vec{\Pi}$.
3. The output of $\begin{cases} \Pi_{\text{OR}}^{W_{b,i}'^{(1)(j)}, \lambda}((Q, P_b), 1, U_{b,i}^{*(1)(j)}) & \text{if } c_{b,i}^{*(j)} = -1 \\ \Pi_{\text{OR}}^{W_{b,i}'^{(1)(j)}, \lambda}((Q, P_b), 0, U_{b,i}^{*(1)(j)}) & \text{if } c_{b,i}^{*(j)} = 1 \end{cases}$ under the randomness $\vec{\Theta}$ equals the output of $\Pi_{\text{OR}}^{W_{b,i}^{(1)(j)}, \lambda}((Q, P_b), 0, H_{1-b,i}^{(1)(j)})$ under the randomness $\vec{\Pi}$.
4. The output of $\begin{cases} \Pi_{\text{OR}}^{W_{b,i}'^{(-1)(j)}, \lambda}((Q, P_b), 0, U_{b,i}^{*(-1)(j)}) & \text{if } c_{b,i}^{*(j)} = -1 \\ \Pi_{\text{OR}}^{W_{b,i}'^{(-1)(j)}, \lambda}((Q, P_b), 1, U_{b,i}^{*(-1)(j)}) & \text{if } c_{b,i}^{*(j)} = 1 \end{cases}$ under the randomness $\vec{\Theta}$ equals the output of $\Pi_{\text{OR}}^{W_{b,i}^{(-1)(j)}, \lambda}((Q, P_b), 0, H_{1-b,i}^{(-1)(j)})$ under the randomness $\vec{\Pi}$.

This properly defines $\vec{\Theta}$ by Lemma 66 and Lemma 62. We have now fully defined $\Phi_{\text{rand}, \vec{h}}$ and it can be checked directly that $\Phi_{\text{rand}, \vec{h}}$ maps the b -side instance I_b to a $(1-b)$ -side instance.

Lemma 67. *Let I be an instance and fix values for rand and $\vec{h}$. Let $\overrightarrow{\rho_{\text{Sign}_1}}$ be the collection of first signer messages of the transcript $\text{trans}(I, \text{rand}, \vec{h})$ and let*

$\overrightarrow{\rho'_{\text{Sign}_1}}$ be the first signer messages of the transcript $\text{trans}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$. It holds that $\overrightarrow{\rho_{\text{Sign}_1}} = \overrightarrow{\rho'_{\text{Sign}_1}}$.

Proof. For $j \in [k+1]$, $\rho_{\text{Sign}_1}^{(j)}$ and $\rho'_{\text{Sign}_1}^{(j)}$ are of the form

$$\begin{aligned}\rho_{\text{Sign}_1}^{(j)} &= (\mathbf{W}_0^{(1)}, \mathbf{W}_0^{(-1)}, \pi_0^{(1)}, \pi_0^{(-1)}, \mathbf{W}_1^{(1)}, \mathbf{W}_1^{(-1)}, \pi_1^{(1)}, \pi_1^{(-1)}), \\ \rho'_{\text{Sign}_1}^{(j)} &= (\mathbf{W}'_0^{(1)}, \mathbf{W}'_0^{(-1)}, \pi'^{(1)}_0, \pi'^{(-1)}_0, \mathbf{W}'_1^{(1)}, \mathbf{W}'_1^{(-1)}, \pi'^{(1)}_1, \pi'^{(-1)}_1).\end{aligned}$$

By Lemma 66 it holds that $\mathbf{W}'_0^{(\pm 1)} = \mathbf{W}_0^{(\pm 1)}$ and $\mathbf{W}'_1^{(\pm 1)} = \mathbf{W}_1^{(\pm 1)}$. Furthermore, the conditions that define $\vec{\Theta}$ in the definition of $\Phi_{\text{rand}, \vec{h}}$ impose precisely that $\pi_0^{(\pm 1)} = \pi'^{(\pm 1)}_0$ and $\pi_1^{(\pm 1)} = \pi'^{(\pm 1)}_1$. Thus, we obtain that $\overrightarrow{\rho_{\text{Sign}_1}} = \overrightarrow{\rho'_{\text{Sign}_1}}$. $\square$

Lemma 68. Let I be an instance and fix values for rand and $\vec{h}$. Let $\overrightarrow{\rho_{\text{Use}}}$ be the collection of first user messages of the transcript $\text{trans}(I, \text{rand}, \vec{h})$ and let $\overrightarrow{\rho'_{\text{Use}}}$ be the first user messages of the transcript $\text{trans}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$. It holds that $\overrightarrow{\rho_{\text{Use}}} = \overrightarrow{\rho'_{\text{Use}}}$.

Proof. The adversary computes the first user message based on the first signer messages using randomness. By Lemma 67, the first signer messages from $\text{trans}(I, \text{rand}, \vec{h})$ are equal to those from $\text{trans}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$. Since the randomness for the adversary is fixed we find that $\overrightarrow{\rho_{\text{Use}}} = \overrightarrow{\rho'_{\text{Use}}}$. $\square$

Note that the result of Lemma 68 can also be stated as $(I, \text{rand}, \vec{h})$ and $(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$ inducing the same query transcript, that is

$$\vec{e}(I, \text{rand}, \vec{h}) = \vec{e}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h}).$$

Lemma 69. Let I be an instance and fix values for rand and $\vec{h}$. Let $\overrightarrow{\rho_{\text{Sign}_2}}$ be the collection of second signer messages of the transcript $\text{trans}(I, \text{rand}, \vec{h})$ and let $\overrightarrow{\rho'_{\text{Sign}_2}}$ be the second messages of the transcript $\text{trans}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$. It holds that $\overrightarrow{\rho_{\text{Sign}_2}} = \overrightarrow{\rho'_{\text{Sign}_2}}$.

Proof. Let $I_b = (b, A_b, \text{pk}_{1-b}, \overrightarrow{\mathbf{H}_b^{(1)}}, \overrightarrow{\mathbf{H}_b^{(-1)}}, \overrightarrow{\mathbf{c}_{1-b}^*}, \overrightarrow{\mathbf{U}_{1-b}^{*(1)}}, \overrightarrow{\mathbf{U}_{1-b}^{*(-1)}}, \vec{\Pi})$ be a b -side instance. We denote the image of I_b under $\Phi_{\text{rand}, \vec{h}}$ by

$$\Phi_{\text{rand}, \vec{h}}(I_b) = (1-b, A_{1-b}, \text{pk}_b, \overrightarrow{\mathbf{H}_{1-b}^{(1)}}, \overrightarrow{\mathbf{H}_{1-b}^{(-1)}}, \overrightarrow{\mathbf{c}_b^*}, \overrightarrow{\mathbf{U}_b^{*(1)}}, \overrightarrow{\mathbf{U}_b^{*(-1)}}, \vec{\Theta})$$

Let $j \in [k+1]$. The second signer messages $\rho_{\text{Sign}_2}^{(j)}$ and $\rho'_{\text{Sign}_2}^{(j)}$ are of the form

$$\begin{aligned}\rho_{\text{Sign}_2}^{(j)} &= (\mathbf{c}_0^*, \mathbf{U}_0^{*(1)}, \mathbf{U}_0^{*(-1)}, \mathbf{c}_1^*, \mathbf{U}_1^{*(1)}, \mathbf{U}_1^{*(-1)}), \\ \rho'_{\text{Sign}_2}^{(j)} &= (\mathbf{c}'_0, \mathbf{U}_0'^{*(1)}, \mathbf{U}_0'^{*(-1)}, \mathbf{c}'_1, \mathbf{U}_1'^{*(1)}, \mathbf{U}_1'^{*(-1)}).\end{aligned}$$

Let $\mathbf{c}$ be the j -th component of user messages $\overrightarrow{\rho_{\text{Use}}} = \overrightarrow{\rho'_{\text{Use}}}$ which are equal by Lemma 68. Note that $\mathbf{c}$ is also the j -th component of the query transcript $\overrightarrow{\mathcal{E}}(I, \text{rand}, \vec{h})$. We denote $\mathbf{H}_{1-b}^{(1)} := \mathbf{H}_{1-b}^{(1)(j)}$ and $\mathbf{H}_{1-b}^{(-1)} := \mathbf{H}_{1-b}^{(-1)(j)}$.

By the definition of $\Phi_{\text{rand}, \vec{h}}$, it holds that $c'_{b,i} = c_i \cdot c_{1-b,i}^*$ for $i \in [\lambda]$. The definition of the second signer algorithm for GABS, tells us that also for all $i \in [\lambda]$,

$$c_{b,i}^* = c_i \cdot c_{1-b,i}^* \text{ and } c'_{1-b,i} = c_i \cdot c'_{b,i}.$$

Recall that $c_i \in \{-1; 1\}$, so that $c_i^2 = 1$. From the equalities it follows that $\mathbf{c}'_0 = \mathbf{c}_0^*$ and $\mathbf{c}'_1 = \mathbf{c}_1^*$.

We now show that $\mathbf{U}_b'^{*(1)} = \mathbf{U}_b^{*(1)}$. From the definition of the second signer algorithm for GABS we know that for $i \in [\lambda]$, $\mathbf{U}_{b,i}^{*(1)}$ is computed as

$$\mathbf{U}_{b,i}^{*(1)} = \begin{cases} A_b^{-1} \cdot \mathbf{H}_{b,i}^{(1)} & \text{if } c_{b,i}^* = -1 \\ \mathbf{H}_{b,i}^{(1)} & \text{if } c_{b,i}^* = 1 \end{cases}.$$

From the definition of $\Phi_{\text{rand}, \vec{h}}$ we obtain that

$$\mathbf{U}_{b,i}'^* = \begin{cases} A_b^{-1} \cdot \mathbf{H}_{b,i}^{(1)} & \text{if } c_{b,i}^* = -1 \\ \mathbf{H}_{b,i}^{(1)} & \text{if } c_{b,i}^* = 1, \end{cases}$$

so that clearly $\mathbf{U}_b'^{*(1)} = \mathbf{U}_b^{*(1)}$. A similar computation shows that also $\mathbf{U}_b'^{*(-1)} = \mathbf{U}_b^{*(-1)}$.

Lastly, we show that $\mathbf{U}_{1-b}'^* = \mathbf{U}_{1-b}^*$. From the definition of $\Phi_{\text{rand}, \vec{h}}$,

$$\mathbf{H}_{1-b,i}' = \begin{cases} A_{1-b} \cdot \mathbf{U}_{1-b,i}^{*(1)} & \text{if } c_{1-b,i}^* = -1 \\ \mathbf{U}_{1-b,i}^{*(1)} & \text{if } c_{1-b,i}^* = 1. \end{cases}$$

Recall that $\mathbf{c}'_{1-b} = \mathbf{c}_{1-b}^*$. The definition of the second signer algorithm for GABS, tells us that for $i \in [\lambda]$, $\mathbf{U}_{1-b,i}'^*$ is computed as

$$\begin{aligned}\mathbf{U}_{1-b,i}'^* &= \begin{cases} A_{1-b}^{-1} \cdot \mathbf{H}_{1-b,i}' & \text{if } c'_{1-b,i} = -1 \\ \mathbf{H}_{1-b,i}' & \text{if } c'_{1-b,i} = 1 \end{cases} \\ &= \begin{cases} A_{1-b}^{-1} \cdot A_{1-b} \cdot \mathbf{U}_{1-b,i}^{*(1)} & \text{if } c_{1-b,i}^* = -1 \\ \mathbf{U}_{1-b,i}^{*(1)} & \text{if } c_{1-b,i}^* = 1 \end{cases} \\ &= \mathbf{U}_{1-b,i}^{*(1)},\end{aligned}$$

so that $\mathbf{U}_{1-b}^{*(1)} = \mathbf{U}_{1-b}^{*(1)}$. As similar computation shows that also $\mathbf{U}_{1-b}^{*(-1)'} = \mathbf{U}_{1-b}^{*(-1)}$.

We conclude that $\overrightarrow{\rho_{\text{Sign}_2}} = \overrightarrow{\rho'_{\text{Sign}_2}}$. $\square$

Lemma 70. *Let I be an instance and fix values for rand and $\vec{h}$. Let $\vec{\sigma}$ be the collection of signatures in the transcript $\text{trans}(I, \text{rand}, \vec{h})$ and let $\vec{\sigma}'$ be the signatures from the transcript $\text{trans}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$. It holds that $\vec{\sigma} = \vec{\sigma}'$.*

Proof. The adversary computes the forged signatures based on the signer messages using randomness. By Lemma 67 and Lemma 69, the signer messages from both transcripts are equal. Since the randomness for the adversary is fixed we find that $\vec{\sigma} = \vec{\sigma}'$. $\square$

Lemma 71. *The map $\Phi_{\text{rand}, \vec{h}}$ respects transcripts, which means the following. Let rand and $\vec{h}$ be fixed. Let I be an instance such that $(I, \text{rand}, \vec{h}) \in \text{Succ}$. It holds that*

$$\text{trans}(I, \text{rand}, \vec{h}) = \text{trans}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h}).$$

Proof. A transcript consists of signer messages, a user message and a signature. By Lemma 67, Lemma 68, Lemma 69 and Lemma 70, all of these components are equal in $\text{trans}(I, \text{rand}, \vec{h})$ and $\text{trans}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$. $\square$

Lemma 72. *Let rand and $\vec{h}$ be fixed. For instances I such that $(I, \text{rand}, \vec{h}) \in \text{Succ}$, $\Phi_{\text{rand}, \vec{h}}$ is a self inverse bijection.*

Proof. Let $(I, \text{rand}, \vec{h}) \in \text{Succ}$. We directly show that

$$\Phi_{\text{rand}, \vec{h}}(\Phi_{\text{rand}, \vec{h}}(I)) = I.$$

The instance I is a b -side instance for some $b \in \{0, 1\}$ and can thus be written as

$$I = (b, A_b, \text{pk}_{1-b} = (P_{1-b}, \pi_0, \pi_1), \overrightarrow{\mathbf{H}_b^{(1)}}, \overrightarrow{\mathbf{H}_b^{(-1)}}, \overrightarrow{\mathbf{c}_{1-b}^*}, \overrightarrow{\mathbf{U}_{1-b}^{*(1)}}, \overrightarrow{\mathbf{U}_{1-b}^{*(-1)}}, \overrightarrow{\Pi}).$$

Furthermore, we write

$$\Phi_{\text{rand}, \vec{h}}(I) = (1-b, A_{1-b}, \text{pk}_b = (P_b, \pi_0, \pi_1), \overrightarrow{\mathbf{H}_{1-b}^{(1)}}, \overrightarrow{\mathbf{H}_{1-b}^{(-1)}}, \overrightarrow{\mathbf{c}_b^*}, \overrightarrow{\mathbf{U}_b^{*(1)}}, \overrightarrow{\mathbf{U}_b^{*(-1)}}, \overrightarrow{\Theta})$$

and

$$\begin{aligned} \Phi_{\text{rand}, \vec{h}}(\Phi_{\text{rand}, \vec{h}}(I)) = (b', A'_b, \text{pk}'_{1-b} = (P'_{1-b}, \pi'_0, \pi'_1), \overrightarrow{\mathbf{G}_b^{(1)}}, \overrightarrow{\mathbf{G}_b^{(-1)}}, \overrightarrow{\mathbf{d}_{b-1}^*}, \\ \overrightarrow{\mathbf{V}_{1-b}^{*(1)}}, \overrightarrow{\mathbf{V}_{1-b}^{*(-1)}}, \overrightarrow{\Pi'}). \end{aligned}$$

We directly see that $b' = 1 - (1 - b) = b$ and $P_b = Q * A_b$, so that A'_b must equal A_b . Furthermore, from the definition of P_{1-b} we obtain that $P'_{1-b} = Q * A_{1-b} = P_{1-b}$, and from the definition of $\Phi_{\text{rand}, \vec{h}}$ we obtain that $\pi'_0 = \pi_0$ and $\pi'_1 = \pi_1$. It follows that $\text{pk}'_{1-b} = \text{pk}_{1-b}$.

Let $j \in [k+1]$ and $i \in [\lambda]$. We have

$$G_{b,i}^{(1)(j)} = \begin{cases} A_b^{-1} \cdot U_{b,i}^{*(1)(j)} & \text{if } c_{b,i}^{*(j)} = -1 \\ U_{b,i}^{*(1)(j)} & \text{if } c_{b,i}^{*(j)} = 1 \end{cases}$$

and

$$U_{b,i}^{*(1)(j)} = \begin{cases} A'_b \cdot H_{b,i}^{(1)(j)} & \text{if } c_{b,i}^{*(j)} = -1 \\ H_{b,i}^{(1)(j)} & \text{if } c_{b,i}^{*(j)} = 1. \end{cases}$$

As $A'_b = A_b$, it follows that $G_{b,i}^{(1)(j)} = H_{b,i}^{(1)(j)}$. A similar computation shows that $G_{b,i}^{(-1)(j)} = H_{b,i}^{(-1)(j)}$. We also have

$$d_{1-b,i}^{*(j)} = d_i^{*(j)} \cdot c_{b,i}^{*(j)} \quad \text{and} \quad c_{b,i}^{*(j)} = c_i^{*(j)} \cdot c_{1-b,i}^{*(j)}$$

where $\vec{d}^* = \vec{e}(\Phi_{\text{rand}, \vec{h}}(I), \text{rand}, \vec{h})$ and $\vec{c}^* = \vec{e}(I, \text{rand}, \vec{h})$. As $\Phi_{\text{rand}, \vec{h}}$ respects transcripts for successful tuples $(I, \text{rand}, \vec{h}) \in \text{Succ}$, $\vec{d}_i^* = \vec{c}_i^*$ and thus $d_{1-b,i}^{*(j)} = c_{1-b,i}^{*(j)}$.

It holds that

$$V_{1-b,i}^{*(1)(j)} = \begin{cases} A_{1-b}^{-1} \cdot H_{1-b,i}^{(1)(j)} & \text{if } d_{1-b,i}^{*(j)} = -1 \\ H_{1-b,i}^{(1)(j)} & \text{if } d_{1-b,i}^{*(j)} = 1 \end{cases}$$

and

$$H_{1-b,i}^{(1)(j)} := \begin{cases} A_{1-b} \cdot U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = -1 \\ U_{1-b,i}^{*(1)(j)} & \text{if } c_{1-b,i}^{*(j)} = 1 \end{cases}.$$

We have already established that $d_{1-b,i}^{*(j)} = c_{1-b,i}^{*(j)}$, so it clearly follows that $V_{1-b,i}^{*(1)(j)} = U_{1-b,i}^{*(1)(j)}$. As a similar computation shows that $V_{1-b,i}^{*(-1)(j)} = U_{1-b,i}^{*(-1)(j)}$.

We know that I and $\Phi_{\text{rand}, \vec{h}}(\Phi_{\text{rand}, \vec{h}}(I))$ are equal in all components except possibly $\vec{\Pi}$ and $\vec{\Pi}'$ and both induce the same first signer messages under rand and $\vec{h}$ by Lemma 67. From the definition of $\Phi_{\text{rand}, \vec{h}}$, we know that $\vec{\Theta}$ and $\vec{\Pi}'$ are defined uniquely by the fact that they maintain the Zero-Knowledge proofs in the first signer messages, so it must hold that $\vec{\Pi} = \vec{\Pi}'$.

We conclude that

$$\begin{aligned} \Phi_{\text{rand}, \vec{h}}(\Phi_{\text{rand}, \vec{h}}(I)) &= (b', A'_b, \text{pk}'_{1-b}, \overrightarrow{G_b^{(1)}}, \overrightarrow{G_b^{(-1)}}, \overrightarrow{d_{b-1}^*}, \overrightarrow{V_{1-b}^{*(1)}}, \overrightarrow{V_{1-b}^{*(-1)}}, \vec{\Pi}') \\ &= (b, A_b, \text{pk}_{1-b}, \overrightarrow{H_b^{(1)}}, \overrightarrow{H_b^{(-1)}}, \overrightarrow{c_{1-b}^*}, \overrightarrow{U_{1-b}^{*(1)}}, \overrightarrow{U_{1-b}^{*(-1)}}, \vec{\Pi}) \\ &= I, \end{aligned}$$

hence $\Phi_{\text{rand}, \vec{h}}$ is a bijection and self-inverse. □

Lemma 73. *Lemma 44 is satisfied for GABS.*

Proof. We have provided a definition for $\Phi_{\text{rand}, \vec{h}}$ and by Lemma 71 and Lemma 72 it is a required for Lemma 44 to hold. □

Witness extractors for GABS.

Definition 74 (Explicit witness extractors $(\text{Ext}_0, \text{Ext}_1)$). *Let I be a fixed instance and fix the value of rand . Let $(\vec{h}, \vec{h}') \in F_j(I, \text{rand})$ for some $j \in [k+1]$. Let $\sigma = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$ and $\sigma' = (\mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1)$ denote the signatures that correspond to the j -th entries of $\vec{h}$, respectively $\vec{h}'$, denoted by $\mathbf{c}^{(j)}$ and $\mathbf{c}'^{(j)}$. For all $i \in [\lambda]$, $c_{0,i} \cdot c_{1,i} = c_i^{(j)}$ and $c'_{0,i} \cdot c'_{1,i} = c_i'^{(j)}$. We adapt a definition for witness extractors $(\text{Ext}_0, \text{Ext}_1)$ from Figure 5. For $b \in \{0, 1\}$ let Ext_b be as in Figure 5, but if the output value A is unequal to $\perp$, then Ext_b outputs (b, A) instead of A .*

Lemma 75. *The witness extractors from Definition 74 satisfy the definition of witness extractors in Definition 43.*

Proof. We consider fixed values for an instance I and randomness rand . Let $(\vec{h}, \vec{h}') \in F_j(I, \text{rand})$ be a successful fork at index $j \in [k+1]$. Let $\sigma = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$ and $\sigma' = (\mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1)$ be as in Definition 74.

For $b \in \{0, 1\}$ and $i \in [\lambda]$ set

$$Y_{b,i} = \begin{cases} P_b * U_{b,i} & \text{if } c_{b,i} = -1 \\ Q * U_{b,i} & \text{if } c_{b,i} = 1 \end{cases} \quad \text{and} \quad Y'_{b,i} = \begin{cases} P_b * U'_{b,i} & \text{if } c'_{b,i} = -1 \\ Q * U'_{b,i} & \text{if } c'_{b,i} = 1 \end{cases}.$$

Since both signatures are accepting it holds that $c_i^{(j)} = c_{0,i} \cdot c_{1,i}$ and $c_i'^{(j)} = c'_{0,i} \cdot c'_{1,i}$ for all $i \in [\lambda]$ and for

$$\mathbf{c}^{(j)} = \mathcal{H}(\mathbf{Y}_0 || \mathbf{Y}_1 || m) \quad \text{and} \quad \mathbf{c}'^{(j)} = \mathcal{H}(\mathbf{Y}'_0 || \mathbf{Y}'_1 || m').$$

The vectors $\vec{h}$ and $\vec{h}'$ agree up to the j -th entry and the randomness of the challenger and the adversary is fixed, so the inputs of the j -th queries must agree. We find that $m = m'$, $\mathbf{Y}_0 = \mathbf{Y}'_0$ and $\mathbf{Y}_1 = \mathbf{Y}'_1$,

Let $b \in \{0, 1\}$. If there does not exist $t \in [\lambda]$ such that $c_{b,t} \neq c'_{b,t}$, then $\text{Ext}_b(\sigma, \sigma')$ outputs $\perp$. If there exists a value $t \in [\lambda]$ such that $c_{b,t} \neq c'_{b,t}$ then it follows from Lemma 61 that $\text{Ext}_b(\sigma, \sigma')$ outputs a b -side witness.

It holds that $\mathbf{c}^{(j)} \neq \mathbf{c}'^{(j)}$, and for $i \in [\lambda]$, $c_{0,i} \cdot c_{1,i} = c_i^{(j)}$ and $c'_{0,i} \cdot c'_{1,i} = c_i'^{(j)}$. It follows that there exists an index $t \in [\lambda]$ such that $c_{0,t} \neq c'_{0,t}$ or there exists

an index $t \in [\lambda]$ such that $c_{1,t} \neq c'_{1,t}$. Therefore, $\text{Ext}_0(\sigma, \sigma')$ or $\text{Ext}_1(\sigma, \sigma')$ must output a valid secret key and thus

$$(\text{Ext}_0(\sigma, \sigma'), \text{Ext}_1(\sigma, \sigma')) \in \{(\text{sk}_0, \perp), (\perp, \text{sk}_1), (\text{sk}_0, \text{sk}_1)\}.$$

We conclude that $(\text{Ext}_0, \text{Ext}_1)$ are witness extractors as in Definition 43. $\square$

Lemma 76. *Lemma 45 is true for the witness extractors $(\text{Ext}_0, \text{Ext}_1)$ from definition 74.*

Proof. We follow the proof in [KLLQ23, Lemma 4.5]. Consider a value $b \in \{0, 1\}$. In the proof of Lemma 75 we have seen that a b -side witness can be extracted from two successful tuples $(I, \text{rand}, \vec{h})$ and $(I, \text{rand}, \vec{h}')$ precisely when $\mathbf{c}_b \neq \mathbf{c}'_b$, where $\sigma = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$ and $\sigma' = (\mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1)$ are the signatures corresponding to h_j and h'_j , respectively.

We proceed via proof by contradiction. Let

$$((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h}'), (I, \text{rand}, \vec{h}'')) \in \Delta_j(I, \text{rand})$$

be a triangle at index j with respect to I and rand . Let $\sigma = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{U}_0, \mathbf{U}_1)$, $\sigma' = (\mathbf{c}'_0, \mathbf{c}'_1, \mathbf{U}'_0, \mathbf{U}'_1)$ and $\sigma'' = (\mathbf{c}''_0, \mathbf{c}''_1, \mathbf{U}''_0, \mathbf{U}''_1)$ denote the signatures corresponding to h_j , h'_j and h''_j , respectively. Suppose that a b -side witness can be extracted at index j from $((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h}'))$ while it cannot be extracted from $((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h}''))$ or $((I, \text{rand}, \vec{h}'), (I, \text{rand}, \vec{h}''))$. It follows that $\mathbf{c}_b = \mathbf{c}''$ and $\mathbf{c}'_b = \mathbf{c}''_b$, so that $\mathbf{c}_b = \mathbf{c}'_b$. The equality $\mathbf{c}_b = \mathbf{c}'_b$ implies that a b -side witness cannot be extracted from $((I, \text{rand}, \vec{h}), (I, \text{rand}, \vec{h}'))$ at index j , thus resulting in a contradiction.

We conclude that 45 is true for the witness extractors $(\text{Ext}_0, \text{Ext}_1)$ from Definition 74. $\square$

Theorem 77 (k -One-More Unforgeability). *If the GAIP_{IG} is hard, then the GABS blind signature protocol satisfies k -one-more unforgeability for any k in $\mathbb{N}$ that is logarithmic in the security parameter. More precisely, there exist universal constants C_1, C_2 in $\mathbb{R}_+$ such that for k, λ in $\mathbb{N}$, for any PPT adversary $\mathcal{A}$ making $Q_{\mathcal{H}}$ hash queries to a random oracle, if $\text{Adv}_{\mathcal{A}}^{k\text{-OMUF}}(\lambda) \geq C_1 \binom{Q_{\mathcal{H}}}{k+1} \cdot 2^{-\lambda}$, then*

$$\text{Adv}_{\mathcal{A}}^{k\text{-OMUF}}(\lambda) \leq \sqrt{\binom{Q_{\mathcal{H}}}{k+1}^2 \frac{(k+1)^3}{C_2} \text{Adv}_{\mathcal{A}}^{\text{GAIP}_{\text{IG}}}(\lambda)}.$$

Proof. By Lemma 65, the GABS construction satisfies the conditions in Definition 35. By Lemma 73, $\Phi_{\text{rand}, \vec{h}}$ is as is required for Lemma 44 to hold. By Definition 74 we obtain witness extractors as in Definition 43 for which Lemma 45 is satisfied.

The key generation for GABS relies on the instance generator IG, which we assume is hard to break.

Thus, the GABS blind signature scheme satisfies all required conditions to apply Theorem 46 and we obtain that it satisfies one-more unforgeability as in Definition 16

□

H Proofs for the GASDVS scheme

In this section we discuss the proofs for the security of the GASDVS construction.

H.1 Correctness of GASDVS.

Theorem 78 (Correctness of GASDVS). *The GASDVS scheme for strong designated verifier signatures satisfies correctness; i.e., there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_+$ such that for any λ in $\mathbb{N}$, $(\text{pk}_s, \text{sk}_s) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$, $(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$, and message $m \in \{0, 1\}^*$,*

$$\Pr \left[\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma, E) = 1 \mid (\sigma, E) \xleftarrow{\$} \text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m) \right] = 1 - \text{negl}(\lambda).$$

Proof. Let $\lambda \in \mathbb{N}$ and let $(\text{pk}_s, \text{sk}_s) \xleftarrow{\$} \text{KeyGen}_{\text{Sign}}(1^\lambda)$ and let $(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}_{\text{Verify}}(1^\lambda)$. Parse the keys as $P_s \leftarrow \text{pk}_s$, $A_s \leftarrow \text{sk}_s$, $(P_v, \text{pk}_K) \leftarrow \text{sk}_v$ and (A_v, sk_K) . Let $m \in \{0, 1\}^*$ and let $(\sigma, E) \xleftarrow{\$} \text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m)$. We show that

$$\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma, E) = 1,$$

that is, we show that $\mathbf{Y}'_s = \mathbf{Y}_s$, $\mathbf{Y}'_v = \mathbf{Y}_v$ and that for all $i \in [\lambda]$, $e_i = c_{s,i} \cdot c_{v,i}$, where $K \leftarrow \text{Decaps}(\text{sk}_K, c)$, σ is parsed as $(c_s, c_v, V, \mathbf{Y}_s, \mathbf{Y}_v) \leftarrow \sigma$, $(\mathbf{U}_s, \mathbf{U}_v) \leftarrow \text{Dec}(K, V)$, $\mathbf{Y}'_s$ and $\mathbf{Y}'_v$ are computed as

$$Y'_{s,i} = \begin{cases} P_s \cdot U_{s,i} & \text{if } c_{s,i} = -1 \\ Q \cdot U_{s,i} & \text{if } c_{s,i} = 1 \end{cases} \quad \text{and} \quad Y'_{v,i} = \begin{cases} P_v \cdot U_{v,i} & \text{if } c_{v,i} = -1 \\ Q \cdot U_{v,i} & \text{if } c_{v,i} = 1 \end{cases}.$$

and $\mathbf{e} = \mathcal{H}(m \| K \| E \| \mathbf{Y}_s \| \mathbf{Y}_v)$.

Let $\epsilon_{\text{IG}}^{\text{Fail}}$ denote the probability, that the instance generator IG outputs $\perp$. Note that $\epsilon_{\text{IG}}^{\text{Fail}}$ is negligible in λ . Let $\epsilon_{\text{KEM}}^{\text{Fail}}$ denote the probability that a KEM encapsulation does not decapsulate correctly and let $\epsilon_{\text{SKE}}^{\text{Fail}}$ denote the probability that an SKE encryption does not decrypt correctly. Both $\epsilon_{\text{KEM}}^{\text{Fail}}$ and $\epsilon_{\text{SKE}}^{\text{Fail}}$ are negligible in λ by the correctness of KEM and SKE.

If the decapsulation under KEM and the decryption under SKE are successful, then the value of $(\mathbf{U}_s, \mathbf{U}_v)$ that we obtain is equal to the value of $(\mathbf{U}_s, \mathbf{U}_v)$ as specified in the description of Sign except with probability bounded by $\epsilon_{\text{KEM}}^{\text{Fail}} + \epsilon_{\text{SKE}}^{\text{Fail}}$. If the value of $(\mathbf{U}_s, \mathbf{U}_v)$ is indeed as such, we are able to show that $\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma, c) = 1$, assuming that all calls to IG are successful.

Consider the computation of $\mathbf{Y}_s, \mathbf{Y}_v, \mathbf{c}_s, \mathbf{c}_v, \mathbf{U}_s, \mathbf{U}_v$ and $\mathbf{W}_s$ as specified in the GASDVS protocol for honest generation of (σ, E) . We first show that

$$\mathbf{Y}'_s = \mathbf{Y}_s \quad \text{and} \quad \mathbf{Y}'_v = \mathbf{Y}_v. \quad (7)$$

Observe that for all $i \in [\lambda]$, $(Y_{s,i}, W_{s,i})$ and $(Y_{v,i}, U_{v,i})$ are outputs of $\text{IG}(Q)$ or $\text{IG}(\text{pk}_v)$. In particular it holds that for all $i \in [\lambda]$,

$$Y_{s,i} = Q * W_{s,i} \quad \text{and} \quad Y_{v,i} = \begin{cases} P_v \cdot U_{v,i} & \text{if } c_{v,i} = -1 \\ Q \cdot U_{v,i} & \text{if } c_{v,i} = 1 \end{cases}.$$

Furthermore, for all $i \in [\lambda]$, $U_{s,i}$ is computed as

$$U_{s,i} = \begin{cases} A_s^{-1} \cdot W_{s,i} & \text{if } c_{s,i} = -1 \\ W_{s,i} & \text{if } c_{s,i} = 1 \end{cases},$$

so we also have that

$$\begin{aligned} Y_{s,i} = Q * W_{s,i} &= \begin{cases} Q * A_s \cdot A_s^{-1} \cdot W_{s,i} & \text{if } c_{s,i} = -1 \\ Q \cdot W_{s,i} & \text{if } c_{s,i} = 1 \end{cases} \\ &= \begin{cases} P_s \cdot U_{s,i} & \text{if } c_{s,i} = -1 \\ Q \cdot U_{s,i} & \text{if } c_{s,i} = 1 \end{cases}. \end{aligned}$$

It follows that the equations in (7) are satisfied.

Observe that $\mathbf{c}_s$ is computed based on $\mathbf{c}_v$ and $\mathbf{e} = \mathcal{H}(m \| K \| E \| \mathbf{Y}_s \| \mathbf{Y}_v)$ such that precisely $e_i = c_{s,i} \cdot c_{v,i}$ for all $i \in [\lambda]$. We conclude that $\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma, c) = 1$.

The generation of a GASDVS signature requires 2λ calls to IG , of which we need to consider the failure probability. It follows that

$$\begin{aligned} &\Pr \left[\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma, E) = 1 \mid (\sigma, E) \stackrel{s}{\leftarrow} \text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m) \right] \\ &\geq 1 - (2\lambda \cdot \epsilon_{\text{IG}}^{\text{Fail}} + \epsilon_{\text{KEM}}^{\text{Fail}} + \epsilon_{\text{SKE}}^{\text{Fail}}). \end{aligned}$$

□

H.2 Non-Transferability of GASDVS.

Theorem 79 (Non-Transferability of GASDVS). *The GASDVS scheme satisfies perfect non-transferability.*

Proof. Let $Q \in \mathcal{Q}$ be the same fixed value is in the GASDVS description and let $\lambda \in \mathbb{N}$ be the security parameter. We define the following PPT algorithm Sim :

$\text{Sim}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m) \rightarrow (\sigma, E)$: The algorithm

1. parses $(A_v, \text{sk}_K) \leftarrow \text{sk}_v, P_s \leftarrow \text{pk}_s, (P_v, \text{pk}_K) \leftarrow \text{pk}_v$;

2. sets $x = (P_s, P_v)$ and $w = (1, A_v)$,
 $(\text{st}, \rho = (\mathbf{Y}_s, \mathbf{Y}_v)) \xleftarrow{\$} \Sigma_{Q,Q,\lambda}^{\text{OR}}.\text{Commit}(x, w)$,
 $(K, E) \xleftarrow{\$} \text{KEM}.\text{Encaps}(\text{pk}_K)$ and $e \leftarrow \mathcal{H}(m \| K \| E \| \mathbf{Y}_s \| \mathbf{Y}_v)$;
3. computes $(\mathbf{c}_s, \mathbf{c}_v, \mathbf{U}_s, \mathbf{U}_v) \xleftarrow{\$} \Sigma_{Q,Q,\lambda}^{\text{OR}}.\text{Prove}(x, w, \text{st}, e)$
4. sets $V \xleftarrow{\$} \text{SKE}.\text{Enc}(K, (\mathbf{U}_s, \mathbf{U}_v))$, $\sigma \leftarrow (\mathbf{c}_s, \mathbf{c}_v, V, \mathbf{Y}_s, \mathbf{Y}_v)$;
5. returns (σ, E) .

Let m be in $\{0;1\}^*$, and let $(\text{pk}_s, \text{sk}_s) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$ and $(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$, where we parse $P_s \leftarrow \text{pk}_s$, $A_s \leftarrow \text{sk}_s$, $(P_v, \text{sk}_K) \leftarrow \text{pk}_v$ and $(A_v, \text{sk}_K) \leftarrow \text{sk}_v$. We show that the output distributions of $\text{Sim}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m)$ and $\text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m)$ are equal.

Observe that the only difference between the algorithms Sign and Sim is in the value of the witness w for the statement $x = (P_s, P_v)$. For Sign , w takes the value $(0, A_s)$ and for Sim , w takes the value $(1, A_v)$.

The algorithms Sign and Sim both function by generating a non-interactive proof $\pi = (\mathbf{c}_s, \mathbf{c}_v, \mathbf{U}_s, \mathbf{U}_v)$ based on the $\Sigma_{Q,Q,\lambda}^{\text{OR}}$ protocol for the statement witness pair (x, w) . Additionally, the value of $(\mathbf{U}_s, \mathbf{U}_v)$ is encrypted and the values of $\mathbf{Y}_s$ and $\mathbf{Y}_v$, which can be directly computed from $\mathbf{c}_s, \mathbf{c}_v, \mathbf{U}_s$ and $\mathbf{U}_v$ are added to the signature. The $\Sigma_{Q,Q,\lambda}^{\text{OR}}$ proof protocol satisfies perfect honest verifier zero knowledge (Theorem 26) and is therefore perfectly witness indistinguishable. It follows that the proofs generated internally by $\text{Sim}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m)$ and $\text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m)$ are statistically indistinguishable. Therefore, the σ components of the signatures they generate, which are such proofs with encrypted $(\mathbf{U}_s, \mathbf{U}_v)$ values and added $(\mathbf{Y}_s, \mathbf{Y}_v)$ values, are also statistically indistinguishable.

The keys for encryption under SKE in Sign and Sim are both generated as $(K, E) \xleftarrow{\$} \text{KEM}.\text{Encaps}(\text{pk}_K)$, so the values for E and E' in $(\sigma, E) \xleftarrow{\$} \text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m)$ and $(\sigma', E') \xleftarrow{\$} \text{Sim}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m)$ are also equally distributed. Therefore, the output distributions of $\text{Sim}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m)$ and $\text{Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m)$ are equal. $\square$

H.3 Strong Unforgeability of GASDVS.

Theorem 80 (Strong Unforgeability of GASDVS). *If the hardness of the GAIP_{IG} holds, then the GASDVS protocol satisfies strong unforgeability in the random oracle model; more precisely, given a PPT adversary $\mathcal{A}$ making q random oracle queries and security parameter λ in $\mathbb{N}$,*

$$\text{Adv}_{\mathcal{A}}^{\text{SUF}}(\lambda) \leq \sqrt{2q \text{Adv}_{\mathcal{A}}^{\text{GAIP}_{\text{IG}}}(\lambda)}.$$

Proof. Adapting the proof strategy of [MTT25] to our KEM-based construction, given a PPT algorithm $\mathcal{A}$ that would have a non-negligible advantage $\text{Adv}_{\mathcal{A}}^{\text{SUF}}$ in the strong unforgeability game for GASDVS, the reduction PPT $\mathcal{R}$ described in this proof will show that it may be used to build an efficient adversary with

non-negligible advantage against the GAIP_{IG}. For λ a security parameter in $\mathbb{N}$, let $*$: $\mathcal{Q} * \mathbb{G} \rightarrow \mathbb{Q}$ be the corresponding group action for IG and let $Q \in \mathcal{Q}$ be fixed. We consider the corresponding NP relation $\mathcal{R}_{Q,\lambda}$ (Definition 22).

Construction of a Simulating Algorithm $\mathcal{W}$. To use the forking Lemma 53 (Appendix E), the ordinary challenger for the strong unforgeability game will be replaced by another PPT routine $\mathcal{W}$, in an indistinguishable manner for $\mathcal{A}$, without using either the signer or the verifier's secret key. This algorithm will take as input a tuple $(x, \mathbf{e}_1, \dots, \mathbf{e}_q)$, where q is the total number of queries that $\mathcal{A}$ makes to the oracles $\mathcal{O}^{\text{Sign}}$, $\mathcal{O}^{\text{Verify}}$ or $\mathcal{O}^{\mathcal{H}}$, and $x \in \mathcal{Q}$ is a GAIP_{IG} instance, generated according to IG. The routines of $\mathcal{W}$ are described hereafter.

Challenge initialisation: on input $x = Q'$, $\mathbf{e}_1, \dots, \mathbf{e}_q \in \{-1; 1\}^\lambda$, $\mathcal{W}$

1. sets a hash table $\mathbb{T} \leftarrow \{\}$, a list $L^{\text{Sign}} \leftarrow \{\}$, $\text{ctr} \leftarrow 0$ and $\alpha \xleftarrow{\$} \{0, 1\}$;
2. if $\alpha = 0$, sets $P_s \leftarrow Q'$ and $((P_v, \text{pk}_K), (A_v, \text{pk}_K)) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$ and ;
else, $P_v \leftarrow Q'$ and $((P_s, \text{pk}_K), (A_s, \text{pk}_K)) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$;
3. sets $\text{pk}_s \leftarrow P_s$, $\text{sk}_s \leftarrow A_s$, $\text{pk}_v \leftarrow (P_v, \text{pk}_K)$ and $\text{sk}_v \leftarrow (A_v, \text{sk}_K)$.
4. gives input $(\text{pk}_s, \text{pk}_v)$ to $\mathcal{A}$ and simulates oracles queries for $\mathcal{A}$.

Simulation of $\mathcal{O}^{\mathcal{H}}$: on input a query $(m, K, E, \mathbf{Y}_s, \mathbf{Y}_v)$ to $\mathcal{O}^{\mathcal{H}}$, $\mathcal{W}$ checks whether $\mathbb{T}[m, K, E, \mathbf{Y}_s, \mathbf{Y}_v]$ exists, and returns the corresponding value $\mathbf{e}$ if it does; else sets $\text{ctr} \leftarrow \text{ctr} + 1$, $\mathbb{T}[m, K, E, \mathbf{Y}_s, \mathbf{Y}_v] \leftarrow \mathbf{e}_{\text{ctr}}$ and returns $\mathbf{e}_{\text{ctr}}$.

Simulation of $\mathcal{O}^{\text{Sign}}$: when receiving a query m to $\mathcal{O}^{\text{Sign}}$, $\mathcal{W}$ uses the algorithm Sim from the proof for Theorem 79, replacing its execution of the hash evaluation by a query to the simulator of $\mathcal{O}^{\mathcal{H}}$.

1. If $\alpha = 0$, $\mathcal{W}$ sets $(\sigma, E) \xleftarrow{\$} \text{Sim}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m)$,
else, $(\sigma, E) \xleftarrow{\$} \text{GASDVS.Sign}(\text{sk}_s, \text{pk}_s, \text{pk}_v, m)$;
2. it sets $L^{\text{Sign}} \leftarrow L^{\text{Sign}} \cup \{(m, (\sigma, E))\}$ and returns (σ, E) .

Simulation of $\mathcal{O}^{\text{Verify}}$: on input a query $(m, (\sigma, E))$ for $\mathcal{O}^{\text{Verify}}$, $\mathcal{W}$ runs GASDVS.Verify($\text{sk}_v, \text{pk}_s, \text{pk}_v, m, \sigma, E$), replacing its execution of a hash evaluation by a query to the simulator of $\mathcal{O}^{\mathcal{H}}$, and returning the output b .

Final procedure: on input $(m^*, (\sigma^*, E^*))$ received from $\mathcal{A}$, setting $K^* \leftarrow \text{KEM.Decaps}(\text{sk}_K, E^*)$ and parsing $(\mathbf{c}_s, \mathbf{c}_v, V, \mathbf{Y}_s, \mathbf{Y}_v) \leftarrow \sigma^*$ the algorithm:

1. sets $(\mathbf{U}_s, \mathbf{U}_v) \leftarrow \text{SKE.Dec}(K, V)$ and $\pi^* \leftarrow (\mathbf{c}_s, \mathbf{c}_v, \mathbf{U}_s, \mathbf{U}_v)$;
2. aborts if $\mathbb{T}[m^*, K^*, E^*, \mathbf{Y}_s, \mathbf{Y}_v]$ does not exist; else if $(m^*, (\sigma^*, E^*)) \in L^{\text{Sign}}$ or $\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m^*, \sigma^*, E^*) \neq 1$, returns $(0, \perp)$;
3. for i^* an index such that $\mathbf{e}_{i^*} = \mathbb{T}[m^*, K^*, E^*, \mathbf{Y}_s, \mathbf{Y}_v]$, returns $(i^*, (\pi^*, \alpha, m^*, \sigma^*, E^*, K^*))$.

Construction of $\mathcal{R}$. This algorithm will take as input a GAIP_{IG} challenge (Q, Q') and runs $\text{Fork}^{\mathcal{W}}$ to extract a solution. Note that Ext_0 and Ext_1 from Figure 5 can also be interpreted as Ext_s and Ext_v , by changing the binary indices to $\{s, v\}$ ones.

$\mathcal{R}(Q') \rightarrow A$: given (Q') ,

1. sets $(d, \text{out}, \widetilde{\text{out}}) \xleftarrow{\$} \text{Fork}^{\mathcal{W}}(Q')$ and aborts if $d = 0$;
2. parses $(\pi^*, \alpha, m^*, \sigma^*, E^*, K^*) \leftarrow \text{out}$ and $(\widetilde{\pi}^*, \widetilde{\alpha}, \widetilde{m}^*, \widetilde{\sigma}^*, \widetilde{E}^*, \widetilde{K}^*) \leftarrow \widetilde{\text{out}}$;
3. if $\alpha = 0$, sets $A_s \leftarrow \text{Ext}_s(\pi^*, \widetilde{\pi}^*)$, else, $A_v \leftarrow \text{Ext}_v(\pi^*, \widetilde{\pi}^*)$;
4. returns A (or $\perp$ if this was the extractor's output).

Analysis of $\mathcal{W}$. Let us first show that $\mathcal{W}$ perfectly simulates the view of $\mathcal{A}$. For the initialisation, the input to $\mathcal{W}$ is received from the GAIP_{IG} challenger, generating $Q' \xleftarrow{\$} \text{IG}(Q)$, so all keys are distributed exactly as when running GASDVS.KeyGen; then, in the random oracle model, the $\mathcal{W}$'s simulation of $\mathcal{O}^{\mathcal{H}}$ and $\mathcal{O}^{\text{Verify}}$ is perfect; when simulating $\mathcal{O}^{\text{Sign}}$, if $\alpha = 1$, the simulation uses GASDVS.Sign to sign, else for $\alpha = 0$, Theorem 79 shows that $\text{Sim}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m)$ perfectly simulates the signing algorithm.

The forking lemma 53's $\text{acc}(\lambda)$ function corresponds to the probability that $\mathcal{W}$ output (j, out) with $j \geq 1$. This event occurs when $\mathcal{A}$ outputs a valid forgery, with at least one corresponding hash query; without such a query, its advantage is bounded by $2^{-\lambda}$, hence $\text{acc}(\lambda) \geq \text{Adv}_{\mathcal{A}}^{\text{SUF}}(\lambda) - \frac{1}{2^\lambda}$.

Analysis of $\mathcal{R}$. Consider an output $(d, \text{out}, \widetilde{\text{out}}) \xleftarrow{\$} \text{Fork}^{\mathcal{W}}(Q')$, parsed as in the description of $\mathcal{R}$ while additionally parsing $(\mathbf{c}_s, \mathbf{c}_v, V, \mathbf{Y}_s, \mathbf{Y}_c) \leftarrow \sigma^*$ and $(\widetilde{\mathbf{c}}_s, \widetilde{\mathbf{c}}_v, \widetilde{V}, \widetilde{\mathbf{Y}}_s, \widetilde{\mathbf{Y}}_c) \leftarrow \widetilde{\sigma}^*$. If $d = 1$, the challenges of the signatures (σ^*, E^*) and $(\widetilde{\sigma}^*, E^*)$ corresponding to out and $\widetilde{\text{out}}$ differ, *i.e.* $e^* \neq \widetilde{e}^*$, while $\text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m^*, \sigma^*, E^*) = \text{Verify}(\text{sk}_v, \text{pk}_s, \text{pk}_v, m^*, \widetilde{\sigma}^*, E^*) = 1$, and $(m^*, (\sigma^*, E^*)) \notin L^{\text{Sign}}, (m^*, (\widetilde{\sigma}^*, E^*)) \notin \widetilde{L}^{\text{Sign}}$, with the equality of keys, message and encapsulation stemming from the fact that $\mathcal{W}$ uses the same randomness in both cases, with identical inputs until e^* and $\widetilde{e}^*$ are queried, and thus also identical states until that point. Via similar reason it also holds that $\mathbf{Y}_s = \widetilde{\mathbf{Y}}_s, \mathbf{Y}_v = \widetilde{\mathbf{Y}}_v$. By validity of the forgeries it holds that for all $i \in [\lambda]$, $c_{s,i} \cdot c_{v,i} = e_i$ and $\widetilde{c}_{s,i} \cdot \widetilde{c}_{v,i} = \widetilde{e}_i$. Since $e^* \neq \widetilde{e}^*$ it follows that $\mathbf{c}_s \neq \widetilde{\mathbf{c}}_s$ or $\mathbf{c}_v \neq \widetilde{\mathbf{c}}_v$. Calling Ext_s and Ext_v on $(\pi^*, \widetilde{\pi}^*)$ hence produces at least one successful extraction of either A_s such that $P_s = Q * A_s$ for $P_s = \text{pk}_s$ or A_v such that $P_v = Q * A_v$ for $\text{pk}_v = (P_v, \text{pk}_K)$, by Lemma 61.

Let us now analyse the probability that $\mathcal{R}$ solves the GAIP_{IG} instance, that is, it outputs A such that $(Q', A) \in \mathcal{R}_{Q,\lambda}$, *i.e.* $Q' = Q * A$. If $d = 0$ the extraction fails, and if $d = 1$ the following will hold: if the extractors found A_s such that $P_s = Q * A_s$, then this is a valid witness for the GAIP_{IG} problem if $\alpha = 0$, which occurs with probability one half; else, they found A_v such that $P_v = Q * A_v$, which is a valid witness for Q' if $\alpha = 1$. Thus, the forking lemma 53 (Appendix E) provides

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{GAIP}_{\text{IG}}}(\lambda) &\geq \frac{1}{2} \cdot \text{frk}(\lambda) \geq \frac{1}{2} \cdot \text{acc}(\lambda) \cdot \left(\frac{\text{acc}(\lambda)}{q} - \frac{1}{2^\lambda} \right) \\ &\geq \frac{1}{2q} \cdot \left(\text{Adv}_{\mathcal{A}}^{\text{SUF}}(\lambda) - \frac{1}{2^\lambda} \right) \cdot \left(\text{Adv}_{\mathcal{A}}^{\text{SUF}}(\lambda) - \frac{q+1}{2^\lambda} \right). \end{aligned}$$

□

H.4 Privacy of Signer Identities of GASDVS.

Theorem 81 (Privacy of Signer's Identity of GASDVS). *The GASDVS protocol satisfies privacy of signer's identity. More precisely, in the random oracle*

model, for any PPT adversary $\mathcal{A}$ and security parameter λ in $\mathbb{N}$, denoting Q_s the number of queries of $\mathcal{A}$ to the signing oracle $\mathcal{O}^{\text{Sign}}$,

$$\text{Adv}_{\mathcal{A}}^{\text{PSI}}(\lambda) \leq 2 \cdot \text{Adv}_{\mathcal{A}}^{\text{SK-CCA}}(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{IND-EAV}}(\lambda) + \frac{Q_s}{2^\lambda} + \text{Adv}_{\mathcal{A}}^{\text{SUF}}(\lambda).$$

Proof. Let $\mathcal{A}$ be a PPT adversary, and λ in $\mathbb{N}$ a security parameter. Consider the following game sequence.

Game G_0 : is the original privacy of the signer identity game. The adversary's advantage is denoted $\text{Adv}_{\mathcal{A}}^{\text{PSI}}(\lambda)$.

Game G_1 : is as the previous game, but the challenger $\mathcal{C}$ now does not receive sk_v, sk_0 and sk_1 at the beginning of the game, and replaces the generation of σ^* by querying $\mathcal{O}^{\text{Sign}}$ on (b, m^*) and setting its output as σ^* . This is a perfectly indistinguishable change for the adversary: $\text{Adv}_{\mathcal{A}}^{G_1}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{PSI}}(\lambda)$;

Game G_2 : is as the previous game, but $\mathcal{C}$ now changes the simulation of the oracles for the adversary: it initiates an empty list $\mathcal{L}^{\text{Sign}}$, and for each query (β, m) the adversary issues to $\mathcal{O}^{\text{Sign}}$, it registers the output signature σ with $\mathcal{L}^{\text{Sign}} \leftarrow \mathcal{L}^{\text{Sign}} \cup \{(m, \sigma)\}$. Then, when $\mathcal{O}^{\text{Verify}}$ or $\mathcal{O}^{\text{Verify}^*}$ are queried with (β', m', σ') , if (m', σ') is not in $\mathcal{L}^{\text{Sign}}$, it replaces the output of the original oracle by 0 (except if (m^*, σ^*) is queried to $\mathcal{O}^{\text{Verify}^*}$: then it still returns $\perp$). The adversary's advantage in distinguishing between this game and the previous one is bounded by its advantage in the strong unforgeability game: $|\text{Adv}_{\mathcal{A}}^{G_2}(\lambda) - \text{Adv}_{\mathcal{A}}^{G_1}(\lambda)| \leq \text{Adv}_{\mathcal{A}}^{\text{SUF}}(\lambda)$.

Game G_3 : is as the previous game, except that now, $\mathcal{O}^{\text{Sign}}$ is replaced by a simulator running its normal procedure and sending not only $\sigma^* = (s^*, E^*)$ to $\mathcal{C}$ but also the session key K^* corresponding to E^* . $\mathcal{C}$ aborts if there exist m and s such that $(m, (s, E^*))$ is in $\mathcal{L}^{\text{Sign}}$. If in any of the subsequent requests of $\mathcal{A}$ to $\mathcal{O}^{\text{Sign}}$, the oracle outputs a signature of the form $(m, (s, E^*))$, it also aborts. The difference between this game and the previous one consists in events that would allow $\mathcal{C}$ to win the SK-CCA game for E^* (with (E^*, K^*) corresponding to an encapsulation generation using pk_K that would yield the session key corresponding to the challenge E^* received as answer to another signing request), hence $|\text{Adv}_{\mathcal{A}}^{G_3}(\lambda) - \text{Adv}_{\mathcal{A}}^{G_2}(\lambda)| \leq \text{Adv}_{\mathcal{A}}^{\text{SK-CCA}}(\lambda)$.

Game G_4 : is as the previous game, except that $\mathcal{C}$ does not receive K^* anymore, which does not change the view of $\mathcal{A}$: $\text{Adv}_{\mathcal{A}}^{G_4}(\lambda) = \text{Adv}_{\mathcal{A}}^{G_3}(\lambda)$.

Game G_5 : is as the previous game, except that the simulator of $\mathcal{O}^{\text{Sign}}(b, m^*)$ now draws $\tilde{K} \xleftarrow{\$} \{0; 1\}^\lambda$ and sets $V' \xleftarrow{\$} \text{SKE.Enc}(\tilde{K}, (U_s^*, U_v^*))$ instead of the usual computation with the KEM session-key before returning $\sigma^* = (c_s^*, c_v^*, V', Y_s^*, Y_v^*, E^*)$ to $\mathcal{C}$. The adversary's advantage in the KEM SK-CCA game bounds the one it has in distinguishing the current game from the previous one: $|\text{Adv}_{\mathcal{A}}^{G_5}(\lambda) - \text{Adv}_{\mathcal{A}}^{G_4}(\lambda)| \leq \text{Adv}_{\mathcal{A}}^{\text{SK-CCA}}(\lambda)$.

Game G_6 : is as the previous game, except that the simulator for $\mathcal{O}^{\text{Sign}}$ now aborts if any of the session-keys on other inputs than (b, m^*) are equal to $\tilde{K}$. Denoting Q_s the number of queries of $\mathcal{A}$ to $\mathcal{O}^{\text{Sign}}$, $|\text{Adv}_{\mathcal{A}}^{G_6}(\lambda) - \text{Adv}_{\mathcal{A}}^{G_5}(\lambda)| \leq \frac{Q_s}{2^\lambda}$.

Game G_7 : is as the previous game, except that now, the simulator of $\mathcal{O}^{\text{Sign}}(b, m^*)$ for $\mathcal{C}$, denoting $\mathbf{I}$ the neutral element of $\mathbb{G}$, sets $\tilde{V} \xleftarrow{\$} \text{SKE.Enc}(\tilde{K}, (\mathbf{I}, \mathbf{I}))$ and returns $\sigma^* = (\mathbf{c}_s^*, \mathbf{c}_v^*, \tilde{V}, \mathbf{Y}_s^*, \mathbf{Y}_v^*, E^*)$ to $\mathcal{C}$. $\mathcal{A}$'s advantage in distinguishing this situation from the previous game is bounded by its IND-EAV advantage: $|\text{Adv}_{\mathcal{A}}^{G_7}(\lambda) - \text{Adv}_{\mathcal{A}}^{G_6}(\lambda)| \leq \text{Adv}_{\mathcal{A}}^{\text{IND-EAV}}(\lambda)$.

Game G_8 : is as the previous game, except that now $\mathcal{C}$ draws $\tilde{\mathbf{c}}_s$ and $\tilde{\mathbf{c}}_v$ uniformly at random from $\{0;1\}^\lambda$ and replaces these values in the challenge signature $\sigma^* = (\tilde{\mathbf{c}}_s, \tilde{\mathbf{c}}_v, \tilde{V}, \mathbf{Y}_s^*, \mathbf{Y}_v^*, E^*)$ it returns to $\mathcal{A}$. In the random oracle model, this is perfectly indistinguishable from the previous game, thus $\text{Adv}_{\mathcal{A}}^{G_8}(\lambda) = \text{Adv}_{\mathcal{A}}^{G_7}(\lambda)$.

Game G_9 : is as the previous game, except that now $\mathcal{C}$ draws $\tilde{\mathbf{Y}}_s, \tilde{\mathbf{Y}}_v \xleftarrow{\$} \text{IG}(Q)$ and replaces these values in the challenge signature $\sigma^* = (\tilde{\mathbf{c}}_s, \tilde{\mathbf{c}}_v, \tilde{V}, \tilde{\mathbf{Y}}_s, \tilde{\mathbf{Y}}_v, E^*)$ it returns to $\mathcal{A}$. As IG is an instance generator satisfying definition 3, this game is perfectly indistinguishable from the previous one: $\text{Adv}_{\mathcal{A}}^{G_9}(\lambda) = \text{Adv}_{\mathcal{A}}^{G_8}(\lambda)$.

Game G_{10} : is as the previous game, but now $\mathcal{C}$ does not query $\mathcal{O}^{\text{Sign}}(b, m^*)$, sets $(\tilde{k}, \tilde{E}) \xleftarrow{\$} \text{KEM.Encaps}(\text{pk}_K)$, aborts if there is an element of the form $(m, (s, \tilde{E}))$ in $\mathcal{L}^{\text{Sign}}$, and sets $\sigma^* = (\tilde{\mathbf{c}}_s, \tilde{\mathbf{c}}_v, \tilde{V}, \tilde{\mathbf{Y}}_s, \tilde{\mathbf{Y}}_v, \tilde{E})$ before returning σ^* to $\mathcal{A}$. Then, it aborts if any of the adversary's requests to $\mathcal{O}^{\text{Sign}}$ results in a new element of the form $(m, (s, \tilde{E}))$ in $\mathcal{L}^{\text{Sign}}$. The adversary's view is not modified, so $\text{Adv}_{\mathcal{A}}^{G_{10}}(\lambda) = \text{Adv}_{\mathcal{A}}^{G_9}(\lambda)$, and moreover, in this last game, $\mathcal{C}$ never uses b , and as $\mathcal{A}$ receives no element dependent on this bit, her advantage is null.

Finally, $\text{Adv}_{\mathcal{A}}^{\text{PSI}}(\lambda) \leq 2 \cdot \text{Adv}_{\mathcal{A}}^{\text{SK-CCA}}(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{IND-EAV}}(\lambda) + \frac{Q_s}{2^\lambda} + \text{Adv}_{\mathcal{A}}^{\text{SUF}}(\lambda)$. $\square$

H.5 Non-Delegatability of GASDVS.

Theorem 82 (Non-Delegatability of GASDVS). *The GASDVS protocol satisfies non-delegatability with knowledge error $2^{-\lambda}$ in the random oracle model if the hardness of the GAIP_{IG} holds.*

Proof. Adapting the proof strategy of [MTT25] to our KEM-based construction, given a PPT forger algorithm $\mathcal{F}$ that on input m^* has success probability ϵ of outputting a valid GASDVS signature for m^* , the extractor Ext described in this proof will show that it may be used to build an efficient extractor that retrieves signer secret key $\text{sk}_s = A_s$ or the A_v component of the verifier secret key.

Additionally, our hash oracle $\mathcal{O}^{\mathcal{H}}$ maintains a list of keys K that are used as input components for hash queries, so that they can be retrieved in polynomial time by searching the list. This allows us to find a key K^* that decrypts the value V as (U_s, U_s) such that an adapted verification algorithm is satisfied.

For λ a security parameter in $\mathbb{N}$, let $\mathcal{Q}$ be the corresponding set for IG and let $Q \in \mathcal{Q}$ be fixed. Let two pairs of keys be given by $(\text{pk}_s, \text{sk}_s) \xleftarrow{\$} \text{KeyGen}(1^\lambda, s)$ and $(\text{pk}_v, \text{sk}_v) \xleftarrow{\$} \text{KeyGen}(1^\lambda, v)$, where we pare $P_s \leftarrow \text{pk}_s, A_s \leftarrow \text{sk}_s, (P_v, \text{pk}_K) \leftarrow \text{pk}_v$

and $(A_v, \text{sk}_K) \leftarrow \text{sk}_v$. Let q denote the amount of hash queries that $\mathcal{F}_{m^*}$ makes to $\mathcal{O}^{\mathcal{H}}$ and let $\kappa := 2^{-\lambda}$ where $2^{-\lambda}$ bounds the probability that the adversary generates a forgery without having queried the hash value for the verification of that forgery. Let ϵ denote the success probability of $\mathcal{F}_{m^*}$.

In order to describe Ext, we make use of the $\text{Fork}^{\mathcal{W}}$ algorithm (Figure 8). For the analysis of Ext we use the forking Lemma 53 (Appendix E). Let us first describe the algorithm $\mathcal{W}^{\mathcal{F}_{m^*}}$.

Description of $\mathcal{W}^{\mathcal{F}_{m^}}$.* On input $(m^*, e_1, \dots, e_q)$, the algorithm $\mathcal{W}$ runs $\mathcal{F}_{m^*}$ and simulates the required hash queries. The values $e_1, \dots, e_q \in \{0, 1\}^\lambda$ are output values for the simulated hash queries.

Challenge initialisation: on input $(m^*, e_1, \dots, e_q)$, $\mathcal{W}$ initialises a hash table $\mathbb{T} \leftarrow \{\}$, a counter $\text{ctr} \leftarrow 0$ and a list $\mathcal{L} \leftarrow \{\}$.

Simulation of $\mathcal{O}^{\mathcal{H}}$: on input (m, K, E, Y_s, Y_v) the simulation is as in the proof for Theorem 80 and additionally sets $\mathcal{L} \leftarrow \mathcal{L} \cup \{K\}$.

Final procedure: $\mathcal{W}$ runs $(m^*, \sigma^*, E^*) \xleftarrow{\$} \mathcal{F}_{m^*}$; searches for $K^* \in \mathcal{L}$ such that $\widetilde{\text{Verify}}(\text{pk}_s, \text{pk}_v, K^*, m^*, \sigma^*) = 1$, where $\widetilde{\text{Verify}}$ is as GASDVS.Verify except that it uses K^* directly as the SKE key instead of using sk_K and retrieving it from decapsulation of E^* ; aborts if no such K^* is found; parses $(c_s, c_v, V, Y_s, Y_v) \leftarrow \sigma^*$; if $\mathbb{T}[m^*, K^*, E^*, Y_s, Y_v]$ does not exist then $\mathcal{W}$ aborts; otherwise, $\mathcal{W}$ computes $(U_s, U_v) \leftarrow \text{SKE.Dec}(K^*, V)$, sets $\pi^* \leftarrow (c_s, c_v, U_s, U_v)$ and returns $(i^*, (\pi^*, m^*, \sigma^*, E^*, K^*, Y_s, Y_v))$, where i^* is an index such that $\mathbb{T}[m^*, K^*, E^*, Y_s, Y_v] = e_{i^*}$.

Description of Ext. This algorithm takes as input a message m^* and runs $\text{Fork}^{\mathcal{W}^{\mathcal{F}_{m^*}}}$ to extract two GASDVS forgeries that correspond to the same hash query. One of the secret keys can then be extracted from these forgeries. Note that Ext_0 and Ext_1 from Figure 5 can also be interpreted as Ext_s and Ext_v , by changing the binary indices to $\{s, v\}$ ones.

$\text{Ext}(m^*) \rightarrow A$: given m^* ,

1. sets $(d, \text{out}, \widetilde{\text{out}}) \xleftarrow{\$} \text{Fork}^{\mathcal{W}^{\mathcal{F}_{m^*}}}(m^*)$ and aborts if $d = 0$;
2. parses $(\pi^*, m^*, \sigma^*, E^*, K^*) \leftarrow \text{out}$, $(\widetilde{\pi}^*, \widetilde{m}^*, \widetilde{\sigma}^*, \widetilde{E}^*, \widetilde{K}^*) \leftarrow \widetilde{\text{out}}$, $(c_s, c_v, V, Y_s, Y_v) \leftarrow \sigma^*$ and $(\widetilde{c}_s, \widetilde{c}_v, \widetilde{V}, \widetilde{Y}_s, \widetilde{Y}_v) \leftarrow \widetilde{\sigma}^*$;
3. if $c_s \neq \widetilde{c}_s$, sets $A'_s \leftarrow \text{Ext}_s(\pi^*, \widetilde{\pi}^*)$, if $c_v \neq \widetilde{c}_v$, sets $A'_v \leftarrow \text{Ext}_v(\pi^*, \widetilde{\pi}^*)$;
4. returns A'_s if $A'_s \neq \perp$ or A'_v if $A'_v \neq \perp$ (or $\perp$ if this was both extractor's output).

Analysis of $\mathcal{W}$. In the random oracle model, $\mathcal{W}^{\mathcal{F}_{m^*}}$'s simulation of $\mathcal{O}^{\mathcal{H}}$ is perfect. Therefore, $\mathcal{W}^{\mathcal{F}_{m^*}}$ perfectly simulated the view of $\mathcal{F}_{m^*}$.

Suppose $\mathcal{F}_{m^*}$ outputs a valid forgery (σ^*, E^*) . The key $K^* = \text{Decaps}(\text{sk}_K, E^*)$ must be in $\mathcal{L}$ if $\mathcal{F}_{m^*}$ made the hash query corresponding to (σ^*, E^*) . The condition that $\widetilde{\text{Verify}}(\text{pk}_s, \text{pk}_v, K^*, m^*, \sigma^*) = 1$ is clearly satisfied for $K^* = \text{Decaps}(\text{sk}_K, E^*)$.

The forking lemma's (Lemma 53) $\text{acc}(\lambda)$ function corresponds to the probability that $\mathcal{W}^{\mathcal{F}_{m^*}}$ output (j, out) with $j \geq 1$. This event occurs when $\mathcal{F}_{m^*}$ outputs a valid forgery, with at least one corresponding hash query; without such a query, its advantage is bounded by $2^{-\lambda}$, hence $\text{acc}(\lambda) \geq \epsilon - \frac{1}{2^\lambda}$.

Analysis of Ext. Consider an output $(d, \text{out}, \widetilde{\text{out}}) \stackrel{\$}{\leftarrow} \text{Fork}^{\mathcal{W}^{\mathcal{F}_{m^*}}}(m^*)$, parsed as in the description of Ext. If $d = 1$, the challenges, for verification under $\widetilde{\text{Verify}}$, of the signatures (m^*, σ^*, E^*) and $(m^*, \widetilde{\sigma}^*, E^*)$ differ, i.e. $e^* \neq \widetilde{e}^*$, while $\widetilde{\text{Verify}}(\text{pk}_s, \text{pk}_v, K^*, m^*, \sigma^*, E^*) = \widetilde{\text{Verify}}(\text{pk}_s, \text{pk}_v, \widetilde{K}^*, m^*, \widetilde{\sigma}^*, E^*) = 1$, with the equality of keys, message and encapsulation stemming from the fact that $\mathcal{W}^{\mathcal{F}_{m^*}}$ uses the same randomness in both cases, with identical inputs until e^* and $\widetilde{e}^*$ are queried, and thus also identical states until that point. Via similar reason it also holds that $Y_s = \widetilde{Y}_s$ and $Y_v = \widetilde{Y}_v$. It holds that $\widetilde{\text{Verify}}(\text{pk}_s, \text{pk}_v, K^*, m^*, \sigma^*, E^*) = \widetilde{\text{Verify}}(\text{pk}_s, \text{pk}_v, \widetilde{K}^*, m^*, \widetilde{\sigma}^*, E^*) = 1$ so that for all $i \in [\lambda]$, $c_{s,i} \cdot c_{v,i} = e_i$ and $\widetilde{c}_{s,i} \cdot \widetilde{c}_{v,i} = \widetilde{e}_i$. Since $e^* \neq \widetilde{e}^*$ it follows that $c_s \neq \widetilde{c}_s$ or $c_v \neq \widetilde{c}_v$. Calling Ext_s and Ext_v on $(\pi^*, \widetilde{\pi}^*)$ hence produces at least one successful extraction of either A'_s such that $P_s = Q * A'_s$ or A'_v such that $P_v = Q * A'_v$ by Lemma 61.

Let us now analyse the probability that Ext outputs $\text{sk}_s = A_s$ or A_v where $\text{sk}_v = (A_v, \text{sk}_K)$, denoted by ϵ_{Ext} . Recall that $\text{pk}_s = P_s$ and $\text{pk}_v = (P_v, \text{pk}_K)$. If $d = 0$ the extraction fails, and if $d = 1$ the following will hold: if the extractors found A'_s such that $P_s = Q * A'_s$, it follows by freeness of the group action that $A'_s = A_s = \text{sk}_s$; else, they found A'_v such that $P_v = Q * A'_v$, in which case it follows that $A'_v = A_v$, where $\text{sk}_v = (A_v, \text{sk}_K)$. Thus, the forking lemma 53 (Appendix E) provides

$$\begin{aligned} \epsilon_{\text{Ext}} &\geq \text{frk}(\lambda) \geq \text{acc}(\lambda) \cdot \left(\frac{\text{acc}(\lambda)}{q} - \frac{1}{2^\lambda} \right) \\ &\geq \left(\epsilon - \frac{1}{2^\lambda} \right) \cdot \left(\frac{\epsilon}{q} - \frac{1}{2^\lambda} \right). \end{aligned}$$

As q is polynomially bounded in λ and $\kappa = \frac{1}{2^\lambda}$, it follows that GASDVS satisfies non-delegatability. $\square$